claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\agent-a41d134f20a2a7d97.jsonl`
- SHA-256: `d66a8731221a5c377f8c87b178a45939e90d11592e50a02c4b473cd8dcd9cc06`
- Source modified: `2026-06-16T03:13:38+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`

Transcript

1. user (2026-06-16T03:12:12.068Z)

Survey the rally-detection QUALITY / HARNESS / strategy design docs in this repo (cwd = the sports highlight indexer) so a new foundational research doc can build ON them and cross-link, not duplicate.

Read and summarize each (skip gracefully if absent): docs/README.md, docs/CODE_MAP.md, docs/EXPERIMENT_HARNESS.md, docs/MULTI_SIGNAL_FUSION_PLAN.md, docs/ANALYZERS_AND_RECONCILERS.md, docs/HOW_RALLY_DETECTION_WORKS.md, docs/QUALITY_ITERATIONS.md, docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md, docs/NEXT_STEPS.md, docs/ROORA_LABELING_GUIDELINES.md, docs/PERSONALIZATION_PLAN.md, and anything under docs/archives/quality-iterations/.

For EACH doc report: (1) what it covers, (2) its STATUS (designed-only / shipped / owner-gated / not-started ? look for explicit status markers), (3) the key decisions/claims/numbers.

Then answer precisely:
- **CRUCIAL:** What did ANALYZERS_AND_RECONCILERS.md §13 (the multi-agent literature review) already cover? List the specific patterns surveyed and the verdicts (e.g. "candidate-windows+scorer = TAL", "reconciler = TAS post-proc", "signal-not-verdict = late/score-level fusion", which were rated novel vs standard prior art). A new research effort must go BEYOND this ? so capture exactly what's already surveyed + cited.
- What are the prior MEASURED results recorded (e.g. Gate-A ?F1 +0.074 6/6 p=0.031; audio +0.079; person ?0.021; the learned-variant/threshold-tuning numbers; shuttle-only F1?0.307; LOVO 0.626)? Capture the numbers + their context.
- What is the stated eval approach (LOVO/LOGO, IoU threshold, honest stats), the eval?serve lesson, and the "signals?learned-scorer, NO special-casing" spine directive?
- What does the owned-model plan say about the base model + licensing constraints (Qwen3-VL/InternVL3; MIT/Apache/BSD only; paid LLMs inference-only never training data; exclude minors)?

Return structured markdown, distilled, with doc + section anchors. NO full dumps.

2. assistant (2026-06-16T03:12:16.569Z)

I'll systematically survey the documentation in the sports highlight indexer repo to extract the key design decisions, measured results, and strategic direction.

3. user (2026-06-16T03:12:16.717Z)

docs\GOLDEN_SET_IMPLEMENTATION.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\PMF_MARKET_RESEARCH.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\HOSTING_PLAN.md
docs\UI_PROPOSAL.md
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\FIELD_VISIT_ANSWER_SHEET.md

```
docs\FIELD_VISIT_PLAYBOOK.md
docs\I18N_PLAN.md
docs\PMF_FIELD_SIGNALS.md
docs\PRODUCT_FIELD_ASSOCIATE_FLYER.md
docs\VIDEO_LOCALITY_MODEL.md
docs\GOLDEN_SET_VENDORING.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\STORAGE_SHARING_MODEL.md
docs\PER_VIDEO_OUTPUT_LAYOUT.md
docs\QUALITY_ITERATIONS.md
docs\ABLATION_LAYER.md
docs\ANALYZERS_AND_RECONCILERS.md
docs\BACKLOG.md
docs\CODE_AUDIT_AND_TEST_HARDENING.md
docs\COMMERCIALIZATION.md
docs\DEFERRED_HARDENING_PLAN.md
docs\EVALUATION.md
docs\EXECUTION_POLICY.md
docs\EXPERIMENT_HARNESS.md
docs\GOLDEN_DATA_SHARING.md
docs\HARDENING_LOOP_HANDOFF.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\NEXT_STEPS.md
docs\PERSONALIZATION_PLAN.md
docs\PLATFORM_ARCHITECTURE.md
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\README.md
docs\ROADMAP.md
docs\ROORA_LABELING_GUIDELINES.md
docs\TRACKNET_WSL_SETUP.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\KHELSUTRA_PRODUCT_OVERVIEW.md
docs\CODE_MAP.md
```

4. user (2026-06-16T03:12:16.805Z)

```
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
docs\archives\checkpoints\2026-06-eval-baseline\README.md
docs\archives\checkpoints\2026-06-logging-reporting\README.md
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
docs\archives\past_projects\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\decisions\DECISIONS.md
docs\archives\TEST_RUN_SUMMARY.md
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
docs\archives\quality-iterations\README.md
docs\archives\checkpoints\2026-06-14-hardening-loop-complete.md
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
docs\archives\README.md
```

5. assistant (2026-06-16T03:12:19.744Z)

```
Now I'll read the key docs you specified:
```

6. user (2026-06-16T03:12:20.089Z)

1 - ROORA_LABELING_GUIDELINES.md ? Latest (v8) vendor labeling guidelines mirrored from the collector's `docs/annotation/`, with the fusion-driven court-calibration / singles-doubles additions marked `? ADDED (fusion)`.
2 - EVALUATION.md ? How we measure and improve quality.
3 - EXPERIMENT_HARNESS.md ? Design (P1 SHIPPED) for A/B + shadow + SxS experimentation on rally-detection variants with **zero production regression** ? experiments are decoupled from deploy (new cues default OFF, promote only on measured LOVO lift, rollback = flip a flag). Composes the existing `backend/eval/` machinery (`calibration`, `regression`, `metrics`); implemented in `backend/eval/experiment.py`.
4 - QUALITY_ITERATIONS.md ? **Living log** of rally-detection quality iterations (hypothesis ? what shipped ? measured/expected effect ? lesson). The reverse-chronological record; terminal-state snapshots get archived under `archives/quality-iterations/`.
5 - ANALYZERS_AND_RECONCILERS.md ? Design (owner-gated, NOT started): a **signal-fusion framework** for rally detection ? producers emit confidence-scored **signals** at three temporal **scopes** (frame detectors / window analyzers / session analyzers), a learned **scorer** weights them, and an auditable **report-first reconciliation** pass lints the decision against the evidence. The spine: **no special-casing in the decision path** (signals?scorer, never `if/else`). Worked examples: a service-fault analyzer, a warm-up/practice span detector, and consuming a per-frame shuttle-state detector. A peer to `EXPERIMENT_HARNESS.md` and `MULTI_SIGNAL_FUSION_PLAN.md` (ADR-007 modular rally layer).
6 - PERSONALIZATION_PLAN.md ? Design (owner-adopted **hybrid**; Phase-0 landing) for the **per-user personalization feature on a generalization-first baseline**: a thin per-user *tuning* layer gated by the honest small-N statistics (a **min-N promotion floor** + a **structural-guard exemption**, never a service gate) + a **contribution flywheel** (free-tier videos pool into the owner-trained baseline; *"a tool that gets better the more you help it"*). Records the **locked owner decisions** (identity, storage, `min_n` semantics, Gate-A default, free-tier pooling). Built so far: per-user promotion gate (#141) + held-out-USER learning curve (#142).
7
8 ## Practical / ongoing
9 - TRACKNET_WSL_SETUP.md
10 - Golden set docs (`GOLDEN_SET_*`)
11 - Evaluation harness and calibration drivers (in `backend/eval/`)
12 - GOLDEN_DATA_SHARING.md ? How the GPU box and CPU dev/agent sessions share golden artifacts (trajectory CSVs, `*_golden_features.json`) via a OneDrive folder; the GPU-extract ? CPU-litmus split. See **#175**.
13
14 ## Historical documents
15 Completed plans, old research, ADRs, and checkpoints live in [`archives/`](archives/).
16
17 - CODE_AUDIT_AND_TEST_HARDENING.md ? **COMPLETED** (2026-06-14; T1?T5 + full loop + verification per #162). Living record of the June 2026 plumbing / contract / architectural audit, 626-test infrastructure, positive test plans for the 5 identified issues, multi-layer regression protocol (including ablation/experiment layer), and pre/post-change verification checklist. See `docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md` and `docs/archives/checkpoints/2026-06-14-hardening-loop-complete.md` for the final archived state. (Top-level is now pointer stub per review; full content in archives/.)
18
19 See `archives/README.md` for the umbrella policy and `archives/past_projects/README.md` for classification of small tech debt / starter projects.

7. user (2026-06-16T03:12:20.515Z)

1 # Experiment harness ? A/B + shadow eval for rally-detection quality (no-regression by design)
2
3 **Status:** **P1 IMPLEMENTED** (`backend/eval/experiment.py` + `tests/test_experiment.py`, shipped
4 2026-06-13) ? the `Variant` layer, `compare()` (labeled A/B, LOVO-honest), `compare_unlabeled()`
5 (SxS on NEW unlabeled footage), and the leaderboard. **P2?P4 remain owner-gated.**

Phases below are sequenced but each is a separate PR.

> **Companion docs:** the cues this experiments *on* are in MULTI_SIGNAL_FUSION_PLAN.md; the scoring mechanics are EVALUATION.md; the no-regression gate + golden labels + `baseline.json` come from GOLDEN_SET_VENDORING.md; the modular-cue architecture is **ADR-007** (archives/decisions/DECISIONS.md); the distilled classifier is **ADR-004**. The 2-layer mental model is HOW_RALLY_DETECTION_WORKS.md.

---

1. Context ? why this, why now

PR #112 (merged) added the multi-signal fusion design ? shuttle + person + audio cues, all **default OFF**. The owner's directive for *how* we evolve quality from here:

> *"Evolve the codebase so we can keep finding new ways to improve quality and experiment with them ? or add them as a signal ? without suffering any regressions if the idea doesn't work out. Rolling back the code would be too dangerous."*

So **experimentation and deployment must be decoupled.** The contract:

1. Merge experimental code freely ? it's **gated OFF**, so it cannot change production behavior.
2. Test it **offline against golden labels** (most experiments never touch the served path at all).
3. Promote a variant to default **only on measured lift**, through a CI eval gate.
4. **"Rollback" = flip a config flag, never `git revert`.** The proven path is a pinned variant that is always present.

The pleasant surprise (Explore audit, 2026-06-13): ~80% of this harness already exists and is simply not composed *as a system*. The missing piece is a thin **variant/experiment layer + the discipline**, not a platform. §4 is the exact code surface so the next session does not re-discover it.

---

2. The thesis ? separate expensive DETECTION from cheap DECISION

The expensive, GPU/WSL-bound, risky part is **detection**: "what signals exist per frame" (shuttle WASB trajectory, person tracks/boxes, audio hits). The cheap, swappable, pure-Python part is the **decision**: "given those signals, where are the rallies" (windowing + gates + the classifier).

> **Run detection once per video, persist all per-cue signals + per-candidate features, then re-score any number of variants OFFLINE ? deterministically, with no GPU and zero production risk.**

This is not aspirational ? `distill_local.py` and `calibrate_local.py` already do exactly this on stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence

substrate.
? Most experiments are pure re-scoring of stored data; they never enter the served pipeline.

---

3. What already exists (the audit ? do NOT rebuild)

Exact public surface, with `file:line`. **This table is the anti-duplicate-work artifact** ? start here.

3.1 Scoring (variant-agnostic ? grades any `List[Interval]`)
`backend/eval/metrics.py`
- `interval_iou(a, b) -> float` (:15) ? temporal IoU of two `(start,end)` intervals.
- `match_intervals(preds, gts, iou_threshold=0.5) -> MatchResult` (:48) ? greedy 1-to-1 IoU matching.
- `score(preds, gts, iou_threshold=0.5, match_result=None) -> Score` (:104) ? P/R/F1, mean IoU, boundary errors; `Score.as_dict()`.
- `pr_curve(preds, gts, thresholds=None) -> List[Score]` (:138).

3.2 A/B + cross-validation primitives (the comparison engine already exists)
`backend/eval/calibration.py`
- `improvement_report(predict_fn, baseline_config, tuned_config, gts, iou_threshold=0.5) -> dict` (:148) ? **before/after metrics + %?. This IS the A/B delta.**
- `leave_one_video_out(videos, configs, metric="f1", iou_threshold=0.5) -> dict` (:113) ? **LOVO: pick on other videos, score on held-out video (honest cross-video).**
- `cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold=0.5) -> dict` (:72) ? within-video k-fold overfit guard.
- `select_best(predict_fn, configs, gts, metric="f1", iou_threshold=0.5) -> (Config, float)` (:61).
- `evaluate(preds, gts, iou_threshold=0.5) -> Score` (:41); `kfold_indices(n, k)` (:46).
- LOVO video dict shape: `{"name": str, "predict_fn": PredictFn, "gts": [Interval]}`.

3.3 No-regression gate + baselines (the promotion gate already exists)
`backend/eval/regression.py`
- `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None, tolerance=DEFAULT_TOLERANCE, baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ? score stored preds vs GT, flag if P **or** R drops > tolerance.
- `regress_with_provider(...)` (:160) ? same, fetching GT via the annotation provider.
- `save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint=None)` (:80) · `load_baselines(path) -> dict` (:68).
- `RegressionResult` (:43): `regressed`, `reasons`, `gt_hash`, baseline P/R, `tolerance`, `has_baseline`; `as_dict()`.
- Default baseline file: `eval_baselines/regression_baseline.json` (:33).

`run_regression.py` ? CI-gatable CLI, `main(argv)` (:52). **Exit codes: 0 ok / 1 regression / 2 no-DB / 3 no-baseline.** Flags: `--video-id --tier --annotations --stage{candidates,final} --detector --iou --tolerance --baseline --update-baseline --allow-missing-baseline --json`.

3.4 Pinnable model artifact
`backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2): `fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75), `predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON) (:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**

3.5 Offline re-scoring substrate (persist once, re-score forever)
`backend/infrastructure/database.py`
- `add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=None, confidence=None, stats=None, detection_params=None) -> Optional[int]` (:318) ? `stats`/`detection_params` stored as JSON; `candidate_segments` table cols incl. `stats`, `detection_params`, `human_label`, `confirmed_segment_id` (schema :97).
- `query_candidate_segments(video_id, detector=None, only_unlabeled=False) -> List[dict]` (:355) ? returns rows with deserialized `stats`.

3.6 Label assignment + feature glue
`backend/eval/training.py`
- `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]` (:50) ? y-labels by IoU match to golden (same matcher as the evaluator).
- `build_training_set(candidate_rows, gt_intervals, iou_threshold=0.5, feature_fn=extract_yolo_features) -> (X, y, intervals)` (:64) ? pluggable `feature_fn`.

3.7 Existing LOVO drivers to mirror (not duplicate)
- `backend/eval/distill_local.py` ? `run(specs, iou=0.5, threshold=0.5, model_out=None)` (:98), CLI `python -m backend.eval.distill_local --manifest videos.json --model-out ...`; `FEATURE_NAMES = [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]` (:40); already does LOVO over Gemini silver labels.
- `backend/eval/calibrate_local.py` ? `run(specs, iou=0.5, metric="f1")` (:77); `LocalConfig = (velocity_thresh, merge_gap, min_window_duration, min_crossings)`; grid + LOVO.
- `backend/eval/calibrate_wasb.py` ? `run(...)`, `load_golden_gts()`; `backend/eval/gt_loader.py::load_gt_intervals(csv_path, *, strict=True)` is THE canonical golden reader.

3.8 Registries + config knobs (variant selection)
- `backend/pipeline/segmenters/base.py` ? `SegmenterRegistry` (:35), singleton `segmenter_registry` (:63); registered (6): `motion`, `gemini`, `yolo_hybrid`, `tracknet_hybrid`, `wasb_hybrid`, `fusion_hybrid`.
- `backend/annotations/base.py` ? `AnnotationProviderRegistry`, singleton `annotation_registry`; providers `file`, `auto`.
- `backend/config/models.py::IndexingConfig` ? `default_segmenter` (:65, default `"yolo_hybrid"`), `detector_model` (:72), `detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75), `min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). The `min_crossings` knob now lives on `FusionConfig` (default `0` = OFF; also `min_players`, both served Gate A/B thresholds ? see fusion P2).

3.9 Confirmed ABSENT (no duplication risk)
Explore found **no** existing experiment / variant / shadow / A-B *machinery* ? only a stray "A/B test" aspiration comment in `backend/pipeline/detectors/base.py` and an "experiment E1" note in `archives/research/AUDIO_RALLY_DETECTION_PLAN.md`. The `regress()` + baseline path is the canonical comparison mechanism.

---

4. The variant abstraction (the one thin thing to add)

A **`Variant` = a declarative recipe**, not a code branch:

```
Variant = (segmenter_name, IndexingConfig overrides, enabled cues + per-cue params, classifier model path/hash)
```

JSON-serializable; selected via the existing `segmenter_registry` + `IndexingConfig.default_segmenter`.
Control = a *pinned, frozen* variant (recipe + model hash) that is always registered and is the default. Adding a new cue = a new Variant differing by one flag ? the control recipe is untouched.

---

5. Three experiment modes

5.1 Offline A/B (primary, label-driven, zero production risk)
`compare(videos, variant_A, variant_B)` ? a thin driver composing **existing** functions:
`query_candidate_segments()` (load persisted features) ? re-score each variant ?

`calibration.improvement_report()` + `calibration.leave_one_video_out()` + `metrics.score()`, graded vs golden (`gt_loader.load_gt_intervals`). Reports per-video **and** LOVO-honest aggregate IoU/P/R/F1 deltas. **This is the A/B test, and it needs almost no new scoring code** ? only the glue.

Honest reporting (default ON, additive): `compare(..., honest_stats=True)` also attaches a `"honest"` block ? per-metric bootstrap **CIs**, a paired **?F1 CI + exact sign test** (C1), and **F1@k + segment-count ratio** (C4) ? *alongside* the bare-mean fields (existing output unchanged; `honest_stats=False` restores the legacy shape). A bare LOVO mean at n?6 is not promotable on its own; the lift is real when the ?F1 CI clears 0 **and** the sign test agrees. Wiring of the course corrections in ANALYZERS_AND_RECONCILERS.md §13/C1+C4 (helpers: `backend/eval/eval_stats.py`, `segmentation_metrics.py`); `record_experiment` carries the honest ?F1 into the leaderboard.

5.2 Shadow (for cues that touch detection, e.g. the person detector)
The new cue runs and **persists its signal** into the candidate `stats`/`detection_params`, but the *served* decision stays control. Offline you then ask "what would variant B have scored?" via §5.1 ? B never affects output until it wins. This is how a detection-touching change (not just a re-scoring change) earns its way in without risk.

5.3 Promotion gate (no-regression, CI-enforced)
`regression.regress()` + `baseline.json` + `run_regression.py` exit codes. A variant becomes the new default **only if** it doesn't regress the golden set beyond `tolerance` (exit 1 blocks). Control is pinned ? **rollback = flip `default_segmenter`/variant config, never a code revert.**

---

6. No-regression guarantees (the heart of the owner's ask)

- **New cues/signals default OFF** ? per-cue enable flag in `IndexingConfig`; merged code can't change behavior until explicitly enabled.
- **Control variant pinned** ? frozen recipe + classifier model hash, always registered & default.
- **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no silent default change.
- **Experiments are records** ? variant-spec hash ? per-video + LOVO scores + label-set hash + timestamp, persisted to a small JSON **experiment leaderboard** (generalizes `baseline.json`). Answers "which signals ever helped, on which video slices," and makes every result reproducible.
- **Decoupled events** ? "merge an experiment" and "change the default" are separate, independently revertible actions.

---

7. What to build later (gap list ? thin, additive, owner-gated)

1. ~~**`backend/eval/experiment.py`** ? a `Variant` dataclass + `compare(videos, a, b)` composing the §3 functions (no new scoring math).~~ **DONE (2026-06-13).** Shipped `Variant` (+ pinned `CONTROL`),

`compare` (labeled A/B with LOVO-honest learned-variant training, mirroring `distill_local`), `compare_unlabeled` (the **SxS-on-new-videos** mode ? agreed / A-only / B-only via `match_intervals`, no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), and `load_video_candidates` (the `query_candidate_segments` bridge). JSON variant recipes first as recommended; a `VariantRegistry` only if the count grows. 19 tests incl. the no-regression invariant.
2. **Persist per-cue features** into candidate `stats` (person: `players_in_court`, `two_sided_motion`, `in_court_ratio`, `shuttle_player_colocation`; shuttle-state: `net_crossings`, `direction_reversals`; audio: hit times) so any future variant re-scores offline ? ties to `MULTI_SIGNAL_FUSION_PLAN.md` §3.2/§4.
3. **Per-cue enable flags** in `IndexingConfig` (default `False`); `default_segmenter` already exists. Add the `min_crossings` knob (fusion P2) here too.
4. **CI promotion lane** using `run_regression.py` against the golden `baseline.json`; write the experiment-leaderboard JSON.

---

8. Phasing (owner-gated; ONE PR per slice, off `master`)

- **P0 ? this doc**, indexed in `docs/README.md`, cross-linked from `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`. *(this deliverable; docs-only, no code.)*
- **P1 ? `experiment.py` `compare()`** ? **DONE (2026-06-13):** thin wrapper over `calibration` + `metrics` + `query_candidate_segments`; pure-logic, fully tested in this dev container (`compare`, `compare_unlabeled`, `record_experiment`, the no-regression invariant). `compare_unlabeled` adds the SxS read on **new unlabeled footage** the owner asked for (run control + candidate on a fresh video, diff where they agree/diverge ? two reels via the existing `VideoCompiler` is the GPU-side follow-on).
- **P2 ? per-cue enable flags + feature persistence** into candidate `stats` (default OFF) ? couples with fusion P1/P2.
- **P3 ? CI promotion gate**: `run_regression.py` lane on the golden `baseline.json`; experiment-leaderboard JSON.

---

9. Scope guardrail

Single-tenant / local-first (per `PLATFORM_ARCHITECTURE.md`): a JSON experiment registry + the existing harness ? explicitly **NOT** MLflow / W&B / a service. Keep it the size of `baseline.json`.

---

10. Verification

- **Offline, runnable in this dev container (no GPU/torch needed for the glue):** unit test that `compare()` is deterministic on synthetic candidate rows; a **no-regression invariant test** asserting "all new-cue flags OFF ? predictions byte-identical to control"; a CI-gate test asserting `run_regression.py` exit codes (0 ok / 1 regress / 3 no-baseline).

220 - **Machine-side (owner, GPU/WSL):** persist candidates once on a real clip, run `compare(control,
221 fusion)` LOVO, confirm the reported lift matches a manual `metrics.score()` spot-check.
222 - **Docs (P0):** links resolve; indexed in `docs/README.md`; cross-linked from
223 `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`.
224 </content>
225

8. user (2026-06-16T03:12:20.724Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-151 of 379 total (37029 tokens, cap 25000). Call Read with offset=152 limit=151 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

1 # CODE_MAP.md
2
3 **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5 **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).
6
7 ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via `segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec` defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**; `match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked `eval_baselines/` and **exits 3 on a missing baseline**; the API validates `video_path`/`output_filename`. New tests: `test_api_endpoints`, `test_compiler`, `test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`, `test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`, `test_import_guard`, `test_cvat_robustness`).
8
9 **LEGEND**
10 - `@path` = file (always repo-relative)
11 - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
12 - `->` = dataflow / call / "produces"
13 - `§` = data contract (canonical shape)
14 - `?` = gotcha / footgun
15 - `?` = registry / plugin extension point
16 - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
17
18 ---
19
20 ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
21
22 Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an existing
23 **registry / extension point** (`?`) over a new top-level file.
24
25 | You're adding? | Put it in? | Notes |
26 |---|---|---|
27 | Backend / runtime code | `backend/<subsystem>/` | `pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`, `infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most additions are **register a class in the relevant `?` registry**, not a new module tree. |
28 | **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) | `backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver. These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not

````
scripts; do NOT move them out. |
| **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app code) | **`scripts/`** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
| **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) | `tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g. `tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
| Training code (model fitting) | `training/` | Behind the CI-enforced import wall (ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
| A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70% coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
| A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only **terminal-state** (DONE/REJECTED) work moves to `docs/archives/`. |
| A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from ruff/pytest. Never imported by app code. |
| A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008: **manifest committed** (provenance), **weights gitignored**. |
| Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean checkout); the no-regression gate reads it. |
| Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.** Per-video isolation is planned in `PER_VIDEO_OUTPUT_LAYOUT.md`. |

**Canonical files that stay at the repo ROOT** (referenced by CI / packaging / docs ? do not move):
`pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) · `run_all_tests.py` (CI: `ci.yml`) ·
`run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) · `run_eval.py`
(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py` moved to
`scripts/doctor.py` (it shadowed the build system); invoke via `python scripts/doctor.py` or `indexer --setup`.
New standalone run-drivers go in **`scripts/`**, not the root.

---

## 1. PIPELINE

### 1A. Runtime: video file -> highlight reel
```
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-defaults, never fatal)
                                                  @backend/config/loader.py::load_config -> @backend/config/models.py::AppConfig
  -> video_id = MD5(whole file)           @backend/utils/hashing.py::compute_video_id (single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint copies)
  -> validate @backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
       FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity -> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see @backend/pipeline/validators/__init__.py)
       [validation FAIL -> persist status="failed", return early; report STILL emitted in finally]
  -> db.add_video(video_id, filepath, status, [r.dict()]) @backend/infrastructure/database.py
  -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
  -> seg = segmenter_registry.get(seg_name)(db, global_config)   ? @backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
  -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
       [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without stitching]
  -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs
````
````

```
? even on validation failure / exception; NEVER raises; grafts response["reports"])
  -> select segment ids -> [(start,end)] (+ stitching padding)
  -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name) @backend/pipeline/stitcher/compiler.py
       per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy -> output/<name>.mp4
```
Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
```
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
 -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted via db.add_candidate_segment]
 -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip, prompt)   ? provider_registry
 -> P4 db.add_play_segment + db.link_candidate_to_segment
```
Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) -> add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

1B. WASB shuttle substrate (P2 of wasb_hybrid)
```
WasbHybridSegmenter.process_video @backend/pipeline/segmenters/wasb_hybrid.py
 -> WasbRunner.healthcheck()  (gate; not ok -> return []) @backend/pipeline/detectors/wasb_runner.py
 -> WasbRunner.run_predict(video, out_dir)
      stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python wasb_infer.py ...'`
      -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
 -> TrackNetRunner.parse_trajectory_csv(csv) @backend/pipeline/detectors/tracknet_runner.py  (shared, re-exported by WasbRunner)
 -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start, velocity_thresh, merge_gap, min_window_duration, chunk_index)  [the model-agnostic WINDOWING BRAIN]
```
(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback (30.0, 1280.0).)

1C. Calibration / eval flow (NO pipeline run unless told)
```
GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage ? {candidates, final}
  -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
  -> harness.evaluate -> EvalReport -> report_to_dict
       -> batch.aggregate (corpus micro/macro)            @backend/eval/batch.py
       -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py [CI gate]
Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate}  @backend/eval/calibration.py
Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned classifier)  @backend/eval/{calibrate_local,distill_local}.py
Learned segmenter (offline): training.build_training_set(X,y) -> classifier.LogisticRegression.fit/save
Gemini refinement driver (tuning, standalone): gemini_refine.{refine_window,full_video_rallies}  @backend/eval/gemini_refine.py
Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run @backend/eval/audio/e1_probe.py
```
Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py`

(manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression), `@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
105
106 ---
107
108 ## 2. SUBSYSTEMS
109
110 ### 2.0 Orchestration & config
111 - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
112   - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
113   - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx (path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 + job_id**. `::_run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches + logs traceback). `::_execute_processing` = the former sync body (hash -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in `result["status"]`. `::_get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
114   - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `::download_highlight` `@GET /api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
115   - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.
116   - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
117   - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
118 - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
119 - `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed. `::_default_db_path` resolves via `backend.config.load_config` (`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
120 - `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as

label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig` (same object the live pipeline feeds segmenters).
- `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run); `::_default_db_path` resolves via `backend.config.load_config`.

- `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`, `StitchingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 7. Use `from backend.config import load_config, AppConfig`.
- `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
  - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching` (`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
  - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts** (`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any section (sections discovered dynamically from `model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to `.get`.
  - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested `tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ? `::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ? a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
  - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_fps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`); `::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`). `::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`, `conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`). `::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`, `output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`). `::StitchingConfig` (`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal of 5 sports. `::Config = AppConfig` alias.
- `@backend/config/loader.py` ? load/save with defaults-merge.
  - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root). `::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
  - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole defaults dict for that section (Pydantic then re-applies field defaults for missing leaves). Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-fatal).
  - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by scripts/doctor.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
- `@config.json` ? on-disk config; mirrors `AppConfig`. ? **`indexing.default_segmenter='gemini'`** diverges from model default `'yolo_hybrid'` ? file wins at runtime.
- `@scripts/doctor.py` (formerly root `setup.py`) ? `::init_default_config` ? delegates to `backend.config.save_default_config` (single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA). Run via `python scripts/doctor.py` or `indexer --setup`.

2.1 Segmenters (under pipeline/segmenters) ?
- `@backend/pipeline/segmenters/base.py`
  - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
  - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new

segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is masked).
140   - `@backend/pipeline/segmenters/__init__.py` ? imports all 6 (`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid,fusion_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.
141   - `@backend/pipeline/segmenters/trajectory_hybrid.py::TrajectoryHybridSegmenter` ? SHARED ENGINE for the trajectory hybrids (2026-06-11 collapse of the copy-paste twins; the "fix BOTH" footgun is retired): healthcheck gate -> P2 trajectory->windows (+persist) -> P3 provider handoff -> P4 DB/link; single `::_shot_count_from` (old wasb/tracknet shot_count drift gone). NOT registered itself. ? Subclass contract: `family` (config section `indexing.<family>.velocity_threshold(0.02)`, `output/<family>` trajectory dir, `<family>-hybrid-<model>` metadata tag), `display_label` (log lines), `name`/`description` properties, `_build_runner(idx_cfg) -> (DetectorRunner, detection_params extras)`. **Two fusion seams (IDENTITY by default ? twins byte-identical, pinned by the equivalence test): `::_enrich_candidate_stats(cw,points,fps,frame_width,video_path,idx_cfg)`** (base = the 4 motion-key stats dict; computed once into `window_stats`, used for persist + the handoff gate) **and `::_select_for_handoff(windows,window_stats,idx_cfg)`** (base = all indices). ? shared knobs still YOLO-named (`log_yolo_candidates/store_yolo_candidates`); whole-video single pass; needs `{PROVIDER}_API_KEY` (missing -> []). Equivalence pinned by `tests/test_trajectory_hybrid_equivalence.py` (detector identity, shot_count, offsets, + the base-hook-identity tests ? parametrized over both twins).
142   - `@backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (name `'fusion_hybrid'`, **default-OFF** multi-signal fusion P2) ? subclasses the engine; `family='fusion'`, `display_label='Fusion'`. `::_build_runner` selects the substrate via `indexing.fusion.substrate` ('wasb'|'tracknet') reusing the existing runner+config. `::_enrich_candidate_stats` overrides to ALWAYS add `net_crossings` (Gate A signal, GPU-free via `rally_gate.gate_windows(..., estimate=self._axis_estimate(points))` ? axis/net cached per trajectory) and, only when `fusion.cue_flags.person`, the ~5 person features (`::_person_features` ? lazy `PersonDetector.detections_per_frame` over the window's ORIGINAL-video frame range ? `fusion_features.compute_person_features`). `::_select_for_handoff` overrides to apply served Gate A (`fusion.min_crossings`) ? Gate B (`fusion.min_players`, person-only) ? default 0/0 ? all windows kept. ? persisted candidates are ALWAYS the full superset (offline substrate intact); only the handoff/play_segments are gated. ? to A/B vs a tuned wasb run, set `fusion.velocity_threshold = wasb.velocity_threshold` (the engine reads `idx_cfg['fusion'].velocity_threshold`). ? person features eager even for gated windows (P3 cleanup). Tests: `tests/test_fusion_hybrid.py`.
143   - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection_params` extras = `weights_path` only.
144   - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `'tracknet_hybrid'`) ? thin subclass: `family='tracknet'`, wires `TrackNetRunner`/`TrackNetConfig`; extras = `weights_path` + `queue_length`.
145   - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO remains; ADR-005). **Stage-1 detection EXTRACTED to `detectors/person_detector.py` (fusion P1, no behavior change)** ? the segmenter now ORCHESTRATES only (chunking, pipelined-vs-sequential P2 prefetch where ffmpeg overlaps sequential GPU, AI-provider handoff, candidate persist/link) and delegates detection to `self.person` (`PersonDetector`). Imports `_DEFAULT_DETECTOR` from person_detector for the `detection_params` snapshot. ? `yolo_tracker`/`bytetrack.yaml` is dead/back-compat; `gemini_*` config keys are deprecated aliases still honored; still mypy-quarantined for pre-existing orchestration debt (the torchvision code that caused it left).
146   - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. Chunks are RE-ENCODED + downscaled to `indexing.ai_chunk_scale_width` (default 1280; 0 = legacy stream-copy) before upload ? stream-copied 4K chunks exceed the provider's `MAX_UPLOAD_MB` and every chunk gets refused ("0 segments / success"); re-encode also fixes keyframe drift at chunk boundaries. Oversize refusals (provider metrics `oversize_skipped`) are aggregated -> `logger.error` + `reporting.run_signals` (`oversize_clips_skipped`) for the report rule. Workers from `indexing.ai_max_workers`. ? chunks only if `duration>chunk_duration`; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning

table).
- `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.

2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
- `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).
- `@backend/pipeline/detectors/tracknet_runner.py`

9. user (2026-06-16T03:12:21.009Z)

Multi-signal rally fusion ? add a human (person) cue alongside the shuttle

Status: **P1 DONE** (2026-06-13) ? the person cue is extracted to
`backend/pipeline/detectors/person_detector.py` (`PersonDetector`), consumed by `yolo_hybrid` with no
behavior change. **P2?P4 remain owner-gated.** Phases below are sequenced but each is a separate PR.
Owner decisions captured in [§9](#9-open-decisions-for-the-owner).

> **Companion docs:** the 2-layer mental model is HOW_RALLY_DETECTION_WORKS.md;
> the modular-fusion architecture this builds on is **ADR-007** (archives/decisions/DECISIONS.md);
> the permissive-dependency rule is **ADR-002 / ADR-005**; the audio cue (the #1 next cue, already
> scaffolded) is archives/research/AUDIO_RALLY_DETECTION_PLAN.md.
> The vendor labels that gate the learned path come from GOLDEN_SET_VENDORING.md
> and the labeling spec in ROORA_LABELING_GUIDELINES.md.
> **How each cue here is added/tested without production regression** ? A/B, shadow eval, and the
> promote-only-on-lift discipline ? is EXPERIMENT_HARNESS.md (every P1?P4 cue
> below lands flag-gated, default OFF, promoted only on measured LOVO lift).

---

1. Context ? why this, why now

Rally detection today runs **one cue at a time**. Each segmenter independently turns a single signal
into rally windows:

Segmenter	Signal	File
`wasb_hybrid` / `tracknet_hybrid`	shuttle velocity (frozen WASB/TrackNet trajectory)	`backend/pipeline/detectors/tracknet_runner.py`
`yolo_hybrid` (the Pydantic `default_segmenter` fallback; `config.json` overrides it to `gemini`)	person velocity (torchvision + ByteTrack)	`backend/pipeline/segmenters/yolo_hybrid.py`
`motion`	raw pixel motion	`backend/pipeline/segmenters/motion.py`
`gemini`	a cloud VLM watching the whole clip	`backend/providers/gemini.py`

There is no fusion. The owner's directive: make the detector **use the shuttle *and* the humans
together** ? and be ready to fold in more signals (audio, later pose) ? delivered
MIT / Apache-2.0 / BSD-clean so we can ramp commercially without a licensing

rethink.

This is **not greenfield** ? it operationalizes work the repo already blessed:

- **`HOW_RALLY_DETECTION_WORKS.md` open question:** *"Fusion: combine shuttle-motion (WASB) with player-motion for static cams."*
- **ADR-007** already designed a **modular "rally layer"** that fuses cues *outside* the frozen WASB model, with **audio as the #1 next cue and person/pose parked as #2**. This plan picks up the parked person cue and generalizes the fusion seam so future cues are "register a producer," not "re-touch the segmenter."
- **`backend/pipeline/detectors/rally_gate.py` already exists** ? the net-crossing gate, explicitly labelled in its own docstring as *"Gate A in the over-fire fix spectrum (**B = player-stance state machine, future**)."* The person cue in this plan **is that named-but-unbuilt Gate B.**

The owner's "held-shuttle" false positive ? what's actually going on

The owner observed a "rally" that was just a player **holding / flicking the shuttle in hand** between points. This is the *exact* failure `rally_gate.py` was written to kill:

> *"WASB over-detects: when a player flicks/tosses the shuttle back to the opponent for service after
> losing a point, that single trajectory looks like a rally. The discriminator is structural: a real
> rally has the shuttle crossing the net MULTIPLE times; a single service feed crosses ~once."*

The structural fix already exists and is validated offline ? but it is NOT wired into the production runtime path. `rally_gate.apply_gate(..., min_crossings=2)` is called only from the **eval drivers** (`eval_partial_labels.py`, `export_wasb_segments.py`, `calibrate_local.py`; `distill_local.py` uses `rally_gate.gate_windows` to compute the `net_crossings` feature, not `apply_gate`); the live segmenters historically didn't apply it. P2 closed this: `FusionConfig.min_crossings` now exists (default `0` = OFF), so the served Gate A is a config flag ? consistent with §10 P2 below, marked DONE. So the single cheapest, highest-confidence win in this whole plan is **P2's first step: fold the existing Gate A into production.** Everything else (the person cue / Gate B, the learned fusion) is robustness on top of that.

Why humans help ? and the honest risk

The beachhead videos are **static tripod** (Bangalore academies, PMF_MARKET_RESEARCH.md), so player presence/motion is a usable cue. ADR-001's warning was specifically about **broadcast pan** cameras, where player-motion fooled the old heuristic ? *that failure mode does not apply to a fixed court cam.* But we keep **the shuttle as the camera-invariant primary** so the general bet stays robust on other camera types. Humans then add:

- **Precision** ? reject shuttle-on-floor / warm-up knock-ups / held-shuttle feeds when

the court isn't occupied by ?2 players in a play posture.
- **Recall** (via the learned classifier) ? features that help bridge shuttle occlusion/blur gaps.

---

2. Core architecture ? one rally-fusion layer, cues are swappable producers

Formalize ADR-007's diagram. Every cue is a **black box that emits a time-aligned signal** (per-frame activity and/or per-window features); **fusion happens in *our* layer, never inside a frozen model.**

```
frames ?? WASB (frozen, MIT)      ?? shuttle trajectory ???
frames ?? person detector (ours)  ?? tracks + boxes ???????
audio  ?? hit detector (ours)     ?? hit events ????????????? RALLY LAYER (ours) ?? rallies
[frames ?? pose/stance (ours; parked #3) ?? stance ????????   feature + decision fusion
```

- **Shuttle = primary, camera-invariant.** Windows still **seed** from `TrackNetRunner.trajectory_to_action_windows()` (`tracknet_runner.py:234`). Other cues *enrich and adjudicate* shuttle-seeded windows; they do **not** replace the shuttle seed. Person runs only over **shuttle-seeded candidate windows + padding**, not the whole video ? which bounds the added cost.
- **A small `RallyCue` contract.** Each producer yields `(frame_idx ? features)` + per-window stats. Adding a cue = register a producer; the existing `SegmenterRegistry` (`segmenters/base.py:35`) is the template for the pattern. Keep the single-cue segmenters registered as fallbacks.

---

3. The two fusion modes (both ? per the owner decision)

3.1 Decision-level gate (safe baseline, ships first, no labels needed)

```
rally_active = shuttle_motion ? (?2 persons in court zone ? recent person track)
```

The `? recent-track` clause tolerates 1?2 frame person-detector misses so the AND-gate can't cause a recall cliff. This composes with the **existing net-crossing Gate A**:

```
keep window  ?  net_crossings ? 2            (Gate A ? EXISTS in rally_gate.py, eval-only today)
                ?  (?2 in-court players ? recent track)   (Gate B ? the person cue, NEW)
```

Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-occupancy evidence for the cases a single trajectory can still fool (e.g. a knock-up that does cross the net). No training data required ? this is the label-free fusion that ships before any golden labels exist.

3.2 Feature-level (primary, the learned path)

The cues emit a **small set of aggregated, scale/translation-invariant per-window features ? NOT raw coordinates.** Raw pixel coordinates would memorize *this* court/camera and fail to generalize off a still-small labeled set; counts / ratios / zone-membership / two-sidedness transfer.

What already feeds the classifier (`distill_local.py:40`, `FEATURE_NAMES`): `duration, peak_velocity, mean_velocity, active_ratio, net_crossings` ? note **`net_crossings` is already a feature** (computed via `rally_gate.gate_windows`).

Person features to add (~4, deliberately few):

Feature	Meaning	Rally signal
`players_in_court`	median count of persistent in-court tracks	? 2 (singles) / 4 (doubles); 0?1 = dead time
`two_sided_motion`	are moving players on **both** net-halves?	rally = two-sided; one person feeding = one-sided
`mean_player_motion`	aggregate normalized player velocity	rally = sustained; resting/standing = low
`in_court_ratio`	fraction of frames with ?2 in-court players	track stability vs flicker
`shuttle_player_colocation`	fraction of frames the shuttle sits inside/adjacent to a person box (hand region)	**held = high; in-flight = mostly free space** ? the direct held-shuttle discriminator

These join the shuttle features in the candidate `stats` JSON (`database.py::add_candidate_segment`) and the feature vector of the **existing distilled classifier ? the tiny numpy-only L2 logistic regression in `backend/eval/classifier.py` (ADR-004), NOT a new net.** It scores each candidate window `P(real rally)` and replaces the Gemini call at runtime.

What the model learns: the **weights** on those few features ? e.g. down-weight a shuttle-motion window with 0?1 in-court players / one-sided motion / high shuttle-colocation (warm-up, floor shuttle, held-shuttle feed); up-weight ?2 players, two-sided, sustained motion, shuttle in free flight.

Small-N discipline (non-negotiable):
- Keep to **~4 person features** ? more features on a still-small labeled set overfit; that's exactly
  why the model stays a logistic regression, not a net.
- Use the **leave-one-video-out (LOVO)** protocol that `distill_local.py` **already implements**: train
  on the other videos' candidates, predict the held-out *video*, score vs its labels ? never a
  held-out *window* from the same video (windows in one video are correlated; that flatters the score).
- Training labels today are **Gemini silver labels** (distilled offline, ADR-004/007 ? Gemini is a
  teacher, never a runtime dependency, never a training-data *source* for owned weights); **golden
  labels are the honest measuring stick**, and as the Roora golden corpus grows they can also become
  training labels. Until the labeled set is larger, treat the learned weights as
  calibration / direction-check, not "a fully general model" ? the decision-level gate (§3.1) is
  the shippable, label-free fusion in the meantime.

> *(Gated on golden labels existing ? the same blocker ADR-007 flags for audio. See
> GOLDEN_SET_VENDORING.md.)*

---

4. Shuttle-state: in-flight vs in-hand (the misclassification, mapped to real code)

The "shuttle moved" seed conflates a shuttle **in flight** with a shuttle **carried in hand**. The discriminators, and where each lives:

Signal	In-flight	Held / static	Status
`net_crossings`	? 1 (usually several)	~0 (held) / ~1 (single feed)	**EXISTS** (`rally_gate.py`); eval-only ? wire to prod
`shuttle_player_colocation`	low (free space)	high (near a hand)	**NEW** (needs the person cue)
`direction_reversals`	several (racket contacts)	~0	derivable from trajectory; partial overlap with crossings
`peak shuttle speed`	high	walking speed	derivable from existing velocity

So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation feature from the person cue**. **No new label column is needed** ? the golden labels already start a rally at the *serve-contact frame* and treat all pre-serve / held-shuttle time as dead time (see ROORA_LABELING_GUIDELINES.md), so held-shuttle moments are negatives by construction and are the ground truth that proves the detector wrong.

---

5. Adversarial mitigations (integrating safely)

- **Court-region mask** ? polygon around the court; drop persons outside it (spectators, coaches, adjacent courts). Config-supplied per camera; ~1 ms. Source: a manual per-camera polygon first; the **4 court corners + net-line** that Roora labels once per video (see the labeling doc) build this mask **and** the net split that `two_sided_motion` / `net_crossings` need.
- **Player-count constraint** ? expect **2 (singles) / 4 (doubles)** persistent in-court tracks; suppress transient/extra detections. The `singles_or_doubles` manifest flag sets the expectation.
- **Temporal association** ? reuse `supervision` **ByteTrack** (already in `yolo_hybrid`); reject singleton detections; smooth across frames.
- **Multi-court rejection** ? proximity to the primary court region (homography optional, later).
- **Domain shift (COCO frontal ? side/overhead court)** ? start **box-only** (robust); keep the detector swappable. Fine-tuning a person detector is **parked** ? we have no in-house person labels, and the **paid-LLM/training guardrail + minors-exclusion** apply to any future person-training data.
- **Fusion-degradation guard** ? shuttle stays primary; person is gate + feature, never the sole trigger; the single-cue segmenters remain registered fallbacks.

---

6. Box vs pose

Box-first. Occupancy + count + two-sided motion + shuttle-colocation is enough for both the gate and the features, and box detection is the cheap, robust win. **2D pose is parked behind the same `RallyCue` interface** ? pose (MediaPipe Pose / RTMPose, both Apache-2.0) buys serve-ready stance gating and shot cues later, at the cost of a second model. This matches ADR-007 parking pose as the #2/#3 cue.

---

7. Performance

The person detector is the **existing in-process torchvision model** run at **frame-skip 3?5** (`yolo_frame_skip` default 3) with track interpolation between sampled frames. The shuttle keeps running in its separate WSL process ? the two are **separate processes**, so a "shared backbone" isn't free; budget realistically as the existing `yolo_hybrid` cost minus skip savings. Person runs **only over shuttle-seeded windows + `ai_handoff_padding`**, not the whole video, which bounds the cost.

---

8. Licensing (the guardrail table ? all green)

Every dependency stays **MIT / Apache-2.0 / BSD** (ADR-002 / ADR-005; `ultralytics` was dropped for AGPL-3.0).

Cue / model	Code	Weights / data	Verdict
Shuttle: WASB-SBDT, TrackNetV3	MIT	frozen badminton weights	? (ADR-001/002)
Person (default): torchvision Faster R-CNN / RetinaNet / FCOS	BSD/MIT	COCO-pretrained	? already in repo (`_DETECTOR_FACTORIES`)
Person (escalation): **YOLOX**, **RT-DETR / RT-DETRv2**	Apache-2.0	COCO / Objects365	?
Pose (parked): **MediaPipe Pose**, **RTMPose / MMPose**	Apache-2.0	COCO-structure	?
Tracking: `supervision` ByteTrack	MIT	n/a	? already in repo
Banned	**Ultralytics YOLOv5/8/11 (AGPL-3.0)** · **MonoTrack (Adobe, non-comm)** · **YOLO-NAS (Deci, non-comm)**	?	?

COCO caveat to record: COCO *annotations* are **CC BY 4.0** (commercial OK *with attribution*); COCO *images* are Flickr-ToS (per-image). Pretrained-**weight** use is clean; record the attribution.

---

9. Open decisions (for the owner)

1. **Court-region mask source** ? manual per-camera polygon (simple, ship first) vs auto court homography/line detection (later). **Recommend manual first**, fed by Roora's court-corner labels.
2. **Person detector default** ? keep torchvision (zero new deps, BSD/MIT) vs adopt YOLOX/RT-DETR (Apache, faster) now. **Recommend torchvision first**, escalate only if eval shows a speed/accuracy gap.
3. **Pose now or parked** ? **recommend parked** behind the interface (box-first).
4. **Golden labels** ? feature-level fusion (P3) is **blocked on golden rally labels**; the source is the VLC `rally-annotator` + the Roora vendor pilot (GOLDEN_SET_VENDORING.md).

---

10. Phased execution (owner-gated; ONE PR per slice, off `master`)

- **P0 ? this design doc** + ROORA_LABELING_GUIDELINES.md, indexed in `docs/README.md`, cross-linked from `HOW_RALLY_DETECTION_WORKS.md` and ADR-007. *(this deliverable; docs-only, no code.)*
- **P1 ? extract the shared person cue:** ? **DONE (2026-06-13).** Lifted the torchvision detector + ByteTrack + velocity out of `yolo_hybrid` into `backend/pipeline/detectors/person_detector.py` (`PersonDetector`); `yolo_hybrid` now orchestrates only and delegates to `self.person` (no behavior change ? regression-safe, proven by the unchanged process_video output tests). New `tests/test_person_detector.py`; suite +1 green; person_detector is type-clean (left mypy quarantine for the moved code). The fusion segmenter (P2) reuses this same producer over shuttle-seeded windows.
- **P2 ? the FusionSegmenter (Gate A + Gate B + person-feature persistence):** ? **DONE (2026-06-14).** Shipped `backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (registry `fusion_hybrid`, **default-OFF** ? `default_segmenter` unchanged), subclassing the trajectory engine via two identity hooks (`_enrich_candidate_stats`, `_select_for_handoff` ? wasb/tracknet stay byte-identical, adversarially verified). It seeds windows from the shuttle substrate, **always persists `net_crossings`** (Gate A signal), and ? when `indexing.fusion.cue_flags.person` is on ? runs the person cue over each window's frames (`PersonDetector.detections_per_frame`) and persists the ~5 person features (`fusion_features.py`, pure + unit-tested). Served gating is optional + default-OFF: `fusion.min_crossings` (Gate A) / `fusion.min_players` (Gate B) skip sub-threshold windows from the handoff while **persisting the full superset** (offline substrate intact). Court mask = optional `fusion.court_polygon` (None ? count-only). Lift to be measured via the harness; held-shuttle-drop + person-feature values are GPU/owner-side.
- **P3 ? feature-level (learned) fusion:** add the person features (incl. `shuttle_player_colocation`) to `distill_local` / `classifier`; recalibrate IoU on the golden set; report **LOVO lift on held-out videos**. *(Gated on golden labels.)*
- **P4 ? additional cues:** wire **audio** (ADR-007's #1, already scaffolded in `backend/pipeline/audio/hit_detector.py`) and later **pose**, through the same `RallyCue` interface. Each is add-a-producer, not a refactor.

---

11. Verification

- **Offline, runnable in this dev container (no GPU/torch/cv2):** unit tests for the fusion gate logic and `RallyCue` aggregation (court-mask filter, player-count constraint, `? recent-track` tolerance) and the shuttle-state signals (net-crossing count ? extends the existing `rally_gate` tests ? and person-colocation) with **synthetic trajectories + person boxes**. A key-license assertion test (no `ultralytics` import; detector factories stay torchvision/Apache).
- **Machine-side (owner, GPU/WSL ? the full pipeline can't run in this container):** run the

  `FusionSegmenter` on a real static-tripod clip; confirm **held-shuttle / warm-up / spectator false
  positives drop** vs shuttle-only (this should reproduce the owner's observed bug and show it fixed ?
  start by confirming Gate-A-in-production alone fixes it); **LOVO IoU lift on the golden set** for the
  learned path.
- **Docs (P0):** links resolve; both docs indexed in `docs/README.md`; this doc cross-linked from
  `HOW_RALLY_DETECTION_WORKS.md` (the fusion open-question) and ADR-007.

10. user (2026-06-16T03:12:21.370Z)

How rally detection works (the 2-layer mental model)

A short conceptual map of how we go from raw video to rally `[start, end]` segments ?
and, crucially, **which layer our annotation can improve.**

```
  video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
                            ?                    ?
                       (WASB CNN,          per-frame trajectory
                        frozen weights)          ?
                                                 ?
                                          velocity signal  (how fast the shuttle moves)
                                                 ?
                                          [SEGMENTATION]  ??? the 3 knobs we tune
                                                 ?
                                                 ?
                                       rally windows  [start, end] ??? IoU vs golden labels
```

Two layers ? keep them separate in your head

1. DETECTION ? "where is the shuttle, this frame?"
WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a
visibility/confidence. Its weights were learned (by gradient descent, on shuttle-coordinate
labels) and we use them **frozen**. We chose a *shuttle* detector on purpose: "shuttle in
flight" is the signal for "a rally is happening," and it's **camera-agnostic** ? unlike
player-motion, which fooled the old heuristic on broadcast cameras that pan with the players.
(Your videos are static tripod, so player-motion would also work ? but shuttle-motion is the
robust general bet.)

2. SEGMENTATION ? "when is a rally?"
From the per-frame `(x, y)` we compute the shuttle's **velocity** (displacement between frames).
Then three knobs turn that into rally intervals:
- **`velocity_thresh`** ? a frame is "active" only if the shuttle is moving faster than this
  (in flight, not lying on the floor / being picked up).
- **`merge_gap`** ? active frames within this many seconds get joined into one rally (bridges
  brief gaps: a shuttle momentarily occluded, or the beat between two hits).
- **`min_window_duration`** ? throw away windows shorter than this (blips / false flickers).

Output: a list of `[start, end]` windows = the rallies.

Where the annotation fits (the key point)

Our Roora/golden labels are **rally `[start, end]` times** ? they say *when* rallies are, not
where the shuttle is. So they can't teach WASB's detector (that needs `(x,y)` labels). They
can grade and tune the **segmentation layer**: we score predicted windows against the golden
rallies with **temporal IoU** (how much they overlap), and pick the 3 knobs that maximise it ?
calibrate on some rallies, measure on held-out ones so the number is honest.

Where sparks could land (open questions ? not commitments)
Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
- **Richer features than raw velocity:** acceleration, trajectory *continuity* (a real rally is a
  smooth arc that survives occlusion), shuttle *height/arc*, or **direction reversals** (each
  reversal ? a shot ? ties to `shots_count`!).
- **A tiny learned segmenter** instead of 3 thresholds: feed the trajectory features to a small
  model trained on the golden rally labels (still the segmentation layer, just smarter).
- **Hysteresis** (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-rally dips.
- **Rhythm/periodicity:** rallies have a hit cadence; dead time doesn't.
- **Fusion:** combine shuttle-motion (WASB) with player-motion for static cams. ? designed in
  MULTI_SIGNAL_FUSION_PLAN.md (one rally-fusion layer; the person cue is
  ADR-007's parked #2 and `rally_gate.py`'s named-but-unbuilt "Gate B").
- **Per-video calibration as a *feature*:** "watch a few labeled rallies ? auto-tune to this
  court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right layer.

The whole point of per-video calibration: a handful of *your* labeled rallies nudges these knobs
to *your* court/camera, and we report the IoU lift on the rallies it never saw.

11. user (2026-06-16T03:12:21.499Z)

A signal-fusion framework for rally detection ? analyzers, scopes & reconcilers

Status: **DESIGN ? owner-gated, NOT started.** A peer to
EXPERIMENT_HARNESS.md and MULTI_SIGNAL_FUSION_PLAN.md.
This doc specs a small **framework** for evolving rally-detection quality: producers that emit **signals**
at three temporal **scopes** (frame / window / session), a **scorer** that *learns to weight* them, and an
auditable **reconciliation** pass that lints the decision against the evidence. The goal is to keep piling
on clever cues **without the pipeline becoming a pile of special cases.** Docs-only; no implementation
code. Each phase below is a separate, owner-gated PR.

> **Companion docs:** the detection-layer producers (shuttle / person / audio / shuttle-state) live in
> MULTI_SIGNAL_FUSION_PLAN.md (`RallyCue` = swappable producer; the
> `_enrich_candidate_stats` hook; scale/translation-invariant feature discipline, §3.2); the no-regression
> A/B + LOVO measurement these ride is EXPERIMENT_HARNESS.md (`Variant`, `compare`,
> `compare_unlabeled`); the scorer is the distilled logistic regression (**ADR-004**,
> `backend/eval/classifier.py`); the modular rally layer is **ADR-007**

> (archives/decisions/DECISIONS.md); the 2-layer mental model is
> HOW_RALLY_DETECTION_WORKS.md; the live quality log is
> QUALITY_ITERATIONS.md.

> **Framing.** Rally detection is a genuinely hard temporal problem ? weak, multi-modal cues; interval-level
> ground truth that is scarce and expensive; a confound (warm-up / knock-ups / multi-court / held-shuttle)
> that looks identical to real play in any single frame or frequency band. This doc is written as a
> **reusable framework**, not a badminton-specific patch: the contracts are sport-agnostic and the design
> is meant to generalize to other weakly-supervised temporal-segmentation problems (§10). Where that
> ambition touches research claims, §10 says plainly what is and isn't yet substantiated.

---

1. The spine ? NO special-casing in the decision path

This is the one principle everything else serves:

> **Every producer (analyzer, detector, reconciler) emits a SIGNAL carrying a value + a confidence/presence.
> The SCORING MODEL consumes those signals and *learns to weight them*. We do NOT add hand-coded `if/else`
> branches into the decision.**

Producers propose; the scorer arbitrates; the experiment harness + golden set decide what gets promoted.
That separation is what lets us add a net-hit service fault, a let-cord, a double-bounce, a shuttle-out
cue, a *warm-up span*, a *shuttle-state* ? each as *"register a producer + a feature column,"* never *"add
a branch."* A signal that doesn't earn its keep stays at weight ? 0; it costs nothing because it is an
unused column, not a live `if`.

The pattern already exists in miniature: `experiment.predict()` (`backend/eval/experiment.py:213`) treats
`net_crossings` and `players_in_court` as **feature columns** ? `_gate_mask` is the trivial scorer
(threshold a column), `LogisticRegression` (ADR-004) the richer learned scorer (weight all columns), a
future scorer richer still. The decision path is *signals ? scorer*, with **zero** `if service_fault:
drop`. Analyzers feed it more columns; reconcilers run *outside* it as an audited post-pass.

```
 DETECTION ? frame-scope producers (expensive, GPU/WSL, run ONCE)
   WASB shuttle (x,y,v,vis) · person boxes+tracks · audio hits · shuttle-state(in_air/contact/inactive)*
                                  ?  per-frame-aligned signals (+ net estimate, window bounds)
                                  ?
                shuttle-seeded candidate windows   (FusionSegmenter, P2)
                                  ?
 ANALYZERS ? decision-layer producers (pure, offline, re-scorable)            [forward seam]
   ?? window-scope  (WindowAnalyzer):  one candidate ? features + events + boundary hints   e.g. service-fault
   ?? session-scope (SessionAnalyzer): whole timeline ? labeled spans + per-window
```

```
membership  e.g. warm-up
                                   ?  every signal ? candidate_segments.stats as feature columns
                                   ?
            SCORING / DECISION  ?  signals ? learned scorer  (weights, NO branches)
            _gate_mask (trivial)  +  LogisticRegression (ADR-004)  +  harness-pinned model
                                   ?  predicted rally segments
                                   ?
      RECONCILERS ? backward seam, "linter for rally decisions" (report-first, idempotent)
        ConsistencyRule registry: decision-vs-evidence conflicts ? Issue + corrected set
                                   ?
        stats · events · spans · corrections record · highlights      EXPERIMENT HARNESS gates EVERY promotion
        (* shuttle-state = a richer frame detector; its PRODUCER lives in MULTI_SIGNAL_FUSION_PLAN, consumed here ? §3.5)
```

---

2. The framework ? signals, scopes, and who arbitrates

The architecture has exactly three kinds of moving part. Capability is added by registering a producer; the decision logic never grows a branch.

Producers emit signals at three temporal SCOPES. A signal at a coarser scope is computed by aggregating finer-scope signals; everything ultimately lands as a per-candidate feature column (or an event/span the scorer or a human consumes).

Scope	Producer	Sees	Emits	Cost	Examples
frame	detector (`RallyCue`)	raw frames / audio	per-frame signal	**expensive** (GPU/WSL)	WASB shuttle, person boxes, audio hits, **shuttle-state** (§3.5)
window	`WindowAnalyzer` (§3.1)	one candidate window's aligned frame-signals	features (+conf), events, boundary hints	cheap, pure	service-fault, double-bounce, let-cord, shuttle-out
session	`SessionAnalyzer` (§3.4)	the whole timeline of windows	labeled spans + per-window membership features	cheap, pure	**warm-up / practice**, game/break phases

The scorer arbitrates. It is the only place a decision is made, and it makes it by *weighting columns*, never branching. Today: `_gate_mask` (threshold) + `LogisticRegression` (learned weights, ADR-004). The confidence column on every signal lets the scorer discount an uncertain producer instead of trusting a binary verdict ? the small-N-safe way to fold in a deterministic-ish cue.

Reconcilers audit. After the scorer decides, the backward seam (§6) lints the decision against the persisted evidence and proposes corrections ? report-first, never silent.

Four invariants make this safe to grow (the load-bearing design claims):
1. **Signal, not verdict** ? every producer's output is a weighted feature; the decision path has no producer-specific branch. ? unbounded cues without combinatorial special-casing.
2. **Expensive detection ? cheap decision** ? frame-scope detection runs once and is persisted; window/ session signals and the scorer are pure re-scoring (EXPERIMENT_HARNESS §2). ? experiments are offline, deterministic, zero production risk.
3. **Report-first reconciliation** ? corrections are recorded and reviewable, never applied silently. ? human-in-the-loop, auditable.

4. **Promote only on measured held-out lift** ? LOVO on the golden set + the `run_regression.py` gate;
    default OFF; rollback = config flip. ? no-regression evolution.

---

3. The forward seam ? injectable analyzers

Pluggable producers ? registered like `segmenter_registry` (`backend/pipeline/segmenters/base.py:35`) ?
that run over the **per-frame-aligned signals already produced**: the WASB shuttle trajectory
(`.frame/.visible/.x/.y` + velocity), the person boxes per frame (`PersonDetector.detections_per_frame`,
`person_detector.py:256`), the net line (`rally_gate.estimate_play_axis`, camera-agnostic), and the window
bounds. Analyzers are **pure** ? no I/O, no torch/cv2 ? mirroring `fusion_features.py`'s discipline, so
their math is unit-testable in the GPU-free dev container.

3.1 Contracts (JSON-serializable; proposed `backend/pipeline/analyzers/`)

```python
@dataclass(frozen=True)
class Signal:                          # one scalar cue + how sure the producer is ? never a verdict
    value: float; confidence: float = 1.0    # persisted as `<producer>__<key>` (+ `_confidence`), [0,1]

@dataclass(frozen=True)
class AnalyzerEvent:                     # ? the `events` table on confirmation (§5)
    type: str; timestamp: float; confidence: float = 1.0   # "net_hit"/"double_bounce"/?; src-absolute s
    player: Optional[str] = None; details: Dict[str, Any] = field(default_factory=dict)

@dataclass(frozen=True)
class BoundaryHint:                      # OPTIONAL deterministic proposal ? never auto-applied
    kind: Literal["start", "end"]; timestamp: float; confidence: float; reason: str

@dataclass(frozen=True)
class AnalyzerResult:
    signals: Dict[str, Signal] = field(default_factory=dict)
    events: List[AnalyzerEvent] = field(default_factory=list)
    boundary_hints: List[BoundaryHint] = field(default_factory=list)

@dataclass
class WindowSignals:                     # read-only pre-produced bundle (no I/O inside the analyzer)
    start: float; end: float; fps: float; frame_width: float; frame_height: float
    shuttle: List["TrajectoryPoint"]     # in-window points (.frame/.visible/.x/.y)
    persons: Dict[int, List[Tuple[int, BBox]]]  # frame_idx ? [(track_id, box)]; {} when person cue OFF
    net: Tuple[str, float]               # (axis, position): rally_gate auto-estimate or config
    frame_states: Dict[int, "ShuttleState"]      # frame_idx ? state; {} until the shuttle-state detector (§3.5)
    base_stats: Dict[str, float]         # motion stats already computed for the window

class WindowAnalyzer(ABC):
    name: str; version: str              # registry key + column prefix; version ? provenance on change
    feature_keys: List[str]; event_types: List[str]    # declared columns/events (for the harness)
    @abstractmethod
```

```
    def analyze(self, w: WindowSignals) -> AnalyzerResult: ...
```

The three roles an `AnalyzerResult` can fill:

Role	Sink	Consumed by
(a) features (with confidence)	`candidate_segments.stats` as `<analyzer>__<key>` (+ `_confidence`)	the scoring model **and** the harness, as feature columns (§4)
(b) events	the `events` table (buffered on the candidate, flushed on confirmation, §5)	history / highlights / audit
(c) boundary hints (optional)	stashed in `stats`; never auto-applied	the reconciler (§6) / a future boundary-aware scorer

3.2 Worked example ? the SERVICE-FAULT analyzer (window-scope)

The owner's held-shuttle bug has a sibling: a **service fault** where the serve hits the net and the shuttle comes to rest in the net region. Pure trajectory, GPU-free, no person cue needed:

- **Input:** the window's shuttle trajectory + the `rally_gate` net estimate.
- **Logic:** the shuttle **enters the net region** (`|coord ? net| < deadband`) **and** its velocity **decays to ? 0** there (comes to rest) ? high-probability net-hit service fault. Confidence scales with how cleanly the halt sits inside the net band.
- **Emits:** a `service_fault` **signal** (value = presence, confidence = halt cleanliness) + a `net_hit` **event** at the halt timestamp + a rally-**END** `BoundaryHint` at that timestamp.

Crucially: the analyzer does **not** drop the window. It *proposes* `service_fault`; the scorer sees `service_fault` + `service_fault_confidence` as two more columns and **learns** whether/how much to down-weight. The boundary hint is an audited proposal the reconciler may act on ? never a branch.

3.3 Injection point ? generalize `_enrich_candidate_stats`

The FusionSegmenter hook (`fusion_hybrid.py:70`) already turns "compute fixed features" into per-window enrichment. We generalize it to **"run each enabled analyzer"** ? additively, so the existing `net_crossings`/person paths are unchanged:

```python
def _enrich_candidate_stats(self, cw, points, fps, frame_width, video_path, idx_cfg):
    stats = super()._enrich_candidate_stats(...)        # base motion keys
    stats["net_crossings"] = ...                        # Gate-A signal (kept)
    # ? person features when the person cue is on (kept) ?
    w = self._window_signals(cw, points, fps, frame_width, video_path, idx_cfg)
    for analyzer in analyzer_registry.enabled(idx_cfg):  # NONE enabled ? byte-identical to control
        res = analyzer.analyze(w)
        for key, sig in res.signals.items():
            stats[f"{analyzer.name}__{key}"] = sig.value
            stats[f"{analyzer.name}__{key}_confidence"] = sig.confidence
        stats.setdefault("_events", []).extend(e.__dict__ for e in res.events)        # flushed in Phase 4
        stats.setdefault("_boundary_hints", []).extend(h.__dict__ for h in res.boundary_hints)
    return stats
```

Flag-gated, **default OFF** via the existing `indexing.fusion.cue_flags` (a sibling `analyzer_flags` map keyed by `analyzer.name`). With every analyzer OFF, the loop is empty ? candidate rows are byte-identical to the substrate ? the no-regression invariant holds (§7). `net_crossings` and the person features are the **prototype analyzers** the seam retrofits first (proving it with zero behavior change); every new cue after that is a pure add.

3.4 Session-scope analyzers ? the WARM-UP / PRACTICE example

Some structure is invisible at the window scope. **Warm-up** looks exactly like a rally in any single window ? shuttle in flight, two players, net crossings, sustained motion ? which is *why* it is a notorious false-positive source. It is only distinguishable by **timeline structure**: a long contiguous active stretch *without* the rally?dead-time?rally cadence of scored play, usually (not always) at the start.

So warm-up needs a producer that sees the **whole sequence of windows**, not one window. That is a sibling tier ? a `SessionAnalyzer` that runs once per video (a second pass, after per-window enrichment) and emits a labeled **span** plus a per-window **membership feature**:

```python
@dataclass(frozen=True)
class Span:
    kind: str                           # "warmup" | "practice" | "break" | "game" | ?
    start: float; end: float; confidence: float; reason: str

@dataclass(frozen=True)
class SessionResult:
    spans: List[Span] = field(default_factory=list)
    # window_index ? signals merged back into THAT window's stats (e.g. {"in_warmup": Signal(0.8, 0.6)})
    window_features: Dict[int, Dict[str, Signal]] = field(default_factory=dict)

class SessionAnalyzer(ABC):
    name: str; version: str
    span_kinds: List[str]; feature_keys: List[str]
    @abstractmethod
    def analyze(self, windows: List[WindowSignals], session: "SessionContext") -> SessionResult: ...
```

`WarmupAnalyzer` (worked):
- **v0 (free, no new model):** find the **leading contiguous high-activity span** with low inter-window dead-time and no serve-reset cadence, over the persisted window timings we already have. Emit `Span(kind="warmup", ?)` + per-window `in_warmup` = overlap fraction of the window with the span.
- **v1 (learned):** fed by the **shuttle-state cadence** (§3.5) / the **audio thwack** cadence ? warm-up = continuous hits *without* the point-ending pause rhythm; generalizes to mid-session practice / between-game knock-ups (drop the position prior).

You framed the consumption exactly right: "so the scorer may consume it." `in_warmup` is a **feature column**; the scorer *learns* to down-weight windows inside the span. **Warm-up is the poster child for no-special-casing**: the naive version is `if t < warmup_end: drop` ? a hand-coded branch hanging off a fuzzy boundary; a 20-second error **deletes a real opening rally**. As a weighted feature, a wrong boundary just slightly mis-weights a couple of windows the other cues rescue. The span is *also* persisted as evidence a reconciler (§6) may use to down-weight a whole cluster, but the primary path is feature ? scorer.

It is measurable today with no new labels: the golden labels already treat warm-up knock-ups / pre-serve as **dead time = negatives** (Roora starts a rally at serve-contact; fusion §1 names "warm-up knock-ups" as a precision target). So an `in_warmup` feature is directly A/B-able on the 6 golden videos for **precision** lift ? that is the promotion gate.

3.5 Consuming a richer detector ? SHUTTLE-STATE (frame-scope producer)

A natural new **detector**: a per-frame shuttle-state classifier ? `in_air` / `racket_contact` / `inactive`. Note the scope: it **processes frames ? a new per-frame signal**, like WASB, so it is a **frame-scope producer in the detection layer** ? its spec belongs in MULTI_SIGNAL_FUSION_PLAN.md (it is a new `RallyCue`), **not** a `WindowAnalyzer`. This doc records **how the framework consumes it** once it exists:

- a **`WindowAnalyzer`** turns the persisted per-frame state into window features ? `racket_contact_count` (? shot cadence), `inactive_ratio` (? dead-time) ? that flow into the scorer as columns;
- a **`SessionAnalyzer`** (warm-up, §3.4) reads its cadence;
- the **reconcilers** (§6) get *stronger* evidence: `trailing_dead_time` / `over_merged` currently infer inactivity from a velocity threshold; a learned `inactive` state is a more direct, occlusion-robust evidence source for "truncate / split here."

Stage it (don't build the model first). Most of the state is derivable for free from signals we already have ? `in_air` vs `inactive` ? WASB velocity + visibility; `racket_contact` ? trajectory **direction reversals** (fusion §4: "each reversal ? a shot") **and** the scaffolded **audio thwack** (`backend/pipeline/audio/hit_detector.py`, ADR-007's #1 cue). So **v0 = a derived `shuttle_state` analyzer** over existing signals (no new model); measure it. **The dedicated per-frame vision model earns its keep only where v0 hits a ceiling: WASB trajectory gaps** ? occlusion, motion blur, fast smashes ? where there is no `(x,y)` to threshold but an appearance model could still call `racket_contact`. **Label blocker:** we have **zero** per-frame state labels (golden labels are rally intervals); a dedicated model needs weak/ distilled labels or a new (minors-gated) annotation task ? the same gate as feature-level fusion. If you can weak-label state from existing signals, prefer the derived analyzer until the gap-filling case is proven.

---

4. How signals reach the scorer ? columns, not branches

The persisted `stats` columns are exactly what the harness already lifts.

`load_video_candidates(...,
feature_names=[...])` (`experiment.py:543`) reads each name out of the row's `stats` JSON into a feature
column; `predict()` (`experiment.py:213`) gates/scores on those columns and merges. So a new producer's
signal becomes a scorer input by **adding its column name to a `Variant`'s `feature_names`** ? no new
scoring code, no branch:

```python
# A learned Variant that lets the scorer weight the new signals (service-fault + warm-up membership):
fusion_plus = Variant(name="fusion+analyzers",
                      feature_names=[*SHUTTLE_FEATURES, "person__players_in_court",
                                     "service_fault__service_fault", "service_fault__service_fault_confidence",
                                     "warmup__in_warmup", "warmup__in_warmup_confidence"])
compare(golden_videos, CONTROL, fusion_plus)        # LOVO-honest B ? A on the 6 golden videos
```

The model **learns the weights** ? down-weight a window with a high-confidence `service_fault` or high
`in_warmup`; up-weight ?2 in-court players + multiple net-crossings. **What the scorer cannot see, it
cannot special-case; what it can see, it weights from data.** Same logistic regression as ADR-004; we add
columns, never a net.

---

5. Persistence

- **Features ? `candidate_segments.stats`** (`database.py:240`). Namespaced `<producer>__<key>`
  (+ `_confidence`). Persisted **as a shadow** even when the served decision ignores them, so any future
  `Variant` re-scores offline (EXPERIMENT_HARNESS §5.2). `version`/`name` recorded for provenance.
- **Spans ? window membership + a session record.** A `SessionAnalyzer`'s per-window `in_<span>` features
  ride in each window's `stats` (so they re-score like any feature); the span list itself (start/end/kind/
  confidence) is a small per-video record (a JSON sidecar first; a table only if it grows).
- **Events ? the `events` table** (`database.py:227`, `add_event(segment_id, ts, type, player, details)`).
  Its FK is to `play_segments`, which exist only **after** the AI handoff confirms a candidate. So
  analyzer events are **buffered on the candidate** (`stats["_events"]`) and **flushed** to `events` in
  Phase 4, when `_candidate_id ? segment_id` is linked (`trajectory_hybrid.py:283`). Candidate-level
  discoveries survive for offline re-scoring; the `events` table stays "events on confirmed segments."
- **Corrections record** (§6) ? report-first means corrections are **recorded, never silent.** Phase one:
  a JSON sidecar shaped like `experiment_leaderboard.json` (one row per issue: segment, kind, evidence,
  before?after, reason, timestamp). When auto-apply is promoted, add a `corrections` table via the
  `PRAGMA user_version` migration ladder (`database.py:88`, `if version < 3:`) so applied corrections are
  queryable and reversible.

---

6. The backward seam ? the reconciliation pass (a linter for rally decisions)

Given **predicted segments + the persisted per-frame/per-window/per-session signals**, detect where the **DECISION contradicts the EVIDENCE** and propose corrections. This is the backward dual of the analyzers: analyzers add evidence *before* the decision; reconcilers audit the decision *against* evidence *after* it.

6.1 Contracts (proposed `backend/pipeline/reconcilers/`)

```python
@dataclass(frozen=True)
class Issue:
    segment_index: int; kind: str       # kind = rule id, e.g. "trailing_dead_time"
    evidence: Dict[str, Any]            # the signal values that triggered it (auditable)
    before: Interval; after: Optional[List[Interval]]   # None=drop · [iv]=corrected · [iv1,iv2]=split
    reason: str; confidence: float

@dataclass(frozen=True)
class Correction:
    issues: List[Issue]; corrected: List[Interval]      # the FULL corrected set (idempotent)

class ConsistencyRule(ABC):
    name: str; precedence: int          # FIXED ordering; lower runs first
    @abstractmethod
    def check(self, seg: Interval, signals: "SegmentSignals") -> Optional[Issue]: ...

def reconcile(segments, signals, rules) -> Correction: ...   # fixed precedence; idempotent
```

6.2 Worked rules (fixed precedence)

#	Rule	Evidence contradiction	Correction
1	`trailing_dead_time`	segment still "in rally" at its end, but the shuttle went inactive earlier	**truncate** to the last-active frame *(the owner's misfire fix)*
2	`no_net_crossing`	`net_crossings < 1` ? nothing ever crossed the net	**drop** (or flag)
3	`over_merged`	a dead-time gap **>** `merge_gap` of shuttle-inactivity sits **inside** one segment	**split** at the gap
4	`boundary_slack`	first sustained shuttle activity / first net-crossing is well inside the start	**tighten** start to serve-contact
5	`inside_warmup`	the segment lies inside a high-confidence `warmup`/`practice` span (§3.4)	**flag** (drop only under apply-flag)

Each rule reads only **persisted evidence** (trajectory activity per frame, `net_crossings`, person occupancy, analyzer signals/events, session spans, shuttle-state) ? it is a deterministic linter, not a re-detector. As richer producers land (shuttle-state, warm-up), rules get **stronger evidence**, not new branches in the decision path.

6.3 Report-first, apply-optional (non-negotiable)

`reconcile()` returns **both** an issues report **and** a corrected set. It **NEVER silently rewrites** (auditability + the repo's honesty/attribution working agreement). The pipeline either:

- **(default)** persists the report and **surfaces it for human review** ? production output unchanged; or
- **(flagged, `reconcile.apply`)** applies the corrected set, **recording every change** in the
  corrections record (before?after, rule, evidence). Rollback = flip the flag.

It is a `Variant` transformation, so its lift is A/B-measurable BEFORE auto-apply. A reconciler is a
pure post-`predict` stage: `reconcile(predict(...), signals, rules).corrected`. Wrapping that as a
`RECONCILED` variant lets `compare(CONTROL, RECONCILED)` report the LOVO lift on the 6 golden videos
(QUALITY_ITERATIONS.md corpus) ? promote to auto-apply **only** on measured lift.

Idempotent; fixed rule precedence. `reconcile(reconcile(x)) == reconcile(x)` (truncating an
already-truncated segment is a no-op); precedence is a fixed table so the result is deterministic and
order-independent of input segment order.

---

7. No-regression discipline (identical to the harness contract)

The same four guarantees EXPERIMENT_HARNESS.md §6 makes ? applied to every producer and the reconciler:

- **Default OFF.** Analyzers gate on `analyzer_flags` (default empty); the reconciler is report-first
  (apply gate default off). Merged code **cannot** change served behavior until explicitly enabled.
- **Shadow-persist, then promote.** Signals/spans/issues are persisted even while ignored, so a `Variant`
  re-scores them offline; a signal becomes a *decision input* only when a Variant that references it wins
  LOVO. (EXPERIMENT_HARNESS §5.2's shadow mode.)
- **Promotion only via the eval gate.** `run_regression.py` against the golden `baseline.json` (exit 1
  blocks); the experiment leaderboard records the spec hash + per-video + LOVO deltas + label-set hash.
- **Rollback = flip a config flag, never `git revert`.** The pinned `CONTROL` variant is always present.

The keystone test (extends the harness's existing invariant): **all analyzer flags OFF + reconciler
report-only ? candidate rows AND predictions byte-identical to control.**

---

8. How it composes with fusion P2

- **Window-analyzers plug into fusion P2's enrichment hook.** `FusionSegmenter._enrich_candidate_stats`
  (`fusion_hybrid.py:70`) is the exact injection point §3.3 generalizes; `net_crossings` and the person
  features become the first registered analyzers (no behavior change). New cues are add-a-producer, per
  ADR-007 / MULTI_SIGNAL_FUSION_PLAN §2.
- **Session-analyzers run as a second pass** over the per-window stats the same hook persists.
- **The shuttle-state detector is a new fusion `RallyCue`** (frame-scope producer) ? it belongs in the

fusion plan; this framework only *consumes* it (§3.5).
- **Reconcilers consume fusion P2's persisted signals** ? the same `candidate_segments.stats` plus the persisted trajectory; no new detection.
- Everything inherits fusion P2's control-reproducing defaults, so the fusion segmenter with all flags off still seeds and hands off identically to `wasb_hybrid`.

---

9. Trade-offs & risks

- **Deterministic vs learned signals.** A producer may emit a verdict-shaped signal (`service_fault=1.0`), but we persist it as a **feature**, so the scorer still arbitrates. Risk: a confidently-wrong cue. Mitigation: the **confidence column** + the **LOVO gate** ? a bad signal earns ? 0 weight, never promotes.
- **The temporal confound is the hard core.** Warm-up-vs-play, multi-court, and held-shuttle all look identical in any single frame / frequency band ? QUALITY_ITERATIONS' standing lesson: "the dead-time confusion is rally-vs-non-rally activity in the same place and frequency band... only the learned model's temporal features crack it." This is *why* warm-up is a **feature the model weights**, not a heuristic span-cutter, and why session-scope structure earns its own tier.
- **Scope discipline.** Put a producer at its *natural* scope: shuttle-state is frame-scope (a detector), not a window analyzer; warm-up is session-scope, not per-window. Mis-scoping forces a producer to either recompute detection or miss the structure it needs.
- **Ordering / idempotence (reconcilers).** Fixed precedence table + an idempotence test (`reconcile?reconcile == reconcile`).
- **Signal, not verdict.** A producer that *decides* (returns keep/drop) re-introduces special-casing; reviews must reject "analyzer returns a boolean keep/drop."
- **Report-first auditability.** Reconcilers never silently mutate output; every applied change is recorded.
- **Small-N feature discipline.** Few, scale/translation-invariant columns (counts/ratios/presence, never raw coords ? MULTI_SIGNAL_FUSION_PLAN §3.2); the model stays a logistic regression, not a net.
- **Labels gate the learned producers.** Shuttle-state and feature-level fusion need labels we don't yet have; warm-up is measurable *now* because golden treats it as negatives. Know which producers are gated.
- **Cost & licensing.** Pure analyzers are GPU-free; person- and shuttle-state-dependent producers pull torch / a second WSL pass ? gate behind their cue flags; all deps stay MIT/Apache/BSD (ADR-002/005).
- **Event-table timing.** The buffer-then-flush wiring (§5) is the one non-obvious integration detail.

---

10. Generality & research positioning

This is written as a **framework**, not a badminton patch. Read sport-agnostically it is: *a registry-based, multi-scope **signal-fusion** architecture for **weakly-supervised temporal event/segment detection**, with a learned arbiter and an **auditable post-hoc correction** layer.* The pieces that generalize:

- **Multi-scope producers ? one feature table ? a learned arbiter.** The frame/window/session taxonomy and the "signal-not-verdict" rule are domain-independent. Any temporal problem with **weak, multi-modal cues at different time scales** (broadcast sports event spotting, surgical-phase recognition, call-center / meeting segmentation, activity logs) fits the same shape: add a producer, add a column, let the arbiter weight it, gate promotion on held-out lift.
- **Detection ? decision (persist once, re-score forever).** Decouples the expensive, hard-to-reproduce perception from the cheap, deterministic decision ? the property that makes the whole thing experimentally honest and cheap to iterate.
- **Auditable reconciliation as a first-class layer.** Most pipelines bury post-hoc fixes as silent if-branches; treating consistency-vs-evidence as a *report-first, A/B-measured* pass is the transferable methodological idea (and the human-in-the-loop / safety story).
- **The honest measuring stick.** Interval-level, scarce ground truth + **leave-one-VIDEO-out** (group by match, never a held-out window from the same video) is the discipline any small-corpus temporal study needs to avoid flattering itself.

What is defensibly contributed vs. what is standard. Multi-modal **late fusion**, weak supervision, and learning a linear/logistic arbiter over hand-designed features are all standard. The *combination* offered here ? (a) a uniform producer-registry across **three temporal scopes**, (b) a hard **no-branch** decision invariant that makes extension monotone, (c) a **report-first, harness-measured reconciliation** layer, and (d) the **detection?decision persistence** substrate that makes every experiment a zero-risk offline re-score ? is what would be written up, with rally detection as the motivating hard problem.

Honesty caveat (attribution working agreement). No external results or citations are asserted here. A real write-up still owes: a proper **related-work survey** (temporal action segmentation/detection, sports event spotting, multimodal fusion, weak supervision ? to confirm which claims are actually novel), **strong baselines**, **per-producer ablations**, and reporting on **>1 sport / camera** to substantiate the generality claim. Today the corpus is **6 matches** (QUALITY_ITERATIONS) ? enough for direction-checks and LOVO sign-tests, **not** enough to claim a general model; corpus growth is the highest-ROI unblock. Treat §10 as a *positioning sketch*, not a results section.

> **Update (2026-06-14): that related-work survey is now done ? see [§13](#13-prior-art-terminology--course-corrections-literature-review-2026-06-14).**
> The honest finding: the *skeleton* (TAL/TAS + late fusion) is standard and must be **cited, not claimed**;
> the only `novel-combination` is the report-first reconciler-as-linter bundle; warm-up-as-a-soft-feature is
> the thinnest genuine gap. §13 also lists the verified citations and the prioritized course corrections.

---

11. Phased execution (owner-gated; ONE PR per slice, off `master`, default OFF)

- **A0 ? this doc**, indexed in `docs/README.md`, cross-linked from the peers + ADR-007. *(this deliverable; docs-only.)*
- **A1 ? analyzer contracts + registry** (`WindowAnalyzer`/`AnalyzerResult`/`Signal`/`AnalyzerEvent`/ `BoundaryHint` + `analyzer_registry`); generalize `_enrich_candidate_stats` to iterate enabled analyzers (none enabled ? byte-identical). Retrofit `net_crossings` + person features as the first two analyzers.
- **A2 ? service-fault analyzer** behind `analyzer_flags.service_fault` (default OFF); signal + `net_hit` event + end boundary hint; shadow-persisted; offline `compare(CONTROL, +service_fault)` LOVO.
- **A3 ? event flush wiring** (buffer analyzer events on candidates ? `add_event` on confirmation).
- **A4 ? `SessionAnalyzer` tier + `WarmupAnalyzer` v0** (leading-span heuristic over window timings), default OFF; persists `warmup__in_warmup` + the span record; measured for **precision** lift on golden.
- **A5 ? derived `shuttle_state` analyzer v0** (velocity + visibility + direction-reversals (+ audio); **no new model**); emits `racket_contact_count` / `inactive_ratio`; measured LOVO. *(The dedicated per-frame shuttle-state DETECTOR is a separate MULTI_SIGNAL_FUSION_PLAN `RallyCue` item ? gated on per-frame labels; build only if v0 is blocked by trajectory gaps.)*
- **R1 ? reconciler contracts + registry + `reconcile()`** (`ConsistencyRule`/`Issue`/`Correction`) with the worked rules; pure, idempotent, unit-tested; **not** wired to apply (report-only).
- **R2 ? reconciler as a `Variant` transform**: optional post-`predict` stage; `compare(CONTROL, RECONCILED)` LOVO; report-only in production.
- **R3 ? corrections persistence + opt-in apply**: JSON sidecar first, then the `corrections` table (`user_version < 3`); apply behind `reconcile.apply`; **rollback = flip the flag.**

Each PR is gated by the no-regression invariant (§7) + `run_regression.py`. Order is a suggestion; A4/A5/R1 are independent and can be reprioritized on owner steer.

---

12. Verification

- **Offline, runnable in this dev container (no GPU/torch/cv2):** pure analyzer math on synthetic trajectories + person boxes (service-fault fires on a synthetic net-halt, silent on a clean rally; `WarmupAnalyzer` v0 finds a synthetic leading knock-up span; derived `shuttle_state` matches a scripted velocity/visibility sequence); the **no-regression invariant** (all flags OFF + reconciler report-only ? candidate rows + predictions byte-identical to control); the **reconciler idempotence** test (`reconcile?reconcile == reconcile`) and fixed-precedence determinism; `compare(CONTROL, variant)` deterministic on synthetic candidate rows.
- **Metrics ? adopt the field's (see [§13](#13-prior-art-terminology--course-corrections-literature-review-2026-06-14)/C4):** report **segmental F1@{10,25,50} + edit score** (TAS) and **tight/loose mAP@?** (action spotting) alongside temporal-IoU. tIoU alone is nearly blind to the over-segmentation the reconciler exists to fix, so a reconciler win ? or an IoU-chasing rule that shreds rally structure ? is invisible under tIoU.

- **Machine-side (owner, GPU/WSL ? full pipeline can't run here):** persist analyzer signals on a real clip; `compare(CONTROL, +service_fault)`, `compare(CONTROL, +warmup)`, `compare(CONTROL, RECONCILED)` LOVO on the 6 golden videos; confirm the owner's **trailing-dead-time misfire** truncates, the **held-shuttle / net-hit service feed** is down-weighted, and **warm-up windows** lose precision-mass; spot-check `net_hit` events on confirmed segments. **Grade with §13/C4 metrics + per-fold CIs (C1).**
- **Docs (A0):** links resolve; indexed in `docs/README.md`; cross-linked from MULTI_SIGNAL_FUSION_PLAN.md, EXPERIMENT_HARNESS.md, HOW_RALLY_DETECTION_WORKS.md, and ADR-007.

---

13. Prior art, terminology & course corrections (literature review, 2026-06-14)

A multi-agent web-research pass surveyed the eight design patterns of this framework against the literature and **adversarially verified every citation** (real-paper check + closer-prior-art hunt). **Result: zero fabricated papers**; a few wrong author-lines / metadata slips were caught and are corrected below. This section is the honest "are we reinventing the wheel?" answer and the actionable course corrections ? all **owner-gated, NOT yet implemented** (they fold into the A*/R* phases of §11).

13.1 Verdict ? what is standard vs. what is ours

> **TL;DR.** The architecture's *skeleton* is standard and must be **cited, not claimed**: candidate-windows + learned scorer = **Temporal Action Localization** (proposal-then-score); the reconciler's over-segmentation fixes = **Temporal Action Segmentation** refinement/post-processing; "signal not verdict / learned arbiter over cue columns" = **late / score-level fusion** realized as **stacked generalization** ? ~20 years old and *already present in racket sports*. The **defensible delta is the engineering/governance envelope**: detection?decision persisted as a re-score substrate + a **report-first, A/B-lift-gated, idempotent, human-in-the-loop reconciler-as-linter** keyed on decision-vs-evidence *contradiction* + the flag-gated no-regression promotion harness, on a deliberately tiny LOVO-by-match golden set with an interpretable scorer. Of the patterns, only the **reconciler-as-linter** rated `novel-combination`; the rest are `partial-overlap`. Park any novelty claim narrowly on (a) that reconciler bundle and (b) **warm-up-as-a-soft-session-feature** (the one slice with genuinely thin prior art).

Pattern (our framing)	Field's name	Closest verified prior art	Verdict
Candidate-windows + scorer + reconciler-as-refinement	Temporal Action Localization / Segmentation + boundary refinement	ASRF (arXiv:2007.06866); Activity Grammars (arXiv:2312.04266); BMN (arXiv:1907.09702)	partial-overlap
"Signal not verdict" ? learned arbiter over cue columns	Late / score-level fusion + stacked generalization	Snoek 2005 (10.1145/1101149.1101236); Wolpert 1992; racket-sports A/V rally-break fusion (CVIU 2009)	partial-overlap
Detection?decision, "persist once, re-score forever"	Frozen-backbone cached features + decoupled proposal/classification	S-CNN (arXiv:1601.02129); CBR (arXiv:1705.01180); Decouple-SSAD (arXiv:1904.07442)	partial-overlap (? field default)

Report-first reconciler (contradiction ? audited correction, measured before apply)	Model assertions + constraint-based repair ("ML linter")	Model Assertions (arXiv:2003.01668); HoloClean (arXiv:1702.00820); TFDV (MLSys 2019); EventAnchor (arXiv:2101.04954)	**novel-combination**
Scarce interval labels + tiny LR + LOVO-by-match	Point/timestamp-supervised TAL + grouped (LOGO/LOSO) CV	SF-Net (arXiv:2003.06845); LOOCV-bias (arXiv:2406.01652); Snorkel (arXiv:1711.10160)	partial-overlap
Multi-scope (frame/window/session); warm-up conditions per-window	Multi-scale/hierarchical modeling + play-break + phase recognition	Tjondronegoro ICME 2003; GL-MSTCN (10.1186/s12938-022-01048-w); MS-TCN (arXiv:1903.01945)	partial-overlap

In-domain prior art that already hard-codes our cues (so our delta is the *no-branch discipline*, not the cues): non-broadcast badminton rally start/end (See et al. 2024/2025), shuttle-state/hit detection (MonoTrack arXiv:2204.01899; Hsu/Yu/Cheng, Sensors 2024 24(13):4372), service-fault (Goh et al., Sensors 2023 23(24):9759), tennis in-play state machines (10.1080/19346182.2013.819007), badminton point-segmentation (Ghosh et al. 2018, arXiv:1712.08714), ShuttleSet (arXiv:2306.04948).

13.2 Terminology to adopt (gloss our nouns with the field's on first use)

Our term	Field's established term	Anchor
`RallyCue` / "producer"	**labeling function** / weak-supervision source; **base model** in a stack	Snorkel; Wolpert 1992
"Signal not verdict / no special-casing"	**late / score-level fusion**; arbiter = **level-1 combiner / meta-learner**	Snoek 2005; Wolpert 1992
ADR-004 LR scorer (`backend/eval/classifier.py`)	**stacked meta-learner / level-1 combiner**	Wolpert 1992
"Detection?decision / persist once, re-score"	**frozen-backbone cached features**; **decoupled proposal/classification**; offline re-scoring	S-CNN; Decouple-SSAD
`WindowAnalyzer` (window-scope)	**temporal action proposal** features / **actionness**	S-CNN; BMN
Session-scope / warm-up	**video-level temporal context / phase segmentation**; **play-break** outer layer; **global-local**	Tjondronegoro 2003; GL-MSTCN
`ConsistencyRule` reconciler	**model/consistency assertions**; **constraint-based repair**; **boundary refinement**; **activity grammar / constrained decode**	Kang 2020; HoloClean; ASRF; Activity Grammars
"Report-first linter"	**data validation / schema anomalies** ("unit tests for data")	TFDV; Deequ
"Over-merged / split" · "tighten boundary slack"	**over-segmentation error** · **boundary regression/refinement**	MS-TCN; ASRF; CBR
"LOVO, group by match"	**leave-one-group-out / -subject-out (LOGO/LOSO)** grouped CV	sklearn `GroupKFold`; Austin 2025
Temporal-IoU metric	**mAP@tIoU** (TAL) + **F1@{10,25,50} & edit score** (TAS) + **tight average-mAP** (spotting)	Lea 2017; SoccerNet; Soares 2022

Keep the internal nouns; just gloss each on first use. Low-effort, high-leverage ? reviewers map instantly to 30 years of literature and see we know the failure modes.

13.3 What we can / cannot claim

- ? **The integrated governance envelope as a whole** ? no found paper bundles all of: report-first contradiction detection over a *persisted temporal decision* + auditable structured interval corrections (truncate/split/drop/tighten) + **per-rule A/B lift-gating before auto-apply** + idempotency/precedence + human-in-the-loop + feeding a learned (value, confidence) scorer.
- ? **"Signal not verdict" as a deliberate *design-philosophy delta*** vs. the in-domain prior art that hard-codes the same cues as if/else verdicts (state it as a philosophy delta, not a

capability they lack).
- ? **Warm-up/practice as a soft session-scope feature** ? thin prior art (only patents/coaching content surfaced); claim cautiously, validate from scratch.
- ? The **fusion mechanism** (multi-cue features ? light scorer for rally segmentation) ? ~20 yrs old in racket sports (CVIU 2009; Ghosh et al. 2018 learn an SVM over per-frame features ? smoothed rallies).
- ? The **persist-once/re-score substrate** as novel ? it is the *default* of the TAD field (datasets ship precomputed features so cheap heads re-train). Claim only the *idempotent/versioned discipline*.
- ? The **cross-check-and-correct insight** ? Hsu et al. 2024 already cross-check shuttle-vs-swing streams; EventAnchor already does human-in-the-loop suggest-then-verify. Our delta is *measured-before-apply + auditability*, not the cross-check.
- ? **Static single-camera as a novel setting**, and ? the **no-regression/flag-gated harness** as a primitive (textbook MLOps) ? claim only "this discipline on a deliberately tiny golden set."

13.4 Course corrections (owner-gated; prioritized)

Tier 1 ? protect the honesty of "measured lift" on ~6 matches (do first):
- **C1 ? bias-aware LOVO.** Standard leave-one-out on ~6 imbalanced groups induces a train-vs-heldout label anti-correlation that biases the estimate (a LR scored ~0.23 auROC on *random* data in the cited study). Use class-balance-preserving folds (RLOOCV) and **report per-fold variance + CIs, never a single mean**, so the promotion gate can't promote noise. *(Austin/Pe'er/Korem 2025, arXiv:2406.01652.)* ? **harness/eval.**
- **C2 ? calibrate each producer's confidence** (Platt/isotonic) *before* the LR, so a CNN softmax, an audio onset, and an integer net-crossing count are comparable and the over-confident cue can't swamp the arbiter. *(MoCaE arXiv:2309.14976; modality competition arXiv:2203.12221.)* ? **producers + scorer.**
- **C3 ? out-of-fold (honest) stacking.** When a producer is itself learned (shuttle-state), feed the LR its *out-of-fold* outputs; extend the group-by-match split to producer training. *(Wolpert 1992.)* ? **detection?decision + harness.**
- **C4 ? report the field's metrics.** Add **segmental F1@{10,25,50} + edit score** and a **tight/loose mAP@?** sweep alongside temporal-IoU: tIoU is nearly blind to over-segmentation, so a reconciler win (or an IoU-chasing rule that shreds structure) is invisible under tIoU alone. *(MS-TCN; ASRF; SoccerNet arXiv:1804.04527; Soares 2022 arXiv:2206.07846.)* ? **harness/eval + reconciler.**

Tier 2 ? architecture/discipline upgrades:
- **C5 ? resolve reconciler rules GLOBALLY** (an activity-grammar / constraint-aware Viterbi decode), not as stacked local edits with fixed precedence (which can conflict / be non-idempotent across orderings). Model the structural priors (transition/duration) from data. *(Activity Grammars arXiv:2312.04266; Constraint-Aware Decoding arXiv:2605.10149 ? read before citing.)* ? **reconciler.**
- **C6 ? brand the reconciler as "model/consistency assertions,"** attach a per-correction confidence, and route only high-confidence corrections to auto-apply, the rest to a margin-ranked human queue; version the rule set as a TFDV-style owner-ratified anomaly schema. *(Kang 2020; HoloClean ~90%

repair precision ? ~10% bad; Confident Learning arXiv:1911.00068.)* ? **reconciler + harness.**
- **C7 ? a few EARLY-fusion interaction columns** (e.g. `shuttle_in_air AND person_two_sided`) ? a linear arbiter can't represent AND/XOR; hand-crossed products recover it without leaving "signal not verdict." *(Snoek 2005.)* ? **producers + scorer.**
- **C8 ? represent a missing cue as an explicit `is_present`/uncertainty column, not a silent 0** ? "cue absent" ? "cue says no" for a linear arbiter. *(DBF arXiv:1511.03183.)* ? **producers + scorer.**
- **C9 ? keep warm-up/session-state a SOFT feature, never a hard pre-filter**, and add a court/context invariance probe: the static court co-occurs with rallies, so the scorer can learn "court visible" instead of "point in play" (action-context confusion). Evaluate on warm-up stretches that share the court but lack point cadence. *(arXiv:2103.16155; GL-MSTCN.)* ? **session-scope producers + scorer.**

Tier 3 ? leverage external resources / higher-effort:
- **C10 ? model cue CORRELATIONS** (Snorkel/MeTaL label-model baseline; strong L1/L2): net-crossing, two-sidedness, shuttle-in-air co-fire, so an unregularized LR double-counts the bundle. *(arXiv:1711.10160.)*
- **C11 ? a temporal-smoothing pass** (Kalman/HMM/Viterbi over the per-window posterior, or an MS-TCN truncated-MSE anti-fragmentation penalty) before cutting intervals ? SmartTennisTV hits F1 ~97.5% largely via Kalman-smoothing noisy per-frame predictions. *(arXiv:1801.01430; MS-TCN.)* ? **reconciler/scorer.**
- **C12 ? pull ShuttleSet/ShuttleSet22 + CoachAI** as an external benchmark + weak-supervision corpus to attack the ~6-match bottleneck. *("Less is More" arXiv:1903.00859; ShuttleSet arXiv:2306.04948.)* ? **harness/eval + producers.**
- **C13 ? benchmark the rule reconciler against a learned boundary refiner** (ASRF/CBR) to *justify* staying rule-based, and ground rules in MonoTrack-style physics/cadence as **soft features, not hard rules** (hard cadence rules would re-introduce special-casing and break on doubles/lets/warm-up). *(MonoTrack arXiv:2204.01899; CBR arXiv:1705.01180.)* ? **reconciler.**

13.5 Pitfalls we are actively walking into (risk register)

- **Over-segmentation hidden by tIoU** ? the dominant error mode of cheap segment decisions; a per-window scorer can look good while fragmenting rallies. ? C4.
- **Late-fusion miscalibration / dominant-cue swamping** (over-confident shuttle CNN, low-discrimination audio). ? C2.
- **LOOCV distributional bias + tiny-N variance** ? promotion gate promotes noise without CIs. ? C1.
- **Stacking leakage** (in-sample learned-producer outputs) ? C3; and **threshold-tuning-on-test** ? keep *every* tunable knob (windowing, merge/NMS, regularization, reconciler thresholds) *inside* the LOVO loop.
- **Proposal-recall ceiling** ? the decision can't recover a rally the windowing missed; measure window recall (AR@AN) separately or you'll misattribute recall loss to the scorer.
- **Report-first decaying into silent rewrite** ? without per-rule lift gating + a confidence gate + idempotency tests, an over-eager rule systematically truncates correct long rallies.
- **Static-tripod transfer gap** ? broadcast cues (camera-switch, replays, crowd audio) that drive the

tennis/soccer baselines *don't exist* on our footage; don't assume their F1s transfer.
- **Detector domain gap inherited downstream** ? WASB/TrackNet/MonoTrack are trained on other rigs; per-frame shuttle accuracy degrades off-distribution and every cue inherits that error. **Quantify detector accuracy on our footage before trusting any cue column** (currently unmeasured).

13.6 Residual gaps & honesty notes

- **Fabrications: none** across ~40 citation checks. Corrected author-lines applied above: CVIU 2009 = **Liu, Huang, Jiang, Xing, Ye, Gao** (not "Kolekar"); SmartTennisTV = **Ghosh & Jawahar**; Sensors 2024 shuttle-fusion = **Hsu, Yu, Cheng**. Metadata fixes: LOOCV-bias is **Science Advances** (10.1126/sciadv.adx6976); ShuttleSet (KDD) = arXiv:2306.04948 vs ShuttleSet22 = arXiv:2306.15664 (distinct); SF-Net evaluates on THUMOS14/GTEA/BEOID.
- **Read before citing** (verified to *exist*, not deep-read; both 2026, concurrent ? don't void novelty but are the nearest precedents): Constraint-Aware Decoding (arXiv:2605.10149, the reconciler end-state analog) and TennisExpert (arXiv:2603.13397, an audio-hits?cluster?filter rally pipeline). PCL-Former (arXiv:2507.06411) ? authorship unverified; cite without asserting authors.
- **Weakest-evidence claims (state plainly):** warm-up-as-soft-feature (novel by *absence of counterexample*, the high from-scratch-validation risk) and the measured-before-apply reconciler bundle (also absence-based).
- **Not yet searched:** a recent badminton full-match frame-level rally/interruption dataset (would relieve scarcity); doubles-specific cue behavior; calibration for *count-valued* cues (net-crossings); whether RLOOCV interacts badly with grouped folds at N=6 (the cited study is record-wise, not group-wise).

12. user (2026-06-16T03:12:21.971Z)

Owned-model implementation plan (roadmap) ? DRAFT for owner review

Status: PLAN / roadmap. No owned-model *implementation* starts until the owner greenlights a specific milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a training source).
Grounded in: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO + learned prototype (`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy workflow (2026-06-09).

Design principle (owner directive): *envision now, build later*
Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be **architecturally envisioned now** so we never hit a mammoth refactor. We **design the seams** (data contracts, export boundaries, the event store) up front and **implement behind them** when each milestone is greenlit. Punting a *feature* is fine; punting its *seam* is not.

North Star (reframed per PR #61 review)
- **Immediate deliverable:** a **training framework + harness** that produces a model with **very high precision/ recall for extracting highlights + rallies from a clip**. *Quality is the first-order objective.*
- **Eventual product:** a **single, sport-agnostic, conversational** video model ?

upload a clip, then ask contextual questions ("longest rally", "make a clip of all service faults", "rallies over 20 shots").
- **On-device is LATER** (a model-compression goal so power users save upload cost/hassle) ? definitely on the roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
- **The moat = the consented dataset + the eval harness, not the weights.**

Hard guardrails (non-negotiable ? "never corrupted")
- **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT / Apache-2.0 / BSD only.** (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-restrictive: Gemma 3 (viral), **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU cap), Commons-Clause repos.
- **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public grounding-CoT datasets that were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision (a real, binding cost).
- **Exclude minors; per-user training data needs a separate explicit opt-in; split data by match.**
- ? **Base-model pretraining provenance is unauditable for *every* open VLM (incl. Qwen3-VL).** Our "no paid-LLM training" rule is enforceable at *our fine-tuning data* layer, not the base's pretraining ? an **explicit owner risk-acceptance** is required to use any open VLM. (Governed by the base's license, not our pipeline.)

The model strategy (verified 2026-06-09)
Framework ? base (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge (runs SFT/GRPO, emits weights); a *base* is the metal you forge. You pick one framework and feed it any clean base.
- **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio + timestamp-grounding + GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core; *avoid its AGPL Studio UI*) as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-time snapshot ? re-verify at impl.
- **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-video context, purpose-built **timestamp emission** (second-level localization), multi-turn video chat. Re-confirm the exact size's LICENSE pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but its Prohibited-Use posture is unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not the primary extractor. **InternVL3 (MIT/Apache)** = clean fallback if a Qwen blocker appears. ? base swap is cheap *only within timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-specific).
- **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss strict temporal boundaries** (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse regions, not accurate boundaries"). **Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-risked on our corpus (held-out AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params (trivially compressible later), and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So: **head now ? VLM later**, strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes, head refines boundaries) over wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur footage may differ ? re-measure.)*

Conversational video-QA ? build the bookkeeping NOW, punt the model
Punt the conversational *feature* until extraction quality is solid (current LOVO ~0.583 ? "very high"; a chat UI
over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn** (expensive; bad at exact
counting; the strong teachers are license-forbidden). **But build the *seam* now** (the brief requires it):
- **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits structured timestamped
  spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-SQL + ffmpeg clip-cut). The
  cited queries ("longest rally", "all service faults", "rallies > 20 shots") are **structured-numeric ? SQL, not a
  VLM job** (SQL counts exactly where VLMs hallucinate).
- **Designed-now seam:** make the **per-video event-index schema a first-class data contract** alongside the M0.1
  corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count, ending_reason/fault_tags,
  derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy (e.g. `service_fault`) up front**
  even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration): `play_segments` today has
  `video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights` isn't a table yet ? so it's an
  extension with new columns + a highlights table, not a no-op. **Punt** the NL parser/agent until quality clears the bar.
- Paid Gemini may answer genuinely open-ended edge questions **at inference** over our structured index (allowed ?
  it reasons over our data, it is not trained on its outputs).

Multisport (single sport-agnostic model)
- **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned across sports (sport = an input
  feature / per-sport calibration). The collector + indexer are already sport-generic. ? **ACTION: rename the repo off
  "badminton" ASAP** (owner emphatic) ? consistent with this.
- **What multisport changes = DATA + DETECTORS (scale up, the accepted "good problem"):** each new sport needs its own
  labeled matches (split by match) + a **clean trajectory detector** to feed the head's features. **The binding blocker
  is per-sport detectors:** WASB (badminton) has the unresolved **C3** checkpoint-provenance issue, and **clean MIT/
  Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet identified.** Sequence by public-data
  richness: **badminton ? tennis/TT ? pickleball/padel.**
- ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not proven ? our corpus is badminton-only,
  and the evidence once cited for it (RacketVision) was *refuted on verification* (it's ball-tracking, not temporal). The
  concept is sound on first principles (rally signal = a racquet-sport invariant) but must be validated once a 2nd-sport
  corpus exists. Don't cite RacketVision as evidence.

Language ? Python everywhere; on-device runtime is a deferred, layer-local export shell
- **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer); offline/batch, no latency hot path.
- **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-process (ffmpeg, WASB-in-WSL, cloud
  GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ? single-digit-ms wins dwarfed by
  ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval harness + provider registry.
- **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate

our dev language:** train in
  Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device shell (Swift/Kotlin/C++/Rust)
  runs the artifact with zero Python. **The one concrete constraint now: train with export-clean ops** so the compression
  target stays reachable without re-architecting. (On-device base stays on the Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
- Escalation order if a real Python CPU hot path ever appears: free-threading ? numpy/vectorize ? Cython/PyO3 ? only then a
  separate service. **None exists today.**

Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)

M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) · *greenlight item*
Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index schema (the conversational seam)
? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end, ending_reason/fault_tags,
shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY MATCH), trajectory_ref,
labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL ActivityNet-JSON · VLM clip?QA),
one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector` (system-of-record; extends PR #13).
Reserve fault/event taxonomy now. Acceptance: round-trip test; split-by-match enforced; **no Gemini-sourced rows in any training view**.

M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-match + LOVO recorded; the
"contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club match ? margin. **Bottleneck =
data + calibration, not method.** Keep growing per new sport.

M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) · *greenlight item*
- **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training violation is
  `backend/eval/distill_local.py` (it trains a classifier on Gemini-silver CSV labels ? `output/local_rally_model.json`)
  and the threshold distillation in `calibrate_local.py`. (NB: `backend/eval/training.py::label_candidates` is a *pure,
  source-agnostic* function ? there is no store by that name; the earlier "training.label_candidates" pointer was wrong.)
  Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the path that regenerates them is one
  command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini = eval/reference/fallback only.
- **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint provenance is a **commercial-ship
  blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a clean MIT/Apache detector swap.
  ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our temporal labels can't train the
  shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies per sport** (incl. any TT-on-WASB use).
  Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).

M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over **ActionFormer (MIT, 1:1
rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not*

the Commons-Clause repo), reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item): a FLOOR of "beat the honest n=6 heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU** target for "very high P/R" (e.g. F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory. Trains on the GPU/WSL box.

Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it becomes the rally/highlight extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no overfitting to declare victory.

Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus + explicit go)
- **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a promising cue here** ? our E1 probe (PR #35) found it weak (~2.2× discrimination, AUC ~0.6) and the foreground-audio idea also failed this session; keep it parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet first).
- **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a temporal-IoU-vs-golden reward, our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on Modal (no-access) / RunPod-Secure.
- **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-hosted VLM vs Gemini) **+ a real VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples use 6?8 GPUs ? **budget is the largest open risk and is currently unquantified**). Don't stand up an owned serving stack until these clear.

Decisions / status
#	item	status
1	Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4 harness**	awaiting owner
2	WASB C3 ? first-class action item (own-weights / clean swap)	**owner: yes** ?
3	Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights)	**owner: yes** ?
4	Collector = corpus + event-store system-of-record	**owner: confirm?**
5	Rename repo off "badminton"	**owner action** ?
6	Conversational-QA: build seam now, punt model	**rec accepted?**

Open risks / unknowns (none block Phase-0 research)
Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R" target undefined · per-sport clean trajectory detectors not yet identified · base-model pretraining provenance unauditable (risk-acceptance) · real VLM VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-train (minors-excluded) for cloud training · on-device 4B latency on 8 GB unmeasured.

Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing discipline holds: the committed next platform slice remains the async job model (single-tenant); only the M0.1?M0.4 foundation is safe to start.

13. user (2026-06-16T03:12:22.222Z)

Rally-detection quality iterations ? living log

Purpose. A running, reverse-chronological log of how we evolve rally-detection quality: each iteration's hypothesis, what shipped, the measured (or expected) effect, and the lesson. This is the **live** continuation of the historical trail in [`archives/quality-iterations/`](archives/quality-iterations/README.md) ? per the archive policy (`CLAUDE.md`), **terminal-state** (DONE/REJECTED) work gets a dated subdir there; this file stays at `docs/` top level and keeps growing, then its entries graduate to the archive when they conclude.

Methodology (non-negotiable). Every quality change rides the experiment harness (EXPERIMENT_HARNESS.md): new cues land **flag-gated, default OFF**; they're graded **offline against golden labels** (per-video + LOVO-honest); promotion to default happens **only on measured lift** through the `run_regression.py` gate; **rollback = flip a config flag, never `git revert`.** Cues are designed in MULTI_SIGNAL_FUSION_PLAN.md; scoring in EVALUATION.md; the 2-layer model in HOW_RALLY_DETECTION_WORKS.md.

Golden corpus = the 6 labelled matches in `output/human_lovo_manifest.json` (owner-confirmed 2026-06-13: use all 6 ? `adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy, testlarge_short, mahadevpura_singles`). The model trained on these is what we run alongside the existing detector on NEW footage for the side-by-side review.

Baselines that drive strategy (IoU 0.5 vs human golden labels)

Path	Mahadevpura (clean-ish)	Kushagra (multi-court)
Heuristic (velocity windowing)	0.657	0.157
Gen-0 owned head (LOVO, n=6)	0.744	0.561
Two-stage WASB+Gemini refine	0.83	?
Gemini wrapper (`gemini-flash-latest`)	0.93	0.66

Reading: clean footage is commoditized (serve via Gemini); **multi-court drops the commodity wrapper to 0.66 ? that is the owned model's ship target.** Baselines tracked in `eval_baselines/`. The owner's observed false positive ? a player **holding/flicking** the shuttle between points counted as a rally ? is the precision bug the fusion gates target.

Current track (live) ? newest first

Served-side Gate-A measurement on the LIVE FusionSegmenter ? the independent run behind the #151 revert *(2026-06-14)*

The owner-gated follow-up flagged by the (now **REVERTED**) SERVED-DEFAULT entry below. An **independent** served-side check ? separate from, and corroborating, the [#148](https://github.com/avidullu/badminton-highlight-indexer/pull/148) litmus + #151's run ? of whether the Gate-A default flip (PR #146) regresses the live path. **Verdict: it does ? the flip was reverted in [#151](https://github.com/avidullu/badminton-highlight-indexer/pull/151); this entry is the detailed measurement.** Driven the **real** served seams (`_enrich_candidate_stats` ? `net_crossings`, `_select_for_handoff` ? the gate) over each golden video's **cached** WASB trajectory, at the **production windowing** `process_video` actually uses (`velocity_thresh=0.02, merge_gap=2.0, min_window=2.0`) ? no WASB re-run (the runner has no cache-hit early-return), no GPU, no Gemini. Config loaded exactly as production (`load_config()`); segmenters from the registry. Repro: `scratch/served_gateA_verify.py` + `scratch/served_gateA_sweep.py` (artifacts in `output/gateA_served_verify/`); the standing productionized guard is `backend/eval/served_gate_a.py` (#148).

- **(a) No-regression invariant CONFIRMED on real served code.** At `min_crossings = 0`, `fusion_hybrid`'s seeding **and** handoff set are **byte-identical to `wasb_hybrid`** on all 6 golden videos (same candidate windows; handoff = all windows = the base seam). The "fusion adds nothing when the gates are off" property is now verified on the *live* served path, not only by code review. ?
- **(b) The +0.074 win does NOT transfer to the served windowing.** The offline promotion (+0.074, 6/6, p=0.031) was measured on the eval harness's **recall-oriented *proposal* windowing** `(0.005, 1.0, 0.5)`, where CONTROL **over-segments** (segment-count ratio 1.68) and Gate-A's job is to clean up that fragmentation (? 1.01). The **served path uses a coarse windowing** `(0.02, 2.0, 2.0)` that already merges aggressively (low over-segmentation), so Gate-A has little FP to remove and instead trims **real** rallies whose WASB trajectory registered only 0?1 net crossings. **Candidate-handoff-level F1 (pre-Gemini, IoU ? 0.5) on the served windowing, n=6:**

min_crossings	mean F1	precision	recall	real-rally windows lost vs off	?F1 vs off
0 (Gate-A OFF = `wasb_hybrid`)	0.300	0.317	0.290	0	?
1	**0.303**	0.327	0.288	1	**+0.003**
2 (shipped default, PR #146)	0.292	0.337	0.261	**12**	**?0.008**
3	0.269	0.338	0.226	28	?0.031

  At the shipped `mc=2`, mean F1 goes **0.300 ? 0.292 (?0.008)** and Gate-A drops **12 windows that match golden rallies** (net_crossings almost all exactly 1.0), concentrated in doubles (`largetest_doubles` ?0.051). All deltas are **tiny / within n=6 noise**, so this is not a catastrophic regression ? but `mc=2` is **not** a served-side *improvement*, and the held-shuttle "guard" is removing genuine rallies, not just feeds.
- **Served relevance (honest).** A dropped candidate window **never reaches Gemini**, so each lost real rally is an **unrecoverable served recall loss**; the precision "gain", by contrast, is partly what Gemini already does at the handoff ? so the served-side tradeoff is at best neutral and plausibly net-negative, i.e. the candidate-level ?0.008 likely *understates* the served downside.
- **Root cause = eval ? serve windowing.** The harness promotes/refuses variants on a windowing the production path does not serve. Gate-A's value is conditional on over-segmentation; the coarse served windowing doesn't over-segment, so the gate inverts. [[decision-rigor-norm]]: the foundational issue (harness windowing ? served windowing) outweighs the single proposal-windowing number.

Outcome ? REVERTED (see the entry below). [#151](https://github.com/avidullu/badminton-highlight-indexer/pull/151) restored the pre-#146 state: `config.json` `indexing.default_segmenter` ? `gemini` (the `indexing.fusion` block removed) and `FusionConfig.min_crossings` default **2 ? 0** ? Gate A is now **OFF by default but still opt-in** (`indexing.fusion.min_crossings: 2`), the persisted-superset invariant intact. Our independent numbers match #151's (mean served ?F1 ?0.008, ~12 rallies dropped, 2/6 losses). If Gate A is ever opted back in on the served path, `mc=1` is the least-bad operating point (?flat F1, 1 rally lost). **Open follow-up:** **align the experiment-harness windowing with the served windowing** (or re-run promotions on the production windowing) so a harness lift actually predicts served behaviour ? the real lesson here. The offline ablation + unit tests remain valid for what they measured.

PERSONALIZATION Phase-0 quality checkpoint ? the eval/safety RAILS for the per-user feature *(2026-06-14)*
The owner adopted the **HYBRID** posture: personalization as a **flag-gated FEATURE on a generalization-first baseline**, paired with a **contribution flywheel** so per-user gains and the shared baseline compound together (full design in PERSONALIZATION_PLAN.md, merged [#144](https://github.com/avidullu/badminton-highlight-indexer/pull/144)). This checkpoint landed the three **honest-eval RAILS** that keep a per-user/small-N tuning layer from promoting noise or silently regressing a user:

- **Per-user promotion gate ? [#141](https://github.com/avidullu/badminton-highlight-indexer/pull/141)** (`backend/eval/per_user_gate.py`): a **min-N floor** + **structural-guard exemption** ? per-user/small-N tuning can't promote noise and can't disable a structural guard (the proven Gate-A held-shuttle guard stays on regardless of a thin user corpus).
- **Held-out-USER learning curve ? [#142](https://github.com/avidullu/badminton-highlight-indexer/pull/142)** (`backend/eval/per_user_eval.py`): turns the vague "<K videos to superior quality" claim into a **falsifiable release number** (the `smallest_k_with_lift` measured on held-out users).
- **Per-user no-regression guard ? [#143](https://github.com/avidullu/badminton-highlight-indexer/pull/143)**: a retrain/baseline-upgrade can **never make a user worse than their incumbent** (frozen per-user CONTROL; a regressing candidate is refused).

Read HONESTLY: these are the **eval/safety rails** for the personalization feature ? **not** new measured rally-detection wins. No F1 moved this checkpoint; what landed is the discipline that lets per-user tuning ship without regressing the baseline or over-claiming on n?6-per-user data.

Locked owner decisions (2026-06-14): identity = *design for the north-star (multi-user/cloud), implement for current use (single-user local)*; the inline per-user classifier sits **behind a cloud-load interface** (local store = one impl); **`min_n` is a PROMOTION-confidence floor, NOT a service/segmentation gate** (a one-video user is ALWAYS fully segmented by the baseline batteries); **free-tier videos ARE pooled** into the shared baseline (consent baked into free terms, minors-exclusion enforced at the pooling query); and the previously-pending structural-guard sub-decision is now LOCKED ? **Gate-A-only structural + `min_n = 5`**.

Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio = promising (not promotable) *(2026-06-14)*

The GPU person-feature extraction finished all 6 golden videos (`fusion_golden.py` ? `output/*_golden_features.json`, the replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats (C1 ? bootstrap 95% CI + paired sign test; never a bare mean at n=6)**:

Variant (honest LOVO, IoU ? 0.5)	F1	?F1 vs prior	95% CI	sign test	CI clears 0
shuttle-only	0.307	?	?	?	?
+ person (P3)	0.286	**?0.021** (?6.7%)	[?0.165, +0.162]	2W/4L (p=0.688)	**No**
+ person + audio (P4 incr.)	0.366	**+0.079** (+27.7%)	[?0.054, +0.203]	4W/2L (p=0.688)	**No**

Read honestly: neither cue clears the promotion bar (both CIs span 0, both sign tests n.s.) at n=6 ? **both stay default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ? `players_in_court` counts spectators); per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising lead ? +0.079 / +27.7%, wins 4/6 (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands **above** shuttle-only (0.307) ? but n=6 can't confirm it. **Next:** more matches (raise N until the sign test can resolve ~+0.08), and measure audio on shuttle-only *directly* (without the person handicap). Recorded in `eval_baselines/experiment_leaderboard.json`. **Negative/uncertain results are kept on purpose ? this is the honest ablation backbone for the paper aim.**

Gate-A served-default flip ? **REVERTED (eval?serve regression)** *(2026-06-14)*

The #146 flip of Gate-A (net-crossings ? 2) to the live served default was REVERTED. A served-side check ? the repeatable litmus in [#148](https://github.com/avidullu/badminton-highlight-indexer/pull/148) (`backend/eval/served_gate_a.py` + `tests/test_served_gate_a_regression.py`), corroborated by an independent served run ? found Gate A is **not a win on the served path**: on the production (COARSE) windowing it is **neutral-to-negative, mean served ?F1 ?0.008**, dropping ~12 real rallies across the golden 6 (2 losses, 0 clear wins). It is a **confirmed served regression**, so the owner directed a revert to restore the known-good state.

Why eval ? serve (the honest root cause). The +0.074 below is a property of the **offline harness's permissive PROPOSAL windowing** (`velocity_threshold=0.005, merge_gap=1.0, min_window=0.5`), which emits many short candidate windows ? leaving abundant held-shuttle junk for Gate A to cut. The **served COARSE windowing** (`velocity_threshold=0.02, merge_gap=2.0, min_window=2.0`) already merges flight bursts into a few long windows and drops short ones (served over-seg ratio already **< 1**, i.e. under-segmentation), so there is little junk left to cut and the gate instead occasionally trims a borderline true window ? slightly negative ?F1. The gate *itself* is correct (#148's control test reproduces +0.074 / 6-of-6 / p=0.031 exactly under proposal windowing); the **operating point is what differs**. This is the canonical eval?serve trap.

What the revert restores (back to the pre-#146 no-regression state):
- `config.json` ? `indexing.default_segmenter` **`fusion_hybrid` ? `gemini`** (the prior

served detector); the explicit `indexing.fusion` block added by #146 is removed.
- `backend/config/models.py` ? `FusionConfig.min_crossings` default **2 ? 0** (Gate A **OFF** by default).
- Tests (`tests/test_config.py`, `tests/test_fusion_hybrid.py`) restored to the Gate-A-OFF defaults; the **persisted-superset invariant is preserved** (gating only ever filtered the AI handoff, never persistence).
- `eval_baselines/experiment_leaderboard.json` ? the #146 `promotion` record carries a `reverted (eval?serve)` note.

Gate A is NOT deleted ? it remains opt-in (`indexing.fusion.min_crossings: 2`) for footage with more held-shuttle/service-feed false positives than the golden 6, and the offline ablation evidence below stands (it is a real proposal-windowing win). The path back to serving it is to **align the served windowing toward the proposal operating point** and re-run the #148 litmus to confirm the lift transfers. The #148 litmus is now the standing guard: it pins the honest measured behaviour (served ~neutral; +0.074 only under proposal windowing) and trips if a future windowing change silently revives or breaks either fact.

Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125 corroboration *(2026-06-14)*
Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness). The measured win (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg 1.68?1.01 ? the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md)) clears the honest bar. **Recommended config:** `indexing.fusion.min_crossings = 2` on the fusion path (the eval-only Gate A folded into production, MULTI_SIGNAL_FUSION_PLAN §3.1). ? The offline win is on the recall-oriented *proposal* candidate set; flipping the **live served default** stayed owner-gated at the time of this entry. **Update: the flip was attempted (#146) and then REVERTED** ? on the served COARSE windowing Gate A is neutral/negative (?0.008), an eval?serve regression; see the REVERTED entry above. Gate A stays opt-in (`indexing.fusion.min_crossings: 2`), default OFF. **Corroborated on real footage:** offline re-score of GX010125_1080p's *cached* trajectory (no GPU, isolated `output/GX010125_run/`) ? 45 candidates ? CONTROL 45 rallies vs **Gate-A 27**; Gate-A dropped **18 low-crossing windows (~1.7 min)** of likely held-shuttle/feed FPs (no golden labels for this footage ? divergence, not F1).

Learned variant FIXED ? [#130](https://github.com/avidullu/badminton-highlight-indexer/pull/130). The first A/B's learned logistic lost to CONTROL and zeroed `testlarge` ? a fixed-0.5-cutoff artifact. Added additive/default-OFF `Variant.tune_threshold` (operating point chosen on the **train** fold) + `calibrate` (Platt/isotonic, C2). Validated on the n=6 golden set: **mean F1 0.307 ? 0.423 (+0.116)**, testlarge 0.000 ? 0.300, and the learned variant now **beats CONTROL (0.388)**. Threshold-in-LOVO is the big lever; calibration is available too.

Next: a real fusion-path run to confirm Gate-A served-side; per-video output layout (PER_VIDEO_OUTPUT_LAYOUT.md) so each video's artifacts are self-contained.

First end-to-end A/B on the golden corpus (n=6) ? **ARCHIVED** *(2026-06-14)*

First real use of the harness + C1?C4 helpers, 100% offline (re-scored persisted WASB trajectories, no GPU). **Gate-A (net-crossings ? 2) is a measured win** ? ?F1 +0.074, 6/6 matches, sign-test **p = 0.031**, CI [+0.042, +0.104], over-segmentation (segment-count ratio) **1.68 ? 1.01**. The window-level 5-feature logistic **did not clear the bar** (?0.081, 2W/4L; a threshold/calibration artifact ? *not* the stronger per-frame prototype). The framework held up: **C1 promoted one variant and refused another; C4 surfaced over-segmentation tIoU hid.** Full record + tables ? [`archives/quality-iterations/2026-06-first-ab-experiment/`](archives/quality-iterations/2026-06-first-ab-experiment/README.md). Next: Gate-A is a candidate for the no-regression promotion gate; the logistic failure is the C2 (calibration) + threshold-in-loop task list.

Eval-honesty helpers (course corrections C1?C4) ? additive, default-OFF *(2026-06-14)*

Why. A multi-agent literature review (folded into ANALYZERS_AND_RECONCILERS.md §13) confirmed the framework's *skeleton* is standard prior art (Temporal Action Localization/Segmentation + late/score-level fusion ? cite, don't claim), and flagged that our **measurement** is walking into known small-N traps that would make "promote only on measured lift" promote noise or hide a real win. Four corrections, shipped as **pure, additive helpers** (no existing `compare`/LOVO/`metrics.score` behaviour changed ? callers opt in):

- **C1 ? never a single mean.** `backend/eval/eval_stats.py`: `mean_with_ci`/`bootstrap_ci` (deterministic, seeded), `paired_sign_test` (the exact distribution-free "won on enough *videos*?" test the corpus already needs), `paired_delta_ci` (B?A with a CI that must clear 0). At n?6 a lone LOVO mean is not trustworthy.
- **C2 ? calibrate cue confidence.** `backend/eval/score_calibration.py`: Platt + isotonic (PAVA) calibrators + `brier_score`/`expected_calibration_error`, so a CNN softmax, an audio onset, and an integer net-crossing count are comparable before the arbiter and the over-confident cue can't swamp the rest.
- **C3 ? honest stacking.** `eval_stats.out_of_fold_predictions`/`grouped_folds` (leave-one-group-out by match) so a *learned* producer's outputs used as arbiter features are out-of-fold (no leakage).
- **C4 ? over-segmentation-visible metrics.** `backend/eval/segmentation_metrics.py`: `f1_at_overlaps` (F1@{10,25,50}), `average_precision`/`mean_average_precision` (mAP@tIoU), `segment_count_ratio`. tIoU alone is blind to a rally split in 3 or two rallies merged ? exactly what the reconciler exists to fix.

Status. Helpers + 24 unit tests landed (pure stdlib, run in the numpy/GPU-free lanes); ruff + mypy clean. **Not yet wired into the default harness flow** (wiring `compare`/`run_regression.py` to report CIs + F1@k is the follow-on, owner-gated). The discipline they encode is the methodology for every future cue measurement ? see §13/C1?C4 for the citations and the remaining (Tier-2/3) corrections.

Multi-signal fusion P2: the FusionSegmenter *(in progress, 2026-06-14)*

Hypothesis. Adjudicating shuttle-seeded windows with the net-crossing **Gate A** (`min_crossings?2`) and a person-occupancy **Gate B** raises **precision** on the held-shuttle / warm-up / one-sided-feed / spectator false positives ? *without* hurting recall, because the shuttle stays the primary, camera-invariant seed.

Scope ? and an honesty boundary from the archive (read this). The [`2026-06-multivideo-lovo`](archives/quality-iterations/2026-06-multivideo-lovo/README.md) iteration already smoke-tested **court-region detection as a fix for the *multi-court contrast* confound** and found it **low-headroom**: only 12?14% of dead-time shuttle detections fall *outside* the foreground court, so a perfect court gate lifts contrast to only ~1.6?2.2× (below the ?3.6× "works" threshold) ? the multi-court problem is **temporal**, and the **learned model**, not a court mask, is its answer. So Gate B here is **NOT** sold as the multi-court fix. It targets a **different, previously-untested** failure mode (held-shuttle/warm-up/one-sided/spectator FPs). Two distinct levers; don't conflate them.

Data support for Gate A. The same archive shows real rallies on the multi-court matches contain **?2 net-crossings 97% of the time (median 5)** while a single service feed crosses ~once ? `min_crossings?2` is a principled, data-backed discriminator (it's the held-shuttle owner-bug killer).

Shipping shape. A NEW `fusion` segmenter, **default OFF** (`default_segmenter` unchanged); Gate A wired into the live trajectory path behind `indexing.min_crossings` (default 0 = OFF ? wasb/tracknet byte-identical today); the court mask is an **optional** config polygon (omit ? Gate B is player-count-only; supply ? masked). Person features (`players_in_court, two_sided_motion, mean_player_motion, in_court_ratio, shuttle_player_colocation`) persisted into candidate `stats` for P3's learned fusion. Reuses `PersonDetector` (P1) + `rally_gate` (already existed) ? no reinvention.

Status (2026-06-14): SHIPPED, default-OFF. `FusionSegmenter` (`backend/pipeline/segmenters/fusion_hybrid.py`, registry key `fusion_hybrid`) + the pure feature math (`backend/pipeline/detectors/fusion_features.py`) + two identity hooks on the trajectory engine (`_enrich_candidate_stats`, `_select_for_handoff`) + `PersonDetector. detections_per_frame` + the `FusionConfig` block. Built design-first (a `fusion-p2-design` fan-out workflow), then **adversarially reviewed by a 31-agent workflow (25 findings ? 19 confirmed)**: the headline confirmation is **NO-REGRESSION verified** ? wasb/tracknet twins persist byte-identical stats and hand off all windows (pinned by the extended equivalence test); default `FusionSegmenter` == its wasb substrate. The confirmed low/medium findings were fixed (frame-dim guards, `round()` frame indices, the per-window axis-estimate redundancy ? a reusable `rally_gate` estimate, honest torch-import docstrings, and 6 new tests closing the person-detector / Gate-B / substrate / colocation-halo gaps). Suite 441?**467 green**; ruff + mypy clean. **Two findings deferred with intent** (reviewer-agreed): person features are computed eagerly even for Gate-A-pruned windows (a P3 redesign, would change the persisted-superset semantics); and `fusion.velocity_threshold` must be set equal to the substrate's to A/B against a tuned wasb run (documented in

`FusionConfig`).
Lift still to be measured offline on the 6 golden videos via `experiment.compare(CONTROL, fusion)`; the
held-shuttle-drop + person-feature values are GPU/owner-side. *(Measured numbers appended here when run ? no claims before.)*

Person cue extracted to a reusable producer (fusion P1) ? [#115](https://github.com/avidullu/badminton-highlight-indexer/pull/115), 2026-06-13
Lifted the torchvision detector + ByteTrack + velocity windowing out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py::PersonDetector`. **No behavior change** (proven by the
unchanged `process_video` output tests); `yolo_hybrid` now orchestrates only. **Why it's a quality move:**
it makes "add the person cue to fusion" an *add-a-producer*, not a segmenter rewrite. Suite 440?441 green.

Experiment harness: A/B without rollback risk ? [#114](https://github.com/avidullu/badminton-highlight-indexer/pull/114), 2026-06-13
Shipped `backend/eval/experiment.py` ? the thin variant/experiment layer (the audit's "~80% already
exists" held; pure glue over `metrics`/`calibration`/`classifier`/`training`/`regression.gt_hash`). Gives
`Variant` (+ pinned `CONTROL`), `compare()` (labeled A/B, LOVO-honest), **`compare_unlabeled()`** (the SxS
read on NEW unlabeled footage ? agreed / A-only / B-only), and a JSON experiment leaderboard. **This is
the instrument every iteration is measured on.** 19 tests incl. the no-regression invariant (all cues OFF
? byte-identical to control). Suite 421?440 green.

Archived predecessors (the historical trail)
Completed iterations live under [`archives/quality-iterations/`](archives/quality-iterations/README.md):
- **`2026-06-initial/`** ? seed of the quality phase.
- **`2026-06-human-gt-tuning/`** ? first eval vs **human** GT on a partial-label video; partial-label
  harness + labeled-window methodology; baseline 0.650 ? tuned 0.742 (*fit-on-self*); found+fixed a
  negative-start ingestion bug (collector PR #13).
- **`2026-06-multivideo-lovo/`** ? the **n=6 corpus + honest LOVO**; established the heuristic is
  footage-fragile (multi-court Kushagra 0.16 / gBaaddy 0.28), the **"contrast" metric** predicts
  heuristic F1 monotonically, and a **learned per-frame prototype overtook the heuristic at n=3 and holds
  ~+0.10** (n=6: learned 0.583 vs heuristic 0.480), with **AUC 0.83?0.92 decoupled from contrast**. This
  is the data-backed green light for the owned model.

Standing lessons (carry-forward)
- **Persist detection once, re-score variants forever.** Expensive/risky = per-frame detection (GPU/WSL);
  cheap/swappable = the decision. Most experiments never enter the served pipeline.
- **LOVO or it didn't happen.** Score on a held-out *video*, never a held-out window from the same video.
  Group by **match**, not file. Sign test needs n ? ~n=10/9-wins for p<0.05.
- **The multi-court confound is TEMPORAL, not spatial/loudness** ? three heuristics were tested and failed
  for the same reason (foreground spatial gate **?0.04**; audio energy **AUC ~0.6**; court-region

separability **12?14% out-of-region**): the dead-time confusion is rally-vs-non-rally activity in the
same place and frequency band. Only the learned model's temporal/structural features crack it (AUC
0.83 on Kushagra/gBaaddy). **Don't re-derive these dead ends.**
- **Court-masking ? multi-court fix** (see above) ? it's a **precision** tool for off-court persons and
one-sided feeds, a *different* lever; measure its lift, don't assume it.
- **Net-crossings are a strong rally signal** ? 97% of real rallies cross ?2× (median 5); single feeds ~1×.
Gate A leans on this.
- **Small-N feature discipline.** Few, scale/translation-invariant features (counts/ratios, not raw
coords) + a logistic regression, not a net ? raw coords memorize this court/camera.
- **Calibration, not representation, is the small-N wall** ? learned probabilities separate (high AUC) but
the operating point doesn't transfer with few training videos. More distinct matches is the highest-ROI
unblock.
- **Gate A already existed, just wasn't wired to production** ? the cheapest precision win is plumbing, not
new capability. Always check what the eval drivers already do that the live path doesn't.

14. user (2026-06-16T03:12:22.180Z)

Next steps (live) ? owned-model + multisport

Updated: 2026-06-13 (**rally-detection quality track**: fusion + experiment-harness design docs merged,
owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior: 2026-06-10 (audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-first delivery ? see
`docs/UI_ALPHA_EXECUTION_PLAN.md`; **PR-1 async jobs (#16) MERGED 2026-06-11, PR #87**; **PR-2
upload/download MERGED 2026-06-11, PR #99** ? live-E2E-verified (3-part chunked upload MD5 byte-identity +
compile?browser-download); next: PR-3 identity. Same day: the **8?9 quality sweep #90?#98 landed** ? ruff/mypy/
coverage-floor CI lanes, SQLite conn-close fix, config unknown-key warning, hybrid-twin collapse into
`trajectory_hybrid.py`, WSL tilde+abspath live-path fixes; suite 388 green; LOVO 0.626 re-verified exact).
Authoritative roadmap:
`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+ ADR-008 for topology). **Latest blow-by-blow:** memory
`current-state.md` / `session-handoff.md`. This file = the prioritized "what to do next," refreshed each session.

? Next-session / GPU-box pickup (2026-06-15)
M0 in-container quick wins are COMPLETE: ship-gate provisional blocker (**#170**), train/eval split guard (**#171**), the **doc-consistency sweep** (**#174** ? corrected 16 drifted docs vs the code), and the **P3 person-feature fusion litmus machinery is already present + suite-green** (`backend/eval/fusion_compare.py` + `ablation.py`: person-driven ?F1 / bootstrap CI / paired sign test; the real-golden `skipif` litmus waits only on GPU-extracted data). ? CI is red repo-wide due to a **GitHub Actions minutes/billing outage** (account-level, not code) ? work is local-gated + `--admin`-merged until Actions is restored (Settings ? Billing ? Actions minutes / spending limit).

Immediate next step ? the real P3 LOVO measurement, split GPU?CPU (the dev box is CPU-only: `torch+cpu`/`cv2` present but no GPU, and the corpus/videos/CSVs/labels are not here):
1. **[owner / GPU box]** `python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-

data\features\2026-06-15-n6"` ? produces `*_golden_features.json` for the 6 golden videos. (GPU + WSL + videos + trajectory CSVs + `.rallies.csv` labels are owner-side only.)
2. Artifacts land in the **shared cloud folder** `C:\Users\avidu\OneDrive\rally-annotator\golden-data\` (OneDrive auto-syncs both ways). Convention + workflow: **[`GOLDEN_DATA_SHARING.md`](GOLDEN_DATA_SHARING.md)**; tracked in **#175**.
3. **[CPU dev/agent session]** `python -m backend.eval.fusion_compare --features-dir <shared>\features\2026-06-15-n6 --record` + `python -m backend.eval.ablation --features-dir <shared>\... --granularity group` ? real person-feature ?F1; copy the JSONs into `output/` to also pass the `skipif` litmus tests (`tests/test_ablation.py::test_litmus_reproduces_gate_a`).

Also queued (owner-gated / pending input):
- Ship-gate target calibration ? **#172** (owner decision; needs corpus growth).
- Tests reorganization ? blocked on the owner's grouping metric (provide it ? an agent will do it).
- Restore CI ? the Actions outage is account-level (billing/minutes), not a repo/config bug.

Leaving (bulky / M2 ? no start without owner go): ActionFormer (M0.4), event-index DB migration (M0.1), Gemini-silver purge (M0.3), cost-to-Gen-2 benchmark + ByteTrack deprecation, multisport, repo rename, PMF execution.

Where we are (one paragraph)
ADR-008 ratified ("repo split driven by productionization, not isolation"): training lives in-repo (`training/`) behind a CI-enforced import wall; the featurizer is single-sourced (`backend/features/rally_seq.py`, train==serve); the **Gen-0 harness ships** (`python -m training.gen0.harness --manifest output/human_lovo_manifest.json --floor eval_baselines/heuristic_lovo_n6.json --version <semver>`) and produced artifact v0.1.0 (LOVO **0.626**, floor passed, sign test 5/6 wins p=0.109 ? honestly not significant, ship=False). The **n=6 owned corpus is in the collector** (262 rallies, Proprietary, commercial export = owned data only after the ShuttleSet reclassification). The **Gemini battery is done** (see table) ? the moat is quantified.

The quality table that now drives strategy (IoU 0.5 vs human golden labels)
Path	Mahadevpura (clean-ish)	Kushagra (multi-court)
Heuristic (velocity windowing)	0.657	0.157
Gen-0 owned head (LOVO, n=6)	0.744	0.561
Two-stage WASB+Gemini refine	0.83	?
Gemini wrapper (`gemini-flash-latest`)	**0.93**	**0.66**

Reading: clean footage is commoditized (serve it with Gemini for cents). **Multi-court drops the commodity wrapper to 0.66 ? that is the owned model's ship target** (its FPs are background-game pickups + dead-time merges, the exact contrast-metric confound). Gen-0 is 0.10 behind with only n=6 ? a corpus-growth-sized gap, not an architecture-sized one. Two-stage refine = the cost-efficient serving path for LONG tapes (uploads candidate clips only); wrapper 503-flakiness (observed all session) makes the provider-fallback registry load-bearing.
Baselines tracked in `eval_baselines/` (heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json).

Rally-detection quality track (design merged 2026-06-13, owner-gated; implementation in progress)
This is the track a desktop/local agent is slated to pick up. Design docs landed (PR #112 + session). All implementation is **owner-gated** (per explicit in the plan); each step = ONE PR off `master`.

Hardening loop (T1-T5 positive tests + audit + finale) COMPLETE (2026-06-14, PRs #156/#157/#160/#161/#163 + #165 + final records). See archived `docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md` (Living Log with every pre/post/review) and `docs/archives/HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md` (full rules + STATE). Top-level now pointer stubs per #165 review. All 5 items + verification + archive done; clean slate.

- **Docs:** `docs/MULTI_SIGNAL_FUSION_PLAN.md` (shuttle + person + audio, one fusion layer) ·
  `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) ·
  `docs/ROORA_LABELING_GUIDELINES.md` (v8) · `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model).
- **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a rally" bug is **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already implements the net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills service-feed/held-shuttle windows, but it is **only called from eval drivers** (and there was no `min_crossings` knob in `IndexingConfig`). **Wiring Gate A into the live segmenter (now via fusion_hybrid) was the single cheapest, highest-confidence quality win.** (P2 FusionSegmenter specifics: registry `fusion_hybrid`, **default-OFF**, seeds from substrate, persists `net_crossings` + optional person features, optional served Gate A/B via config knobs; adversarially reviewed; suite green. See `tests/test_served_gate_a_regression.py` for the honest finding that on production windowing Gate A is neutral/negative in some cases, hence kept opt-in with knob for safety, and the leaderboard revert note.)
- **Current status (2026-06-14):** P1 (person cue extraction to reusable `PersonDetector`) + P2 (FusionSegmenter + Gate A signal + served handoff gating) + experiment harness P1 DONE (adversarial review + NO-REGRESSION; suite green). **P3 next (this PR + follow-ups):** learned fusion (person features ? harness `compare` LOVO lift on the 6 golden videos; eager-person-compute optimisation); then P4 audio via hit_detector.
- **Pick-up order (each = ONE PR off `master`, owner approval first, flag-gated default-OFF, promote only on measured LOVO):**
  1. (P2 first-step, largely done via fusion_hybrid) Wire Gate A into production served path + config knob (min_crossings); apply in hybrids at runtime. (Now available opt-in; served neutral on production windowing per litmus ? kept opt-in with knob for safety.)
  2. P3 first step (this work item): harness `compare` LOVO litmus for person-on variant (players_in_court + two_sided + colocation etc. from `fusion_features`) on the 6 golden; add the eager compute optimisation note/landing. Pure + harness (testable here).
  3. P3/P4 continuation: audio cue integration + full learned promotion if lift proven.
- **Discipline (from EXPERIMENT_HARNESS.md):** every new cue **flag-gated, default OFF**; promote to default **only on measured LOVO lift** through `run_regression.py` (exit 0/1/3); pinned `CONTROL` ? rollback = config flip. Harness re-uses ~80% existing code.
- ? Dev container no GPU/torch/cv2 ? Gate A, person cue logic, harness, and pure fusion paths are testable here; real-video / golden LOVO / ablation confirmation is owner-side (GPU/WSL machine).

Discipline reminder: owner approval before starting any owner-gated item; one PR per step; babysit the PR (poll/fix/reply) until merge observed, *then* advance.

Prioritized next steps
1. **[owner, in progress] GROW THE GOLDEN CORPUS** ? the single highest-leverage move; every lever below feeds on it.
   Priority order for new videos: **(a) multi-court badminton** (the target distribution AND where the ship target
   lives), **(b) ?3 matches per new sport ? table tennis first** (WASB transfers at F1 0.909; pickleball labels are
   corpus-gold but not trainable until a clean MIT/Apache ball detector exists), (c) capture variety per
   `VIDEO_RECORDING_GUIDELINES.md`; **adult sessions only for the training pool** (consent guardrail). Per video:
   label ? `curate import-local --normalize --license Proprietary --fps <true fps>` ? re-run the Gen-0 harness.
   The paired sign test needs n: at n=10 with 9 wins, p?0.011 ? significance is reachable this month.
2. **[owner decision] Set the ship-gate target** ? now informed: floor = heuristic 0.480 (necessary), **multi-court
   reference = wrapper 0.66** (the number that makes the owned model worth serving), clean-footage ceiling = 0.93
   (not the target ? Gemini serves that tier). Suggested shape: "LOVO F1@tIoU0.5 ? 0.70

on multi-court folds with
    paired-sign p<0.05 vs heuristic" ? owner to ratify/adjust.
3. **M0.4 next increment ($0/local):** swap the Gen-0 LogReg for **ActionFormer/ASFormer** (MIT, minutes on the
    2070S ? still lightweight, NOT the heavyweight Gen-2 stack) inside the existing harness; per-video threshold
    calibration (the LOVO-vs-oracle gap is visible per fold in the harness output).
4. **Multisport thread:** ingest Roora pickleball/TT CSVs (`import-local --normalize`); merged-prototype LOVO across
    badminton+TT once TT has ?3 matches; identify a clean pickleball ball detector. ? TT-on-WASB serving stays gated
    by C3 (provenance multiplies per sport).
5. **PMF validation (cheaper than any engineering month):** C13 academy interviews (+ the two added questions:
    "multi-court footage nobody watches?" / "pay per-match to index it?"); camera pilot at the warmest 2-3 academies
    (consent at capture; demo served via the Gemini path or human-polish ? the reel pattern exists:
    `output/MahadevpuraSingles_highlights.mp4`). **Field kit COMPLETE (2026-06-12, owner ratified the
    physical-visit approach; reframed product-feedback-first per owner spec):**
    `docs/PRODUCT_FIELD_ASSOCIATE_FLYER.md` (the recruiting flyer ? owner fills ?/contact placeholders and recruits) +
    `docs/FIELD_VISIT_PLAYBOOK.md` (the hired associate's visit playbook: coach + student script, demo-after-Q6
    rule, ground rules, Phase-2 record-and-show-back stretch goal) + `docs/FIELD_VISIT_ANSWER_SHEET.md` (per-visit
    capture form incl. demo-reaction + student blocks) + `docs/PMF_FIELD_SIGNALS.md` (G/A/R anchors + **pre-registered
    decision rules** ? PROCEED / PIVOT A-C / FALSIFIED thresholds fixed before the first visit).
6. **Housekeeping:** ~~rotate the Gemini API key~~ **DONE 2026-06-10** (owner-verified working; confirm old-key
    deletion in AI Studio); **rename repo off "badminton"** (owner action,
    in BACKLOG); collector=SoR confirmation; off-peak niceties: none pending ? the battery completed.

Pointers / artifacts
- Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Topology: ADR-008 (`docs/archives/decisions/DECISIONS.md`) ·
  Study: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md` · Quality history:
  `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
- Harness: `training/gen0/harness.py` (one command ? train ? LOVO ? gate ? artifact bundle; weights gitignored,
  manifest reviewable). Featurizer: `backend/features/rally_seq.py` (FEATURE_SCHEMA v1).
- Corpus: collector DB (system-of-record, 6 matches/262 rallies) · eval manifest `output/human_lovo_manifest.json`
  (gitignored) · durable trajectories + Roora files: `C:\Users\avidu\Projects\Annotation Setup\Collect\Trajectories\` ·
  labels: `...\Collect\Golden Labelled\`.
- Gemini ops learnings: serial uploads (`indexing.ai_max_workers=1`) + chunk re-encode
  (`indexing.ai_chunk_scale_width=1280`) + 6-attempt exponential generate retry ? a cold/prepaid project burst-throttles
  with a MISLEADING "credits depleted" 429; ramp gradually.
- ? No owned-model implementation beyond greenlit M0.x until owner greenlights the next milestone. Committed next
  PLATFORM slice = async job model (single-tenant).

15. user (2026-06-16T03:12:22.648Z)

Personalization Plan ? a per-user feature on a generalization-first baseline

Status: DESIGN ? owner-adopted (hybrid); Phase-0 engineering landing. This is the authoritative

plan for the personalization track. The detailed strategy analysis + the full pros/cons that led here
live in the working artifacts `scratch/PERSONALIZATION_PIVOT_ANALYSIS.md` (the decision-grade
counter-case) and `scratch/PERSONALIZATION_FEATURE_DESIGN.md` (the code-grounded design); this doc is
the checked-in summary + decisions + phased build plan. Peer to
OWNED_MODEL_IMPLEMENTATION_PLAN.md,
EXPERIMENT_HARNESS.md, and MULTI_SIGNAL_FUSION_PLAN.md.

0. Thesis & strategic rationale (why *hybrid*, not personalization-first)

Foundation principle (owner): *"a tool that gets better the more you help it."* Contribution
(data / video / diagnostics / annotation) improves BOTH the user's own model AND the shared baseline;
payoff scales with contribution.

A directional pivot ? *personalization-first* (ship a baseline that continually self-optimizes to one
user's own footage) ? was researched in depth. The verdict was **HYBRID, not all-in**:

- **Adopt personalization as a flag-gated FEATURE on a generalization-first baseline.** The harness
  genuinely makes per-user *tuning* near-free (it is already a corpus-relative A/B engine; a per-user
  corpus is just a special case). This is real and worth shipping.
- **Do NOT bet the strategy on per-user *learning*.** Per-user A/B is the n?6 small-sample regime
  worse (sharding data when you should pool); the differentiated parts (a learned per-user model
  serving path, the correction?label write path, a user/tenant dimension) are largely unbuilt; a
  naively-overfit per-user model loses the cross-user data flywheel and has no shared baseline to
  regress against.
- **The owner's contribution-flywheel + free-tier-pooling closes the one real hole** the counter-case
  found: by having (free-tier) users contribute to the *shared* baseline, the compounding cross-user
  moat is preserved *alongside* personalization ? personalization **and** flywheel, not either/or.

Net: a generalization-first **baseline** ("batteries"), a thin **per-user tuning** layer gated by the
honest small-N statistics (with a hard floor + structural-guard protection), and a **contribution
loop** that feeds both.

1. Owner decisions (locked 2026-06-14)

1. **Identity** ? auth WILL land; *design for the north star (multi-user / cloud), implement for
   current use (single-user local)*. `user_id` nullable now (NULL = local install), device/install
   UUID for the alpha; real auth + a `users` table later.
2. **Per-user model storage** ? inline `classifier_json` in the (local) DB now, but **behind a load
   interface** so that post-auth it is fetched from the **cloud-hosted DB run by the service** that
   generates base models and orchestrates the product. The local store is one implementation of that
   interface.
3. **`min_n` is a PROMOTION-confidence floor, NOT a service/segmentation gate.** A

brand-new user with one video is ALWAYS fully segmented by the baseline batteries ? never denied, never "give me more videos first." `min_n` only gates whether a *per-user* A/B is trusted enough to OVERRIDE a baseline cue default *for that user*; below it (or if the CI / sign test don't clear) ? keep the baseline. **Training-corpus policy:** *no more than needed, no less than all-if-few* ? fit on the `smallest_k_with_lift` videos (the held-out-USER learning curve), else ALL available.

4. **"Flip Gate-A to default"** = ship the validated Gate-A held-shuttle guard as the **default detection path** (today it lives in the opt-in `FusionSegmenter`, which is not the default segmenter). Changes served behavior ? **gated on a golden-set regression run + owner OK**; still open (see §5).
5. **Flywheel = backend property** (not in-product marketing). A **free tier** is likely and **free-tier videos ARE pooled** into the baseline (consent baked into the free terms; minors-exclusion still enforced) ? the clean value-exchange that feeds the owner-trained base model. Paid tiers get stronger privacy / opt-out.

Pending sub-decision: structural-guard scope ? recommended **Gate-A only** (the proven structural held-shuttle guard; its value is in failure modes a user's corpus may not yet contain), with **Gate-B** (person-occupancy / `min_players`) left *subject* to per-user A/B; **`min_n = 5`**. **Gate-A-only structural, `min_n = 5` ? owner-confirmed 2026-06-14.**

2. Architecture

Two-layer decoupling (already real). Detection (GPU, run once): `FusionSegmenter` persists the **full superset** of per-window features into `candidate_segments.stats`; only the served handoff is gated. Decision (CPU, swappable): `experiment.Variant` + `predict()` re-score persisted features. **Personalization == a per-user `Variant`** (cue_flags + `min_crossings`/`min_players` + optional LogReg + threshold). No segmenter rewrite ? a per-user `FusionConfig` overlay resolved upstream.

The single highest-leverage wiring: `fusion_hybrid.py::_select_for_handoff` is the production decision seam. When a per-user model is pinned AND the promotion gate says ON, apply `LogisticRegression.predict_proba` over `window_stats` (the same math `experiment.predict()` runs offline). The **load half of the bridge already exists** ? `backend/pipeline/personalization/serving.py::load_user_classifier` rehydrates the stored `classifier_json` (and `resolve_user_fusion_config` overlays the per-user `fusion_config`) from the v5 store ? but it is **not yet wired into the served path**: `_select_for_handoff` still reads only the static `idx_cfg["fusion"]`. Closing that one seam (the first change to served behavior, so owner-gated) is the remaining #1 wall.

Per-user data model (migrations follow the `PRAGMA user_version` ladder; current = v5):

- **v4** ? ? **shipped.** Nullable `user_id` on `videos` + `candidate_segments` (NULL = local;

byte-identical to today).
- **v5** ? ? **shipped.** `user_models` store: `(user_id, version, baseline_version, feature_schema_hash, fusion_config, classifier_json, promotion_evidence, n_videos_at_fit, is_active)`. Inline `classifier_json` (tenant-safe); `feature_schema_hash` is the MINOR-vs-MAJOR upgrade switch.
- **v6** ? *pending.* `contribution_consent` ledger (default-deny; minors-exclusion enforced **at the pooling query**, not advisory).

3. Constructs (the gap list + what is built)

#	Construct	Status
1	Per-user store + serving bridge ? **store + load half built** (v5 `user_models` + `serving.py::load_user_classifier`); only wiring `_select_for_handoff` remains	Phase 1 (the #1 wall ? load half done, wiring pending)
2	Correction-loop ? per-user labels (`human_label` setter + confirm/reject API; recall-protecting de-bias prior)	Phase 2 (`human_label` is a read-only stub today)
3	**Per-user promotion gate** (min-N floor + structural-guard exemption)	? **DONE ? PR #141** (`backend/eval/per_user_gate.py`)
4	Contribution flywheel (opt-in data/video/diagnostics ? shared baseline; payoff ? contribution; consent + minors-exclusion)	Phase 2
5	User-as-annotator charter (rally-annotator: auto-generate a ~2-min high-uncertainty segment from the user's own video; guide labeling ? grounding)	Phase 3
6a	**Held-out-USER learning curve** (the falsifiable "<K videos to superior quality" criterion)	? **DONE ? PR #142** (`backend/eval/per_user_eval.py`)
6b	Per-user no-regression guard (frozen per-user CONTROL; never accept a regressing retrain/upgrade)	? **in flight** (`feat/per-user-no-regression`)
7	Day-0 "batteries" = the validated label-free Gate-A/B fusion (?F1 +0.074, 6/6, p=0.031) shipped as the default	Phase 0, owner-gated (§5)

4. Phased PR-sized build plan + status

Phase 0 ? safety + day-0 batteries (no schema / API / served-behavior change; pure-Python + tests):
- PR-0.1 per-user promotion gate ? ? **#141 merged**
- PR-0.2 held-out-USER learning curve ? ? **#142 merged**
- PR-0.3 per-user no-regression guard ? ? in flight
- PR-0.4 ship Gate-A as the day-0 battery ? **owner-gated** (§5; needs a golden-set regression run)
- PR-0.5 within-user LOVO measurement (the "confirm-or-kill" experiment) ? **owner-gated** (needs multi-video-per-user data on the GPU box)

Phase 1 ? per-user tuning serving + store: ? already landed ? v4 migration (nullable `user_id`), v5 `user_models` + CRUD (`upsert_user_model` / `get_active_user_model`), and `serving.py` (`resolve_user_fusion_config` overlay resolver + `load_user_classifier`). **Remaining:** `set_user_id` (and the Phase-2 `set_human_label` writer); **wire the bridge into `_select_for_handoff` (owner-gated ? served path)**; `/retune` + `/model` trust-feature endpoints.

Phase 2 ? correction loop + flywheel: `POST /api/candidates/{id}/label` (the first `human_label` writer); recall-protecting de-bias prior; v6 consent ledger + minors-exclusion; **opt-in contribution upload (owner-gated ? privacy surface)**; free-tier pooling.

Phase 3 ? annotator charter + taxonomy + baseline upgrades: active-learning task picker; rally-annotator CSV ingest ? per-user labels; **coarse hard/soft shot proxy (owner-gated)**; **baseline-upgrade governance (MINOR carry / MAJOR re-fit on `feature_schema_hash`) + optional

federated nudge
(owner-gated, counsel).**

5. Open decisions (owner)

1. **Structural-guard scope + `min_n` value** ? lock **Gate-A-only structural, `min_n = 5`** (recommended)?
2. **Gate-A default flip (PR-0.4)** ? Gate-A lives in the non-default `FusionSegmenter`, so "ship as
   default" is a path choice (make that segmenter the default vs port the gate into the default path).
   Recommended sequence: **run a golden-set regression first**, bring the number, *then* approve.
3. **Per-user `user_id` for the alpha** ? device/install UUID acceptable, or must auth land first?
4. **Local-install flywheel default** ? fully-local zero-pooling until consent is explicitly granted?

6. Honest risks (carried from the analysis)

- **Small-N per-user eval noise** is central and *worse* per-user ? mitigated structurally: the gate
  refuses below `min_n` and falls back to the baseline; never auto-promotes in the regime the tooling
  exists to refuse in. PR-0.5 confirms-or-kills the core premise before any serving build-out.
- **Cold-start** ? the day-0 Gate-A battery (zero per-user labels) is the floor for the median user.
- **Privacy deferral** ? do not advertise "data never leaves your machine" until on-device ships;
  near-term pooling carries the full cloud (DPA / consent / minors) posture.
- **Compute** ? heavyweight VLM personalization (on-device Qwen3-VL fine-tune) is **out of v1 scope**;
  v1 is the cheap LogReg / threshold / cue-ON-OFF layer only.

7. References
- `scratch/PERSONALIZATION_PIVOT_ANALYSIS.md` ? the decision-grade strategy analysis (pros/cons, harness-fit, counter-case, recommendation).
- `scratch/PERSONALIZATION_FEATURE_DESIGN.md` ? the code-grounded design (schema, serving bridge, APIs, critical files).
- Built: `backend/eval/per_user_gate.py` (#141), `backend/eval/per_user_eval.py` (#142).
- Harness: EXPERIMENT_HARNESS.md, `backend/eval/experiment.py`, `backend/eval/eval_stats.py`.

16. user (2026-06-16T03:12:22.968Z)

Roora Annotation Guidelines ? Rally Golden Set

Version: v8 (2026-06-13). Supersedes the older Drive copies (v3/v5/v6/v7) **and** the internal
"DRAFT v2" label ? those numbers never matched each other; **v8 is the single label to use with
Roora going forward.** Content is otherwise the finalized 2026-06-12 guideline plus the
fusion-driven additions marked **`? ADDED (fusion)`** below.

> **?? Canonical home / provenance.** **This in-repo file is the sole canonical copy of the guideline**
> ? it lives here because the multi-signal-fusion features that motivate the new fields (court
> calibration, singles/doubles) originate in *this* repo (see
> MULTI_SIGNAL_FUSION_PLAN.md and
> GOLDEN_SET_VENDORING.md, ADR-006). The annotation deliverable still imports
> into the sibling **`sports-data-collector`** repo via `python -m src.cli.curate

import-local`; if a
> mirror is ever staged there for the collector's intake docs, it must be regenerated from this file
> (and the `? ADDED (fusion)` callouts are exactly the deltas to port).
>
> **`TODO(avidullu)` markers are preserved** ? they are owner-only items (Drive links, intake
> manifest, NDA counsel review) and must be filled before sending to Roora. Search for `TODO(avidullu)`.

Brief (vendor spec) + Illustrated Guide (rater how-to), in one doc.

PART A ? Roora Annotation Brief (vendor-facing spec)

Project: racket-sport rally segmentation golden / regression dataset.
Sports in scope: Badminton, Pickleball, Table Tennis (each in singles AND doubles).

This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV that imports directly via `python -m src.cli.curate import-local`.

0. What we need in one sentence

For each provided match video, produce ONE CSV file listing EVERY rally with its start time, end time, number of shots, and how it ended ? EXCLUDING all dead time.

1. Deliverable format (canonical)

One CSV per video (UTF-8), named exactly `<video_id>.csv` (we supply the `video_id`). One row per rally, chronological, header row required.

Columns:

Column	Type / values	Meaning
`rally_number`	integer, 1-based, chronological ? no gaps, no repeats	rally index
`start_mmss`	`m:ss.mmm` (e.g. `1:12.40`)	rater-entered serve-contact time
`end_mmss`	`m:ss.mmm`	rater-entered ball/shuttle-dead time
`start_time`	decimal **seconds** (e.g. `72.40`) ? **AUTO-FILLED** from `start_mmss`	machine time
`end_time`	decimal **seconds** ? AUTO-FILLED from `end_mmss`	machine time
`shots_count`	integer	total racket/paddle/bat contacts (serve = shot 1)
`ending_reason`	`winner` | `forced_error` | `unforced_error` | `service_fault` | `let` | `other`	WHY it ended
`last_shot_outcome`	`in` | `net` | `out` | `n_a`	WHERE the last shot landed (`n_a` for service_fault/let)
`score_before`	optional, `"A-B"` ? blank if not confidently readable	score before
`score_after`	optional, `"A-B"`	score after
`notes`	optional free text ? **REQUIRED when `ending_reason = other`**	edge-case note

Times (mm:ss ? decimal seconds): raters type only `start_mmss` / `end_mmss`. The template Google Sheet fills `start_time` / `end_time` with this formula (shown for `start_time`, row 2):

```
=IF(B2="","", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 + VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
```

On "Download as CSV" the computed decimal seconds export. (If your tool already shows decimal seconds, type those into `start_time` / `end_time` and leave the mm:ss columns blank.)

Example (badminton):

```
rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot_outcome,score_before,score_after,notes
1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no pressure)
```

Critical format rules:
- `start_time` / `end_time` that reach us MUST be decimal seconds (the formula guarantees this).
- Times are relative to the **START OF THE FILE WE PROVIDE**. Do NOT trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum. (The file we share may be an **annotation-optimized copy** of the original ? lighter and faster to frame-step, but frame-for-frame identical. Treat it exactly like the original: annotate it as-is.)
- `end_time > start_time` for every row.
- No overlaps: `end_time[N] <= start_time[N+1]`.
- Everything between one rally's `end_time` and the next rally's `start_time` is DEAD TIME and gets NO row.

2. The three non-negotiable rules (full detail + pictures in the Guide)

1. **IGNORE ALL DEAD TIME.** Only label active play.
2. **INCLUDE EVERY RALLY ? EXCLUDE NONE.** Every served point, including service faults, 1?2 shot rallies, and lets/replays. A missed rally is the worst error in this project.
   - **EXCEPTION:** warm-up / knock-up is NOT a rally ? pre-match and between-games warm-up looks like rallying but isn't scored play, so don't label it. Tell it apart by: no real serve; cooperative (not competitive) hitting; the score not progressing; often several shuttles/balls in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag it.
   - **SECOND EXCEPTION:** a rally already in progress when the file starts (or still in progress when it ends) gets NO row ? its true start (or end) is outside the footage and must never be guessed or extrapolated. Note it in the delivery instead.
3. **A SHOT = ONE CONTACT** with the racket/paddle/bat. Serve = shot 1. Bounces and net-clips are not shots.

3. Scope & phasing

- **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table tennis), each < 3 min with ~10?12 rallies. This is the paid calibration round; we review jointly and lock the Guide before scale-up.
- **Phase B ? Scale (after a successful pilot):** ~12?15 videos PER SPORT (~36?45 total), each ~30 min. Same format, rules, and template.

Doubles are included across the program. Rally rules are identical to singles; the badminton- and pickleball-specific doubles notes are in the Guide.

4. Quality bar (acceptance criteria)

Automated validation (no overlaps, positive durations, schema-valid) runs on every file, then a ~20% human spot-check. A file is returned for rework (in scope) if it misses:

- **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
- **Boundaries:** `start_time` / `end_time` within **±0.3 s** of true serve-contact / landing.
  - ?? Boundaries that all sit on a frame-sampling grid (e.g. every 30th frame) STRUCTURALLY FAIL this bar and are detected automatically ? the whole file is returned for rework. See §7 for the required method.
- **`shots_count`:** exact for rallies ? 15 shots; ±1 allowed for longer/faster rallies.
- **`ending_reason`:** one of the six allowed values, per the Guide's decision tree.
- **Format:** decimal seconds present, valid columns, non-overlapping, chronological.

> **? ADDED (fusion) ? court calibration as a day-1 quality lever.** The owner's pipeline produced a bad highlight where a held shuttle was mislabeled as a rally. The boundary definition (§ Part 3: start = serve **contact**, not the toss/hold) already excludes that ? so faithful boundaries are themselves a quality checkpoint. The one NEW field we need (next section) is **court calibration**, which lets the detector reason about court occupancy and net-crossings.

A. Court calibration & intake manifest ? ADDED (fusion)

> **? ADDED (fusion).** *Why:* the multi-signal detector (MULTI_SIGNAL_FUSION_PLAN.md) needs to know **where the court is** and **where the net is** to (a) ignore people outside the court, (b) require ?2 players on court for a rally, and (c) count shuttle net-crossings. This is a **one-time, per-video** annotation ? cheap, high-leverage, and **required in deliverables from day 1.**

Per-video, recorded once in the intake manifest (NOT in the per-rally CSV):

Manifest column	Type	Meaning
`video_id`	string	(existing) we supply it
`fps`	number	(existing) we supply it
`court_corners`	4 points `(x,y)` in pixels, clockwise from the near-left corner	the playable court rectangle
`net_line`	2 endpoints `(x,y)` in pixels	the net, left post ? right post
`singles_or_doubles`	`singles` | `doubles`	sets the expected on-court player count (2 vs 4)

- Pick **one clear frame** with the empty/visible court and click the 4 corners + 2 net endpoints.
- If the camera does not move for the whole video (the normal case), one calibration covers the file. If the camera is moved mid-file, flag it ? we'll split the file.
- Worked example (badminton, 1920×1080): `court_corners=[(360,980),(1560,980),(1330,300),(590,300)]`, `net_line=[(470,640),(1450,640)]`, `singles_or_doubles=singles`.

(After the first batch we may add more per-shot fields ? see the §B appendix ? and renegotiate price.)

5. Per-sport quick reference (full version + examples in the Guide)

- **Badminton (singles & doubles)** ? start: racket?shuttle serve contact · end: shuttle floors / fault · shot: each racket?shuttle contact.
- **Pickleball (singles & doubles)** ? start: paddle?ball serve contact · end: double bounce / out / net / fault · shot: each paddle?ball contact (floor bounces are NOT shots; two-bounce rule is normal play).
- **Table tennis** ? start: racket?ball serve strike (not the toss) · end: failed legal return · shot: each racket?ball contact (table bounces are NOT shots). Net-cord serve = let.

6. `ending_reason` values (decision tree in the Guide)

This vocabulary attributes WHY the point ended; it is shared across all net-separated sports (badminton, pickleball, table tennis, tennis). Record WHERE the last shot physically landed in the separate `last_shot_outcome` column (`in` | `net` | `out` | `n_a`) ? e.g. a forced error that flew long is `ending_reason=forced_error, last_shot_outcome=out`.

- `winner` ? last shot landed IN and the opponent made no legal contact (a clean winner).
- `forced_error` ? the losing player erred (netted / out / missed) UNDER PRESSURE ? stretched, wrong-footed, or rushed by the opponent's strong shot.
- `unforced_error` ? the losing player erred on a ROUTINE, UNPRESSURED ball they had time and position to make.
- `service_fault` ? the serve itself failed / was illegal.
- `let` ? the point was replayed.
- `other` ? anything else / cannot tell. MUST add a note.

7. Capture method (required for boundary frames)

Use any tool that shows current time (`mm:ss.mmm` or seconds, or frame + known fps) and can step one frame forward/back: a frame-steppable player (VLC frame-step) or a timeline tool (Label Studio / CVAT). The deliverable is the CSV in the schema above ? or, if you work in CVAT, the CVAT export as-is (we recompute each rally's time from its box frame range; seconds = `frame_number / fps`, fps provided per video).

Boundary placement is the non-negotiable part:
- Set up the CVAT task at **FRAME STEP 1** on the file we provide (it is encoded to make this fast ? frame-stepping and back-scrubbing stay snappy).
- The FIRST box of every rally goes on the EXACT serve-contact frame; the LAST box on the EXACT shuttle/ball-dead frame. Frame-step to find them.
- Boxes IN BETWEEN may stay sparse (e.g. every 30th frame) ? only the two boundary frames must be exact.
- Self-check before submitting: every start/end within ±0.3 s of the true frame (Guide).

8. Logistics & contacts

- **TODO(avidullu):** paste the Google Drive folder link where source videos are shared with Roora.
- Per-video fps + `video_id` (+ the calibration columns from §A): provided in the intake manifest (template `INTAKE_MANIFEST_TEMPLATE.csv`). **TODO(avidullu):** fill the manifest (one row per video)

  and share its link.
- **TODO(avidullu):** paste the Drive folder link where Roora uploads completed `<video_id>.csv` files.
- Blank annotation template Sheet (mm:ss ? seconds formula from §1 pre-filled):
  TODO(avidullu): create it and paste the link.
- **TODO(avidullu):** add the escalation channel (email / WhatsApp group) for edge-case questions.
- NDA before any footage is shared ? template `NDA_TEMPLATE.md`. **TODO(avidullu):** have counsel
  review, fill the blanks, and have Roora sign.
- Roora contact on file: info@rooragrp.com (HSR Layout, Bangalore).

> **? ADDED (fusion) ? data-handling guardrails (put in the brief / contract):**
> - **Exclude minors entirely.** If footage contains children, that video does **not** enter the
>   labeled pool ? do not annotate it; flag it back to us.
> - **No biometric / identity labeling.** Refer to players only by **per-video pseudonyms** (e.g.
>   "near side" / "far side") ? never a real name, and never a cross-video person ID. We do **not**
>   collect gait or biometric annotations.
> - **No pose / skeleton / keypoint annotation** is requested.
> - **Work-for-hire IP assignment** (atop the NDA): the labels Roora produces are assigned to us as
>   owned work product.

9. Pricing request (please quote)

1. Price per rally and per minute of footage.
2. Pilot price (3 short videos, ~30?36 rallies) as a paid calibration round.
3. Indicative scale price for ~36?45 longer videos.
4. Turnaround for the pilot and throughput (rallies/day) at scale.
5. QA process and rework policy.
6. Whether annotators have racket-sport familiarity (helpful, not required).
7. Confirmation you can deliver the CSV schema above **(incl. the §A intake-manifest calibration columns).**

PART B ? Illustrated Annotation Guide (rater how-to)

Sports: Badminton, Pickleball, Table Tennis (singles AND doubles).

> **Illustrations:** boxes marked `[SCREENSHOT: ...]` are placeholders.
> **TODO(avidullu):** replace every `[SCREENSHOT: ...]` placeholder with a real frame grab from the
> pilot videos before sending this guide to Roora.

Part 1 ? What you are producing

One CSV per video (from the template Sheet), one row per rally:
`rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes`.

You type: `rally_number`, `start_mmss`, `end_mmss`, `shots_count`, `ending_reason`, `last_shot_outcome`
(+ optional score/notes). The `start_time` / `end_time` decimal-second columns FILL THEMSELVES via a
formula already in the template.

For every rally you record:
- When it started (serve contact) ? `start_mmss` (`m:ss.mmm`, e.g. `1:12.40`)
- When it ended (shuttle/ball dead) ? `end_mmss`
- How many shots happened ? `shots_count`
- Why it ended ? `ending_reason` · Where the last shot landed ? `last_shot_outcome`

Everything else (gaps between rallies) you IGNORE.

Part 2 ? The three golden rules

RULE 1: IGNORE ALL DEAD TIME. A rally is CONTINUOUS active play. The moment play stops, the rally
is over. The time until the next serve is DEAD TIME and is never labeled.

```
  DEAD        RALLY 1                 DEAD             RALLY 2            DEAD
 (ignore) |==================|     (ignore)     |==============|     (ignore)
          ^                  ^                  ^              ^
        serve              shuttle            serve         shuttle
        contact            lands             contact        lands
        =start_1           =end_1            =start_2        =end_2
```

Dead time includes: walking back to serve, picking up the shuttle/ball, wiping sweat, bouncing the
ball before serving, the score being announced, replays/slow-mo, crowd or coach shots, changeovers,
towel/medical timeouts, and PRE-MATCH / BETWEEN-GAMES WARM-UP (knock-up) ? see the Rule 2 caveat below.

RULE 2: INCLUDE EVERY RALLY ? EVEN THE UGLY ONES. Every served point gets a row, including:
- Service faults (serve into net/out/illegal) ? usually `shots_count = 1`, `ending_reason = service_fault`.
- Very short rallies (serve + one error).
- Lets / replays ? `ending_reason = let`; the replayed point is its OWN next row.

A missed rally is the worst error in this project ? the model must learn that short and failed rallies
are still rallies.

> ?? **BUT WARM-UP / KNOCK-UP IS NOT A RALLY** ? a common trap, because Rule 2 says "include
> everything." Pre-match and between-games warm-up LOOKS like rallying but is not scored play; do not
> label it. How to tell it apart:
> - **No real serve.** Real play starts with a proper serve from the service position (diagonal,
>   behind the service line). Warm-up has players feeding cooperatively from mid-court with no serve.
> - **Cooperative, not competitive.** Gentle hits straight to each other; nobody is trying to win the
>   point (no smashes-to-win, no scrambling/diving).
> - **Score isn't progressing.** No umpire / scoreboard not running / players casually retrieve and
>   re-feed; the score doesn't change. Often several shuttles/balls in use at once.
> - **Per sport:** badminton & pickleball ? real play starts with the underhand serve from the service
>   court / behind the baseline; table tennis ? real play starts with the tossed, two-bounce serve.
>
> When unsure: find the first proper serve where the score starts to progress ? labeling begins there.
> If you genuinely can't tell whether the match has started, flag the clip to us ? don't guess.

RULE 3: A SHOT = ONE CONTACT WITH THE RACKET / PADDLE / BAT. Count every time the shuttle/ball
touches a racket, paddle, or bat. The serve is shot 1.
- Count: any racket/paddle/bat contact (serve, return, smash, block?).

- Don't count: floor bounces, table bounces, net clips, the toss before a serve.

5-shot example: serve(1) ? return(2) ? hit(3) ? hit(4) ? winner(5) ? `shots_count = 5`,
`ending_reason = winner`.

Part 3 ? Where exactly does a rally start and end?

START (`start_mmss`): the exact frame the server's racket/paddle/bat CONTACTS the shuttle/ball.
Not the toss, not the wind-up ? the contact frame. `[SCREENSHOT: serve-contact frame]`

> **? NOTE (fusion):** this is the field that fixes the "held-shuttle" false positive ? a player
> holding/flicking the shuttle before serving is **dead time**, so it never gets a row. Faithful
> serve-**contact** boundaries are exactly what the detector is graded against.

END (`end_mmss`): the exact frame the shuttle/ball becomes DEAD:
- Badminton: shuttle touches the floor, or play stops on a fault.
- Pickleball: the second floor bounce, or the frame the ball hits the net / lands out.
- Table tennis: the second bounce on one side, a non-serve net hit, or off the table.

`[SCREENSHOT: end frame]`

If occluded/off-screen, pick the closest clear frame and add a note. **Boundary tolerance: ±0.3 s
(~±9 frames at 30 fps).**

Part 4 ? `ending_reason`: how to choose

Pick exactly ONE value for how the rally ended. Stop at the first match:
1. Did the SERVE itself fail / was illegal (and ended the point)? ? `service_fault`
2. Was the point REPLAYED (let / interruption)? ? `let`
3. Did the final shot land IN and the opponent make NO legal contact? ? `winner` (a clean winner)
4. Did the losing player make an error (netted / out / missed) UNDER PRESSURE ? stretched,
   wrong-footed, or rushed by a strong shot? ? `forced_error`
5. Did the losing player make an error on a ROUTINE, UNPRESSURED ball they had the time and position
   to make? ? `unforced_error`
6. None of the above / cannot tell ? `other` (add a note)

FORCED vs UNFORCED ? the one judgment call. Decide using the opponent's PRECEDING shot:
- `forced_error` = the opponent's shot CAUSED the miss (pace, placement, deception; the player barely
  reached it or was pulled out of position).
- `unforced_error` = a clean, makeable ball the player missed on their own.
- When genuinely unsure between the two, choose `unforced_error` and add a brief note.

Notes:
- A clean winner means NO legal contact by the opponent. If they got a racket/paddle/bat on it but
  netted or hit out, that is a `forced_error` / `unforced_error` (their shot), not a winner.
- A serve into the net is `service_fault` (rule 1 beats everything).

THEN SET `last_shot_outcome` ? where the last shot physically went. This is a SEPARATE,
observational column (no judgment): just look at the last shot.
- `in` ? landed in (winners; also a put-away the opponent couldn't reach).
- `net` ? went into the net.
- `out` ? landed outside the court / past the lines.

- `n_a` ? not applicable: use for `service_fault` and `let`.

So each rally gets both: WHY (`ending_reason`) and WHERE (`last_shot_outcome`) ? e.g. a smash winner = `winner, in`; a clear forced long = `forced_error, out`; a routine ball netted = `unforced_error, net`.

Part 5 ? Per-sport detail

Badminton
- **Serve:** underhand, below the waist, diagonal. Start = racket?shuttle contact.
- **Rally end:** shuttle hits the floor, or play stops on a fault.
- **Shots:** every racket?shuttle contact; serve = shot 1.
- **Service faults** (1-shot rally, `service_fault`): serve into net, serve outside service court, contact above waist, foot fault.

Worked example: `[SCREENSHOT: badminton serve]`
- Rally 4: serve `1:28.30`, long rally, smash winner, shuttle lands `1:44.10`, 12 contacts.
  `4, 1:28.30, 1:44.10, (auto), (auto), 12, winner, in, 3-5, 4-5,`
- Rally 5: serve `1:57.00` straight into the net.
  `5, 1:57.00, 1:57.90, (auto), (auto), 1, service_fault, n_a, 4-5, 4-6,`

Badminton ? doubles (2 per side). All rally rules are IDENTICAL to singles:
- start = serve contact; end = shuttle down / fault; a SHOT = any racket?shuttle contact by EITHER player on a side.
- `shots_count` = TOTAL contacts across both teams in the rally. Do not track which player hit ? just count every contact.
- Net exchanges (drives, flat pushes) come fast ? frame-step so you neither miss nor double-count.
- Only the designated receiver returns the serve; if the wrong player returns, the rally ends on a fault ? still a row (`service_fault` if it's a serve/receiving-position fault, else `other` + note).
- Two teammates may NOT both hit the shuttle in succession (double-hit). Two contacts on the SAME side back-to-back = a fault ending the rally ? count both contacts, then the rally ends.

Pickleball
- **Serve:** underhand, paddle below waist, hit diagonally; must clear the non-volley zone (the "kitchen"). Start = paddle?ball contact.
- **Two-bounce rule:** the serve must bounce once in the receiver's court and the return must bounce too, before either side may volley. These bounces are NORMAL play, not rally ends, and bounces are NOT shots.
- **Rally end:** ball bounces twice, lands out, hits the net (fails to cross), or a fault (e.g. volleying in the kitchen).
- **Shots:** every paddle?ball contact only; serve = shot 1; floor bounces are not shots.
- **Service faults** (`service_fault`): serve into net, out/long, short in the kitchen, foot fault.

Worked example: `[SCREENSHOT: pickleball serve + kitchen line]`
- Rally 2: serve `0:33.10`; serve bounces, return bounces, a few volleys; receiver nets an easy one at `0:41.85`; 6 paddle contacts.
  `2, 0:33.10, 0:41.85, (auto), (auto), 6, unforced_error, net, 1-0, 2-0, easy ball netted`

Pickleball ? doubles (the standard format, 2 per side). All rally rules IDENTICAL to singles:
- start = serve contact; a SHOT = any paddle?ball contact by EITHER teammate; floor bounces still not shots; two-bounce rule still applies.
- `shots_count` = TOTAL paddle contacts across both teams; don't track which player.
- Serving sequence (FYI only ? you do NOT annotate it): both partners serve before the serve passes to the opponents, except the very first service turn of the game. The called score has three numbers. Ignore for annotation; just mark boundaries / shots / ending.
- If both teammates contact the ball in succession, that's a fault ? rally ends; count the contacts, set the ending per the fault.

Table tennis
- **Serve:** ball tossed up from an open palm, then struck so it bounces once on each side. Start = the racket?ball strike (not the toss).
- **Rally end:** a player fails a legal return ? ball double-bounces on their side, goes off the table, or is hit into the net on a non-serve shot.
- **Shots:** every racket?ball contact only; serve = shot 1; table bounces are not shots.
- **Let:** a serve that clips the net but is otherwise legal ? the point is replayed. `ending_reason = let`, and the replay is its own next row.
- **Service faults** (`service_fault`): missed/illegal toss, serve into net, serve off the table.
- **Speed warning:** contacts can be < 0.2 s apart ? step frame-by-frame to count shots.
- **Doubles** (if a clip appears): partners must hit in STRICT ALTERNATION and serve diagonally; a missed alternation is a fault that ends the rally. Same rule ? count every racket?ball contact as a shot; `shots_count` is the team total.

Worked example: `[SCREENSHOT: table-tennis serve contact]`
- Rally 9: serve `4:10.40`; fast 7-contact rally; forehand winner, opponent no contact; dead `4:14.05`.
  `9, 4:10.40, 4:14.05, (auto), (auto), 7, winner, in, 6-9, 6-10,`
- Rally 10: serve `4:22.10` clips the net (otherwise good) ? let.
  `10, 4:22.10, 4:23.00, (auto), (auto), 1, let, n_a, 6-10, 6-10, net-cord serve, replayed`

Part 6 ? Edge cases & FAQ

- **Two rallies with almost no gap?** Still two rows. `end_time[N] <= start_time[N+1]`, never overlap.
- **Interrupted & replayed?** Label the interrupted attempt as its own row (`ending_reason=let`), then the replayed point as the next row.
- **Serve tossed but caught / re-served (no contact)?** No contact = no rally yet. Start at the actual serve contact; don't label the aborted toss.
- **Opponent reached the ball but netted/hit out?** That's a `forced_error`/`unforced_error` (their shot), not winner ? forced if your shot pressured them, unforced if it was routine.
- **Can't read the scoreboard?** Leave `score_before`/`score_after` blank. Never guess.
- **"Let" where the rules say play on?** If play continued, it's NOT a let ? keep one rally and judge the real ending.
- **Warm-up / knock-up / practice points?** Not real rallies ? do not label them. Start at the first real, scored serve. See the Rule 2 caveat (Part 2). When unsure whether the match has started, flag the clip ? don't guess.

- **Doubles?** Same rules ? a shot is any one racket/paddle contact by whichever player hits it. Never
  attribute `shots_count` to an individual, just total the contacts.
- **Server completely whiffs (no contact)?** Rare. True zero contact ? `shots_count = 0`,
  `ending_reason = service_fault`, add a note. A graze = 1 contact = 1 shot.

Part 7 ? Self-check before you submit (per video)

- [ ] `rally_number` is 1,2,3? no gaps or repeats.
- [ ] Every `start_time < end_time` (check the auto-filled decimal columns populated).
- [ ] No overlaps: `end_time[N] <= start_time[N+1]`.
- [ ] Times entered as `m:ss.mmm`; decimal `start_time`/`end_time` auto-filled and look right.
- [ ] No dead-time rows (scan the gaps ? dead time, not missing rallies).
- [ ] Every served point present, including service faults and lets.
- [ ] `shots_count` looks right (re-step any rally you rushed; doubles = team total).
- [ ] `ending_reason` in {winner, forced_error, unforced_error, service_fault, let, other}; notes
  filled when `other`.
- [ ] `last_shot_outcome` in {in, net, out, n_a} (`n_a` for service_fault/let).
- [ ] File downloaded as `<video_id>.csv`, header row present, UTF-8.
- [ ] Did NOT trim / re-encode / rename the source video.
- [ ] **? (fusion)** intake-manifest row filled: `court_corners`, `net_line`, `singles_or_doubles`.

Part 8 ? Mini glossary

- **Rally** ? one continuous passage of play from serve contact to the shuttle/ball dead.
- **Shot** ? one contact between the shuttle/ball and a racket/paddle/bat (either teammate in doubles).
- **Dead time** ? any time the shuttle/ball is not in active play. Never labeled.
- **Service fault** ? a failed/illegal serve that ends the point. Still a labeled rally.
- **Let** ? a replayed point. Labeled as its own rally with `ending_reason=let`.
- **Winner / forced_error / unforced_error** ? the final shot was a clean winner (opponent made no
  legal contact) / the loser's error was forced by a pressuring shot / the loser's error was on a
  routine, unpressured ball.
- **start_time / end_time** ? decimal seconds from the start of the provided file (auto-filled from mm:ss).
- **? court_corners / net_line / singles_or_doubles** ? per-video calibration in the intake manifest (§A).

Appendix B ? Tier-B extensions (NOT in the day-1 deliverable)   ? ADDED (fusion)

> Pilot these on the 3-video calibration round, then renegotiate price before scaling. **Do not start
> these without explicit instruction** ? they are listed so Roora can quote them.
>
> - **Per-shot hit timestamps** ? a timestamp for *each* racket/paddle/bat contact (not just the
>   `shots_count` total). Extends the §7 CVAT method (box every contact, not only the two boundary
>   frames). Trains/validates the audio hit-detector and shot-cadence features.
> - **Per-shot type** ? for each contact, a label from {serve, clear, drop, smash, net, drive, lift}.
>   Needs its own short guideline + an inter-annotator agreement check.

17. assistant (2026-06-16T03:12:25.948Z)

Now I'll read the archives/quality-iterations directory to see what's there:

18. user (2026-06-16T03:12:26.277Z)

1 # quality-iterations/
2
3 This directory holds completed phases of quality improvement work.
4
5 As we receive new labeled videos and iterate on model accuracy, recall/precision, audio fusion, candidate quality, etc., each major iteration or batch of improvements will be captured here once it reaches a terminal state (DONE / COMPLETED / REJECTED).
6
7 See `../README.md` (under archives/) for the overall archival policy.
8
9 ## Organization
10 - Dated subdirectories, e.g. `2026-06-initial/`
11 - Each iteration should include:
12 - A README.md with STATUS, trigger (new data, specific goals), key changes, and links to relevant checkpoints under `../checkpoints/`.
13 - References to updated eval baselines, regression results, report enhancements, etc.
14
15 This keeps the live `docs/` clean while preserving a rich historical trail for regression detection across quality work.
16
17 ## Iterations
18 - **`2026-06-initial/`** ? SEED. Opened the quality phase ahead of the awaited labeled-video batch (status: in progress; superseded for first results by the iteration below).
19 - **`2026-06-human-gt-tuning/`** ? **COMPLETED (2026-06-08).** First eval against **human** ground truth on a partially-labeled video: built the partial-label eval harness + labeled-window methodology, produced the first trustworthy F1 (baseline 0.650 ? tuned 0.742, *fit-on-self*), per-rally failure analysis, a config sweep, and ? via adversarial review ? found + fixed a real ingestion bug (negative-start import abort). Cross-repo: `sports-data-collector` PR #13.
20 - **`2026-06-multivideo-lovo/`** ? **COMPLETED (2026-06-08 ? extended to n=6, 2026-06-09).** Built a **6-match** human corpus (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court; 3 cameras/frame-rates) + cross-video **leave-one-video-out**. Key finding: the unsupervised windowing heuristic is **footage-fragile** ? it fails on **multi-court** footage (background games; Kushagra 0.16 / gBaaddy 0.28) where the shuttle signal is present (97% of rallies have motion) but rally-vs-dead-time isn't velocity-separable. **Prototyped a learned per-frame segmenter** (`backend/eval/rally_seq_proto.py`): held-out AUC **0.83?0.92 on every match, decoupled from the multi-court confound**. **The learned model OVERTOOK the heuristic at n=3 and holds ~+0.10**: n=6 LOVO mean **learned 0.583 vs heuristic 0.480**. The **"contrast" metric** (in-rally÷dead-time shuttle visibility) predicts heuristic F1 monotonically across 5 matches. Data-backed green light for `../../research/OWNED_MODEL_TRAINING_STUDY.md`.
21 - **`2026-06-first-ab-experiment/`** ? **COMPLETED (2026-06-14).** The **first end-to-end A/B** on the n=6 golden corpus through the experiment harness + the C1?C4 measurement-honesty helpers, run **100% offline** (re-scored persisted WASB trajectories, no GPU). One consistent candidate set re-scored three ways: **Gate-A (net-crossings ? 2) is a real win** ? ?F1 +0.074, **6/6 matches, sign-test p = 0.031, CI [+0.042, +0.104]**, over-segmentation (segment-count ratio) **1.68 ? 1.01**; the **window-level 5-feature logistic does NOT clear the bar** (?0.081, 2W/4L, a threshold/calibration artifact ? distinct from the stronger per-frame prototype above). The headline is methodological: **C1 confidently promoted one variant and refused another**, and **C4 surfaced over-segmentation tIoU hid** ? the framework held up on first real use.
22

19. assistant (2026-06-16T03:13:38.777Z)

Perfect. Now let me compile the comprehensive survey. I have enough information to provide a detailed breakdown.

SURVEY: Rally-Detection Design Docs ? Status, Key Decisions, Measured Results

Document Index & Status

Document	Status	Coverage	Key Decisions
README.md	Living index	Documentation roadmap, cross-references	Indexes all active docs; hardening loop marked COMPLETED (2026-06-14)
CODE_MAP.md	Living reference	Repo structure, contracts, file placement	Canonical source for file organization; 2026-06-10 audit sweep added ~8 behavioral changes + 8 new tests
EXPERIMENT_HARNESS.md	**P1 SHIPPED** (2026-06-13)	A/B + shadow eval, LOVO-honest measurement, promotion gate	`Variant` abstraction, `compare()`, `compare_unlabeled()`, experiment leaderboard; **zero-regression by design** (cues default OFF, promote only on measured lift)
MULTI_SIGNAL_FUSION_PLAN.md	**P1 DONE** (2026-06-13)	Shuttle + person + audio, one fusion layer	Person cue extracted to reusable `PersonDetector`; P2 `FusionSegmenter` shipped (default-OFF); **licensing all MIT/Apache/BSD**
ANALYZERS_AND_RECONCILERS.md	**Design ? owner-gated, NOT started**	Signal-fusion framework; frame/window/session scopes; no special-casing spine	Multi-agent literature review (§13) completed; identifies standard vs. novel patterns; course corrections C1?C13 documented
HOW_RALLY_DETECTION_WORKS.md	Living conceptual map	2-layer model (detection vs. segmentation)	Clarifies shuttle vs. person cues; open-question list includes fusion, per-video calibration, learned segmenter
QUALITY_ITERATIONS.md	**Living log**	Reverse-chronological quality moves, measured effects	Baseline table (heuristic 0.480 ? Gen-0 0.626 ? Gemini 0.93); lessons carry-forward (small-N discipline, LOVO, learned model needed)
OWNED_MODEL_IMPLEMENTATION_PLAN.md	**Phase-0 PLAN**	Roadmap to owned-model + multisport	Framework = ms-swift (Apache-2.0); base = Qwen3-VL (Apache-2.0); **START WITH TAL HEAD, NOT VLM**; licensing constraints (MIT/Apache/BSD only; no paid-LLM training)
NEXT_STEPS.md	**Living**	Prioritized next moves, pointers	M0 in-container quick wins COMPLETE; GPU-box P3 measurement next; corpus growth = single highest-leverage
ROORA_LABELING_GUIDELINES.md	**v8 (2026-06-13)**	Vendor annotation spec + rater guide	Court calibration, singles/doubles, intake manifest; 3 non-negotiables (ignore dead time, include every rally, one shot = one contact)
PERSONALIZATION_PLAN.md	**Design ? HYBRID adopted**	Per-user tuning on generalization baseline	Owner decisions locked (2026-06-14): identity (design north-star, implement local), `min_n` = 5 (promotion floor, not service gate), Gate-A-only structural guard

---

CRUCIAL: §13 Literature Review (ANALYZERS_AND_RECONCILERS.md)

What the multi-agent literature review surveyed and the verdicts:

Patterns Surveyed (§13.1, Verdict Table)

Pattern (our framing)	Field's term	Closest prior art	Verdict
Candidate-windows + scorer + reconciler	Temporal Action Localization/Segmentation (TAL/TAS) + boundary refinement	ASRF (arXiv:2007.06866), BMN, Activity Grammars	**partial-overlap**
"Signal not verdict" ? learned arbiter	Late/score-level fusion + stacked generalization	Snoek 2005, Wolpert 1992, CVIU 2009 racket-sports	**partial-overlap**
Detection?decision, persist-once/re-score	Frozen-backbone cached features + decoupled proposal/classification	S-CNN, CBR, Decouple-SSAD	**partial-overlap** (? field default)
Report-first reconciler (contradiction ? audited correction, measured before apply)	Model/consistency assertions, constraint-based repair	Model Assertions (2003.01668), HoloClean, TFDV, EventAnchor	**novel-combination**
Scarce interval labels + tiny LR + LOVO-by-match	Point-supervised TAL + grouped (LOGO/LOSO) CV	SF-Net, LOOCV-bias (arXiv:2406.01652), Snorkel	**partial-overlap**
Multi-scope (frame/window/session); warm-up conditions per-window	Multi-scale/hierarchical modeling + play-break + phase recognition	Tjondronegoro 2003, GL-MSTCN, MS-TCN	**partial-overlap**

Verdict Summary (§13.1 TL;DR)

> *"The architecture's skeleton is standard and must be **cited, not claimed**: candidate-windows + learned scorer = **Temporal Action Localization** (proposal-then-score); reconciler

over-segmentation fixes = **Temporal Action Segmentation** refinement; "signal not verdict / learned arbiter over cue columns" = **late / score-level fusion** realized as **stacked generalization** ? ~20 years old and already present in racket sports. The defensible delta is the engineering/governance envelope: detection?decision persisted as a re-score substrate + a **report-first, A/B-lift-gated, idempotent, human-in-the-loop reconciler-as-linter** keyed on decision-vs-evidence contradiction."*

Only the reconciler-as-linter and warm-up-as-soft-session-feature rated `novel-combination`; the rest are `partial-overlap`.

In-Domain Prior Art Already Hard-Codes These Cues (Our Delta = No-Branch Discipline)

- Non-broadcast badminton rally start/end (See et al. 2024/2025)
- Shuttle-state/hit detection (MonoTrack arXiv:2204.01899; Hsu/Yu/Cheng, Sensors 2024)
- Service-fault (Goh et al., Sensors 2023)
- Tennis in-play state machines
- Badminton point-segmentation (Ghosh et al. 2018)
- ShuttleSet (arXiv:2306.04948)

The delta is not cues ? it's the *no-branch, weight-learned-cues* discipline + the measured-before-apply reconciler.

Prior Measured Results

Quality Baseline Table (QUALITY_ITERATIONS.md, NEXT_STEPS.md)

Path	Mahadevpura (clean)	Kushagra (multi-court)	Notes
Heuristic (velocity windowing)	0.657	0.157	**footage-fragile**; dead-time confusion on multi-court
Gen-0 owned head (LOVO, n=6)	0.744	0.561	LogReg on 5 trajectory features; threshold-in-LOVO needed (C2)
Two-stage WASB+Gemini refine	0.83	?	Cost-efficient for long tapes
Gemini wrapper (gemini-2.5-flash)	**0.93**	**0.66**	**Commoditized path for clean footage; multi-court target for owned model**

Reading: Own model's ship target = **0.66 on multi-court** (the contrast-confound problem); Gen-0 is 0.10 behind on multi-court, **corpus-growth-sized gap, not architecture-sized**.

Gate-A Measurement (QUALITY_ITERATIONS.md, "Acting on the First A/B")

- **?F1 +0.074** (6/6 golden matches, sign-test **p=0.031**, CI [+0.042, +0.104])
- **Over-segmentation: segment-count ratio 1.68 ? 1.01**
- **Real-rally windows lost: 12 windows across 6 videos on served COARSE windowing** (net_crossings ~1.0; the gate's threshold of ?2 is too strict for the served path)
- **Eval ? serve lesson:** the +0.074 is conditional on the eval harness's recall-oriented *proposal* windowing (0.005 velocity, 1.0 merge-gap, 0.5 min-duration); served windowing is coarse (0.02, 2.0, 2.0) and already merges aggressively. **Gate-A flipped to default and then REVERTED** on served-side regression check (**served ?F1 ?0.008, ~12 real rallies dropped**). Now **opt-in** (`indexing.fusion.min_crossings: 2`), default OFF.

Person + Audio Fusion Measurement (QUALITY_ITERATIONS.md, "Fusion P3 + P4 MEASURED")

Variant (LOVO, n=6, IoU ? 0.5)	F1	?F1	95% CI	Sign test	CI clears 0	Status
Shuttle-only (CONTROL)	0.307	?	?	?	?	?
+ person (P3)	0.286	**?0.021**	[?0.165, +0.162]	2W/4L (p=0.688)	**No**	Default-OFF
+ person + audio (P4)	0.366	**+0.079**	[?0.054, +0.203]	4W/2L (p=0.688)	**No**	Default-OFF (promising, not promotable)

Person adds noise (no court polygons ? `players_in_court` counts spectators); **audio is the more promising lead** (+0.079 / +27.7%, wins 4/6 videos), but **n=6 can't confirm it**. Both stay default-OFF (promote-only-on-lift discipline). Recorded in `eval_baselines/experiment_leaderboard.json`.

Personalization Phase-0 (QUALITY_ITERATIONS.md checkpoint entry, 2026-06-14)

Three **honest-eval RAILS** that keep per-user tuning from promoting noise:

1. **Per-user promotion gate** (PR #141): min-N floor (5 videos) + structural-guard exemption (Gate-A stays on)
2. **Held-out-USER learning curve** (PR #142): falsifiable "<K videos to superior quality" metric
3. **Per-user no-regression guard** (PR #143): incumbent CONTROL; candidate never regresses user

---

Stated Eval Approach & Key Lessons

Evaluation Framework (EXPERIMENT_HARNESS.md §3; EVALUATION.md; QUALITY_ITERATIONS.md)

- **LOVO (Leave-One-Video-Out):** score on held-out *video*, never held-out *window* (windows in one video are correlated). Group by **match** (group-aware LOGO/LOSO).
- **IoU threshold:** default 0.5 temporal IoU for matching + scoring.
- **Honest stats (C1?C4, QUALITY_ITERATIONS.md):**
  - **C1:** Bootstrap 95% CIs + paired sign test (never a single mean at n?6); RLOOCV for bias-aware folds
  - **C2:** Calibrate each producer's confidence (Platt/isotonic) before the scorer
  - **C3:** Out-of-fold stacking (learned producer ? leave-one-group-out outputs to the LR)
  - **C4:** Report field metrics (F1@{10,25,50}, edit score, tight/loose mAP) alongside temporal-IoU; tIoU alone is blind to over-segmentation
- **Eval ? serve lesson:** the experiment harness windowing (recall-oriented, permissive candidates) differs from the served path (coarse, few candidates). A harness win doesn't predict served behavior ? align windowing or re-run promotions on served windowing.

"Signals?Learned-Scorer, NO Special-Casing" Spine (ANALYZERS_AND_RECONCILERS.md §1)

> *"Every producer emits a SIGNAL carrying a value + confidence. The SCORING MODEL consumes those signals and learns to weight them. We do NOT add hand-coded `if/else` branches into the decision."*

- The **SVM/LogReg learned scorer** (ADR-004, `backend/eval/classifier.py`) is the only place a decision is made.
- Analyzers propose signals as feature columns; the scorer learns weights.
- Reconcilers run *outside* the decision path as a post-hoc linter (report-first, never silent).
- **Invariant:** all analyzer flags OFF + reconciler report-only ? candidate rows AND predictions byte-identical to control.

No-Regression Guarantees (EXPERIMENT_HARNESS.md §6)

1. **New cues default OFF** ? per-cue enable flag; merged code can't change behavior until explicitly enabled.
2. **Control variant pinned** ? frozen recipe + classifier model hash, always registered & default.
3. **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no silent default change.
4. **Experiments are records** ? variant-spec hash ? per-video + LOVO scores + label-set hash + timestamp, persisted to JSON leaderboard.
5. **Decoupled events** ? "merge an experiment" and "change the default" are separate, independently revertible actions.

Owned-Model Plan: Base Model & Licensing Constraints

Design Principle (OWNED_MODEL_IMPLEMENTATION_PLAN.md §0)

> *"Envision now, build later. Every deferred piece must be architecturally envisioned now so we never hit a mammoth refactor. Design the seams (data contracts, export boundaries, event store) up front and implement behind them when each milestone is greenlit."*

Hard Guardrails (Non-Negotiable)

1. **Commercial-clean licensing for EVERY dependency ? code, data, AND weights: MIT / Apache-2.0 / BSD only.** Reject viral/NC/custom-restrictive (Gemma 3, Llama 4 700M-MAU cap, Commons-Clause).
2. **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public grounding-CoT datasets that were Gemini-generated (e.g. TVG-R1's 56K).
3. **Exclude minors; per-user training data needs explicit opt-in; split data by match.**
4. **? Base-model pretraining provenance is unauditable for *every* open VLM** ? explicit owner risk-acceptance required.

Model Strategy (Verified 2026-06-09)

Component	Choice	Rationale
Framework	**ms-swift (Apache-2.0)**	One forge for video + audio + timestamp-grounding + GRPO/RFT; keep **unsloth** (Apache-2.0) for 8 GB local SFT/compression (on-device goal)
Base (VLM)	**Qwen3-VL (Apache-2.0)**	Long-video context, timestamp emission, multi-turn chat; **InternVL3** (MIT/Apache) = fallback
Gemma-4	AMBER (bench-only)	Apache grant likely but Prohibited-Use unconfirmed; video/timestamp capability disputed
START POINT	**TAL head, NOT VLM**	Fine-tuned VLMs ?50% R@0.7 on open-domain; our head = de-risked AUC 0.83?0.92, trains in minutes, doubles as pseudo-label teacher + honest baseline VLM must beat; **head now ? VLM later**

Conversational QA Architecture (Seam Built Now, Model Deferred)

Punt the conversational *feature* until extraction quality ? "very high." But build the *seam* now:

- **Extract once, query many:** TAL head ? later VLM ? per-video **EVENT STORE** ? **structured QUERY layer** (text-to-SQL + ffmpeg clip-cut)
- **Per-video event-index schema (first-class data contract):** `{video_id(MD5), rally_ordinal, sport, start, end, shot_count, ending_reason/fault_tags, derived_clip_ref, provenance, confidence}`
- **Reserve fault-tag taxonomy now** (e.g. `service_fault`), even if unpopulated
- **Paid Gemini may answer open-ended edge questions at inference** (allowed ? it reasons over our data, not trained on its outputs)

Multisport (Single Sport-Agnostic Model)

- **Model/framework: UNCHANGED** ? one shared TAL head + Qwen3-VL, fine-tuned across sports
- **What multisport changes = DATA + DETECTORS (scale up):** each sport needs labeled matches (split by match) + clean trajectory detector
- **Binding blocker:** clean MIT/Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel not yet identified; WASB has unresolved C3 checkpoint-provenance issue
- **? Single-model-multisport-temporal-generalization is OPEN HYPOTHESIS**, not proven (badminton-only corpus; RacketVision refuted on verification)

Phase Structure & Status

Core Quality Track (Default-OFF, Promote-Only-On-Measured-Lift)

Shipped (PR #112 + 2026-06-13):
- P1: Person cue extracted to reusable `PersonDetector` (PR #115)
- P1: Experiment harness (`compare()`, `compare_unlabeled()`, leaderboard) ? shipped 2026-06-13 ?
- P2: `FusionSegmenter` (Gate A signal + person-feature persistence), default-OFF ? shipped

2026-06-14 ?

In progress / owner-gated:
- P3: Feature-level (learned) fusion ? person features + LOVO lift measurement
- P4: Audio cue integration (hit_detector already scaffolded in ADR-007)

Hardening Loop (COMPLETED 2026-06-14)

- **T1?T5 positive tests** for 5 identified issues + multi-layer regression protocol
- **626-test suite** + ablation/experiment layer + pre/post-change verification checklist
- **Archived:** `docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md` + `docs/archives/HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md`

Critical Cross-Links for New Foundational Research

To avoid duplication, a new foundational doc should:

1. **Cite §13 literature review explicitly** ? don't re-derive the skeleton (TAL/TAS/late-fusion) or in-domain prior art
2. **Anchor on the "no-branch spine"** (ANALYZERS_AND_RECONCILERS.md §1) ? the core governance principle
3. **Reference the eval ? serve lesson** (QUALITY_ITERATIONS.md, Gate-A revert) ? the honest pitfall
4. **Build on the framework's three temporal scopes** (frame/window/session) ? the reusable architecture
5. **Ground course corrections C1?C4 and Tier-2/3** (§13.4?C13) ? the open research debts
6. **Carry the small-N discipline** (few features, logistic regression, LOVO-by-match, not held-out-window)
7. **Inherit the licensed-only, no-paid-LLM-training, minors-excluded guardrails** (OWNED_MODEL_IMPLEMENTATION_PLAN.md, ROORA_LABELING_GUIDELINES.md §8)

This survey is complete and all document/section anchors are canonical (file paths + §X references as they appear in the codebase).